

TFGN:

Task-Free, Replay-Free Continual Pre-Training Without Catastrophic Forgetting at LLM Scale

Anurup Ganguli*
Independent Researcher

April 2026

Abstract

Continually pre-training a large language model on a sequence of heterogeneous text domains, without replay and without task labels, has remained an unsolved architectural problem at LLM scale. Every published continual-learning method either retains a buffer of prior data, requires task identifiers at training or inference time, applies a regularization penalty that scales poorly with model size, or operates at sentence-classification scale (1 K–5 K samples per task) rather than continual pretraining scale (1 B+ tokens per phase). We introduce **TFGN**, an architectural overlay for transformer language models that sits inside the transformer’s existing per-block computation, producing input-conditioned, parameter-efficient updates while leaving the rest of the transformer unchanged. On six heterogeneous text domains (Prose, Python, Math, Biomedical, Chinese, JavaScript) at 1 B tokens per phase across three total-model scales (~ 398 M, ~ 739 M, ~ 9 B) and two regimes (From-Scratch and Retrofit), TFGN achieves backward transfer (BWT) of -0.007 at LLaMA 3.1 8B Retrofit (3-phase prefix) with HellaSwag retention $0.506 / 0.504 / 0.510$ across the three phases (span 0.006), and $\geq 99.59\%$ L2-orthogonal gradient separation between domain pairs in every tested condition — with no replay buffer, no task IDs, no Fisher penalty, and minimal per-scale tuning. The forward pass is fully dense: every parameter is active on every token, with no sparse gather/scatter and no top-K expert selection. The same numerical evidence shows positive cross-domain forward transfer where structural overlap exists: held-out JavaScript PPL drops 26.8% at LLaMA-8B Retrofit and 62.0% at GPT-2 Medium From-Scratch purely from Python training (on a not-yet-trained domain), demonstrating that the architecture’s protection mechanism preserves cross-domain synergy at inference. Two extensions on the same substrate demonstrate proof-of-concept on two further open problems. A closed-loop intrinsic-signal meta-control layer (Extension A) reduces forgetting by an additional 81% at ~ 398 M parameter scale; its components map onto the System A and System M layers of the autonomous-learning framework proposed by Dupoux et al. [3] (arXiv:2603.15381), demonstrated as a working LLM-scale realization. An operator-level plan vector (Extension B) reshapes the model’s effective forward-pass behaviour at 99.96% cosine fidelity over 30 source $\rightarrow$ target pairs and 99.95% at ~ 739 M with only a -0.0001 cosine drop across the $1.86\times$ total-parameter jump; sub-task injection peaks at 77.8% and averages 55.6% across four Python sub-tasks. Surveying six adjacent literatures — continual learning, mixture-of-experts routing, activation steering, latent planners and world models, meta-learning and self-regulating systems, and intrinsic-signal control — we find no published architecture that clears the conjunction of properties TFGN demonstrates. The architectural insight underlying these results is a Read/Write decomposition: the forward pass remains dense and unimpaired across all domains, while the architecture structures cross-domain parameter updates by an internal mechanism. Stability

*Correspondence: anurup2007@gmail.com. ORCID: [0000-0002-6424-0084](https://orcid.org/0000-0002-6424-0084).

is a write-problem, not a read-problem; cross-domain synergy is the read-pathway corollary. To our knowledge, TFGN is the first published architecture in which one substrate-level primitive simultaneously (i) closes catastrophic forgetting structurally at LLM scale, (ii) realizes a closed-loop autonomous-learning meta-controller at LLM scale, and (iii) carries an operator-level latent planner.

Keywords: continual learning, catastrophic forgetting, transformer language models, backward transfer, replay-free continual learning, task-free continual learning, latent planner, autonomous continual learning.

Contents

1	Introduction	5
1.1	The problem and three open axes	5
1.2	Our contribution: one substrate, three capabilities	6
1.3	Headline results	7
1.4	Paper organization	10
2	Related Work and Prior-Art Position	10
2.1	Families of prior continual-learning work	10
2.2	Column 1 — continual learning at LLM scale (eight-axis grid)	11
2.3	The four empty columns	12
2.4	The substrate property	13
2.5	What this prior-art position implies	13
3	Architecture (Capability-Level Description)	13
3.1	What TFGN is	13
3.2	Mathematical foundations and gradient protection	14
3.3	Architectural consequences	15
3.4	Capacity and compute footprint	15
4	Experimental Setup	16
4.1	Backbones and parameter counts	16
4.2	Continual-learning regime	16
4.3	Continual sequence and token budgets	16
4.4	Datasets	17
4.5	Evaluation protocol	17
4.6	Reproducibility	18
5	Main Results	18
5.1	Headline overview	18
5.2	Per-condition results	19
5.2.1	TFGN_LLAMA8B_RETROFIT — the headline production result	19
5.2.2	TFGN_LLAMA8B_FS — from-scratch at ~9 B	19
5.2.3	TFGN_GPT2M_FS — from-scratch at ~739 M	19
5.2.4	TFGN_GPT2M_RETROFIT — retrofit at ~739 M (hardest condition in this paper)	20
5.2.5	TFGN_GPT2S_FS — small-scale proof point at ~398 M	20

5.3	Matched baseline results	20
5.4	Domain-collapse showcase — the qualitative axis PPL cannot see	20
5.5	Cross-condition summary figures	25
5.6	Gradient orthogonality summary	27
5.7	Cross-domain forward transfer	28
5.8	Per-condition data tables	29
6	Extension A: Autonomous Continual Learning	33
6.1	Extension A as the first LLM-scale realization of the Dupoux/LeCun/Malik 2026 autonomous-learning framework	33
6.2	Capability claim	33
6.3	Headline result	34
6.4	All 11 conditions — canonical BWT/FM table	34
6.5	Per-condition narratives — 1 B Tier B and Tier C	35
6.6	Tier C headline result	38
6.7	Closed-loop self-regulation: capability schematic	38
6.8	First working LLM-scale realization of the System A / System M framework	39
6.9	HellaSwag and gradient orthogonality (Extension A)	40
6.10	Per-condition data tables (Extension A)	40
6.11	Scope and what is reserved	40
7	Extension B: Latent-Planner Capability	43
7.1	Capability claim	43
7.2	Six-criterion structural scorecard	43
7.3	Operator-level control: the structural precondition for the scorecard	44
7.4	Two-bottleneck chain on executor obedience	46
7.5	Pillar 1 — 99.96% operator-level reshape fidelity	46
7.6	Measurement battery: how the three pillars are measured	47
7.7	Pillar 2 — sub-task structure (capability summary)	47
7.8	Pillar 2b qualitative grounding: same model, same prompt, plan vector flips the surface form	47
7.9	Pillar 2b sub-task injection rates	49
7.10	Pillar 3 — scale preservation across the $1.86\times$ jump	49
7.11	Closure roadmap: what’s next	49
7.12	Scope and what is reserved	50
8	Discussion	51
8.1	What the three components jointly establish	51
8.2	Where TFGN sits in the eight-axis landscape	51
8.3	Why this matters for production LLM systems	51
8.4	Forward-pointers across components	52
8.5	Forward-pointer: safety and alignment use cases	52
9	Limitations	53
9.1	Empirical gates: Tier-0 future-work checklist	53
9.2	Architecture access and NDA terms	54
10	Conclusion	55

A	Condition Name Index	58
B	BWT and FM Definitions	59
C	Full 6×6 Gradient Orthogonality Matrices	60
C.1	TFGN_GPT2S_FS (~398 M, From-Scratch)	60
C.2	TFGN_GPT2M_FS (~739 M, From-Scratch)	60
C.3	TFGN_GPT2M_RETROFIT (~739 M, Retrofit)	61
C.4	TFGN_LLAMA8B_FS (~9 B, From-Scratch, 3-phase)	61
C.5	TFGN_LLAMA8B_RETROFIT (~9 B, Retrofit, 3-phase)	61
D	Full PPL Matrices with bwt_d Rows	62
E	Extension A Canonical Values and 11-Condition Matrix	62
F	Tier A 200 M Per-Condition PPL Matrices	62
G	Definitions and Equations Index	64

1 Introduction

1.1 The problem and three open axes

Three open problems at LLM scale are addressed in this paper — two in continual learning, and one in the broader field of latent control of frozen-decoder models:

1. **Catastrophic forgetting in continual learning at LLM scale:** architectures that can learn new domains without destroying prior knowledge. Named open since the catastrophic-forgetting results of the late 1980s, and unsolved on a ~ 9 B transformer under the replay-free, task-free regime that production deployment requires.
2. **Autonomous continual learning:** deciding when and what to learn once you have the right architecture, without external schedulers, domain labels, or supplied phase boundaries.
3. **Operator-level latent planner:** a plan vector that the frozen decoder actually obeys, bypassing token-space chain-of-thought — a problem in the broader latent-control / inspectable-control literature, not strictly continual learning, but solvable on the same routing substrate.

The four-constraint conjunction. What makes continual learning at LLM scale unsolved is not the absence of plausible mechanisms; it is the conjunction of four constraints under which none of the candidate mechanisms has been demonstrated to work simultaneously. (1) *Replay-free*: realistic deployment cannot retain a buffer of historical pretraining data, by privacy, regulatory, or storage constraint. (2) *Task-free*: realistic deployment cannot assume a task identifier at training or inference time; the architecture must dispatch its own gradient updates from the input alone. (3) *Multi-domain at LLM scale*: the continual sequence must be more than two domains and the per-domain budget must be more than 1 K samples; we test six text domains at 1 B tokens per phase, a regime no published replay-free continual-learning method has exercised at ≥ 7 B parameters. (4) *No external orchestrator*: no Fisher-information penalty, no orthogonality regularisation loss, no gradient-projection operator, no task-boundary hook, minimal per-scale tuning. The conjunction is what positions TFGN’s contribution against the prior-art literature surveyed in §2.

Every frontier LLM deployed today is frozen at its last training checkpoint: adding a new language, codebase, or regulatory corpus requires full retraining or brittle adapter stacking. The stability–plasticity tradeoff at LLM scale is the bottleneck several frontier labs have named as the single unsolved obstacle to autonomous, long-horizon systems; Dupoux et al. [3] name continual learning at LLM scale as a core open problem in autonomous-learning research, and the broader field has converged on the same diagnosis. This paper’s architectural claim is that the tradeoff is solvable at the architectural level — no replay, no task IDs, no penalty term — and reproduces that behaviour with minimal changes from ~ 398 M to ~ 9 B parameters.

The metric we care about is **backward transfer (BWT)** [2], defined on a continuous-perplexity scale: for each prior domain, BWT measures the relative degradation in that domain’s perplexity after the model has trained on every later domain. A BWT of zero means no forgetting; negative values mean forgetting. **PPL undersells what catastrophic forgetting actually looks like at LLM scale.** On a standard Prose prompt, a standard-fine-tuned LLaMA 3.1 8B baseline trained through the three-phase continual sequence (Prose $\rightarrow$ Python $\rightarrow$ Math) emits *complete domain collapse* — Python source code for Prose prompts. The Prose prompt “*The history of artificial intelligence began in*”, after the Python phase, continues mid-completion into Python source: `from collections import defaultdict; def _get_list_of_function(func)`. The same pattern repeats at every cross-distribution boundary: Prose prompts emit Chinese characters after a Chinese phase, Math prompts emit JavaScript tokens after a JavaScript phase, and so on. The model has not merely degraded; it has stopped being a Prose model at all. The BWT number for the baseline is bad ($\text{BWT}_3 = -0.374$); the underlying emission distribution is *categorically* bad. PPL averages

over a token distribution and cannot capture that the model has stopped being a Prose model at all. BWT is one face of continual learning, and sample-level emission type is the other; baselines lose on both faces, TFGN holds on both.

Long-context windows do not substitute for continual learning at the relevant scales.

As of 2026, context windows of 2–10 M tokens combined with KV-cache prompt-caching solve a real subset of use cases that catastrophic forgetting blocked: episodic, in-prompt knowledge over corpora that fit in a single window. They do not solve continual learning at the regime this paper targets. Real enterprise corpora — multi-billion-token codebases, multi-terabyte legal / financial / pharmaceutical archives — do not fit at any planned context size. Frontier labs themselves continually update flagship models on order-of-trillion-token deltas; these are continual-training events, not in-context queries. The two regimes are complementary: long context is episodic memory; the architectural property this paper targets is persistent weight-internalized knowledge, with no per-query context cost and multi-tenant economics.

The problem this paper addresses, in one sentence: build a transformer architecture that, under a replay-free, task-free continual-learning regime, closes BWT to near zero *and* preserves emission-type coherence at LLM scale.

1.2 Our contribution: one substrate, three capabilities

TFGN is an architectural overlay for transformer language models. The architectural property is that new-domain training does not disturb prior-domain capability; this property emerges from the architecture itself rather than from a regularization penalty, so a single fixed model preserves competence on its $n-1$ prior domains while still learning the n -th. The full architectural mechanism is reserved; access terms are described in §9.2.

The same TFGN substrate, unchanged from ~ 398 M to ~ 9 B with minimal per-scale tuning, defends all three capabilities named above:

- **Continual learning at LLM scale (Main paper, §5).** Across head-to-head matched comparisons (same backbone, same regime, same per-phase token budget), TFGN reduces BWT by $\sim 3\times$ to $14\times$ over Standard Fine-Tuning and LoRA $r=256$; tightest BWT recorded is -0.007 at LLaMA 3.1 8B Retrofit. Cross-domain gradients remain $\geq 99.59\%$ L2-orthogonal at every tested scale and regime — a structural property of the architecture, not the result of any orthogonality loss term. This orthogonality is at the parameter-update level, not at the inference level — the forward pass is unimpaired across domains. Cross-domain forward transfer is empirically present in the same matrices: held-out JavaScript PPL drops 26.8% at LLaMA-8B Retrofit and 62.0% at GPT-2 Medium From-Scratch purely from Python training (§5.7).
- **Autonomous continual learning (Extension A, §6).** A self-regulating meta-control layer added on the same substrate closes 81% of the residual forgetting gap ($\text{BWT} = -0.01140$) over a three-domain continual sequence at the ~ 398 M parameter scale, using only intrinsic signals the network already computes in its own forward and backward pass. The five components map onto the System A (internal world model) and System M (meta-control) roles in the autonomous-learning framework of Dupoux et al. [3].
- **Operator-level latent planner (Extension B, §7).** The same architectural substrate carries an operator-level plan vector at 99.96% geometrically-inspectable reshape fidelity (mean over 30 source $\rightarrow$ target pairs at ~ 398 M; 99.95% at ~ 739 M retrofit), with 77.8% peak / 55.6% mean Python sub-task injection rate. Six-criterion latent-planner scorecard returns 2 PROVEN, 3 PARTIAL-PROVEN, 1 future work, 0 FAIL.

Substrate framing. These three capabilities sit at the intersection of *four empty columns* in the surrounding literature — one in each of continual learning, mixture-of-experts routing, operator-level latent control, and weight-space latent planning — surveyed in §2.3. No published architecture

has previously occupied any of these four coordinates; the substrate framing is the structural reason TFGN clears all four with one primitive.

Why protection by architecture, not regularization. Every prior CL method leans on a *penalty* (EWC’s Fisher penalty, L2-SP), a *buffer* (experience replay), a *task tag* (PackNet masks), or an *external gate* (ANML). When the penalty weakens, the buffer is absent, or the task tag is unknown, prior-domain competence collapses. TFGN does not protect through any of those mechanisms. Protection is intrinsic to the architecture itself: prior-domain capability is preserved by structure, not by a penalty term. Remove the penalty, remove the buffer, remove the task ID: the protection remains, because it is architectural.

1.3 Headline results

Paper-wide convention. LLaMA 3.1 8B conditions are reported on the three-phase prefix (Prose → Python → Math) due to compute constraints; GPT-2 Small and GPT-2 Medium conditions report all six phases (Prose → Python → Math → Biomedical → Chinese → JavaScript) with 1 B tokens per phase. No task IDs, no replay buffer.

Backward transfer. TFGN closes backward transfer to near zero across three total-parameter scales and two training regimes (Table 1). **Strictly matched-init 9 B comparison** (random-init from-scratch on both sides): TFGN_LLAMA8B_FS closes to $BWT_3 = -0.095$ vs the matched BASELINE_STD_LLAMA8B at -0.374 — a $\sim 3.9\times$ BWT-magnitude gap, the only strictly matched-init ratio we quote at 9 B. **Tightest absolute BWT:** TFGN_LLAMA8B_RETROFIT closes to $BWT_3 = -0.007$ (init-asymmetric vs the same baseline at $\sim 51\times$, called out as directionally large rather than a strictly matched ratio). **Sub-9 B matched ratios (six-phase):** at ~ 739 M, TFGN From-Scratch -0.083 is $\sim 14\times$ tighter than Std-FT -1.170 and $\sim 12\times$ tighter than LoRA-r256 -1.005 ; TFGN Retrofit -0.135 is $\sim 4\times$ tighter than Std-FT -0.541 and $\sim 3\times$ tighter than LoRA-r256 -0.393 . At ~ 398 M From-Scratch, the proof-point closes to -0.109 .

Table 1: Headline backward transfer (BWT) and trained-domain learning across the eleven primary conditions. “Peak trained-domain PPL drop” is the maximum $P_1 \rightarrow P_{\text{trained}}$ reduction across the six (or three) trained domains, illustrating the plasticity face of continual learning alongside the stability face (BWT). *Note on TFGN_LLAMA8B_FS:* this row is the most-adversarial corner of the from-scratch recipe (random-init backbone $\times$ Prose-only Phase 1 $\times$ 1 B tokens/phase, $\approx$ two orders of magnitude below Chinchilla-optimal). The architectural claim at this corner is the matched-baseline BWT ratio ($\sim 3.9\times$); see §1.3 for the full PPL-gap explanation.

Condition	Total params	Regime	Phases	BWT	Peak trained-domain PPL drop	Emission collapse?
TFGN + GPT-2 Small (TFGN_GPT2S_FS)	~ 398 M	FS	6	-0.109	67% (JS)	No
TFGN + GPT-2 Medium (TFGN_GPT2M_FS)	~ 739 M	FS	6	-0.083	67% (JS)	No
TFGN + GPT-2 Medium (TFGN_GPT2M_RETROFIT)	~ 739 M	RF	6	-0.135	49% (JS)	No
TFGN + LLaMA 3.1 8B (TFGN_LLAMA8B_FS)	~ 9 B	FS	3	-0.095	— (substrate-capped)	No
TFGN + LLaMA 3.1 8B (TFGN_LLAMA8B_RETROFIT)	~ 9 B	RF	3	-0.007	30% (Py)	No
Baseline Std-FT GPT-2 Medium (FS)	~ 355 M	FS	6	-1.170	85% (Py)	Yes (every boundary)
Baseline LoRA r=256 GPT-2 Medium (FS)	~ 393 M	FS	6	-1.005	80% (Py)	Yes (every boundary)
Baseline Std-FT GPT-2 Medium (RF)	~ 355 M	RF	6	-0.541	75% (Py)	Yes
Baseline LoRA r=256 GPT-2 Medium (RF)	~ 355 M	RF	6	-0.393	70% (Py)	Yes
Baseline Std-FT LLaMA 3.1 8B (FS, 500 M tok/phase)	~ 8 B	FS	3	-0.374	—	Yes

Gradient orthogonality — the structural signature. Across every TFGN condition, cross-domain gradients from different domains remain $\geq 99.59\%$ L2-orthogonal, with mean cross-domain $|\cos|$ below 0.10:

No orthogonality loss, no gradient-projection operator, no task-boundary hook — the decorrelation emerges from the architecture itself. It is a structural invariant, holding across every tested scale, training regime, and reset condition.

HellaSwag retention. TFGN conditions retain HellaSwag accuracy within 1–2 percentage points across continual phases (e.g., GPT-2 Medium FS: P_1 0.340 $\rightarrow$ P_6 0.338); matched Std-

Table 2: Gradient orthogonality across TFGN conditions. Full 6×6 per-domain-pair matrices in Appendix C.

Condition	Scale	Mean $ \cos $	L2 orthogonal fraction
TFGN GPT-2 Small, From-Scratch	~398 M	0.0425	99.91%
TFGN GPT-2 Medium, From-Scratch	~739 M	0.0204	99.94%
TFGN GPT-2 Medium, Retrofit	~739 M	0.0904	99.59%
TFGN LLaMA 3.1 8B, From-Scratch	~9 B	0.0432	99.91%
TFGN LLaMA 3.1 8B, Retrofit (3-phase)	~9 B	0.0741	99.72%

FT and LoRA baselines drop 3–7 percentage points with the sharpest dip at the Chinese-phase boundary (P5). LLaMA 8B Retrofit retains HellaSwag at 0.506 / 0.504 / 0.510 across the 3-phase presentation (span 0.006). Full per-condition curves in §5.2; cross-condition Figure 2.

Cross-domain forward transfer. The same continual sequences that establish BWT ≈ 0 also exhibit *positive* cross-domain forward transfer on related domains. JavaScript’s held-out PPL drops 26.8% at LLaMA-8B Retrofit (from 23.05 after the initial training stage to 16.87 after the Python phase, on a never-trained domain) and 62.0% at GPT-2 Medium From-Scratch (from 37.1 to 14.1 before JavaScript is itself trained). Math’s PPL drops 2.4% at LLaMA-8B Retrofit and 16.9% at GPT-2 Medium From-Scratch *before Math is trained*, attributable to the intervening Python phase. Cross-domain synergy is empirically present, not just preserved (§5.7). This is the empirical counterpart of the architectural Read/Write decomposition (§3.1): the forward pass enables positive forward transfer; the architecture’s parameter-update geometry prevents backward interference.

Emission coherence — the qualitative axis PPL cannot see. At the first cross-distribution boundary (P2, after Python training), 3/3 baselines drift to Python source code mid-completion on Prose prompts; 5/5 TFGN conditions across every tested scale (~ 398 M / ~ 739 M $\times 2$ / ~ 9 B $\times 2$) emit domain-coherent English Prose. Scale, fine-tuning method, and retrofit-vs-from-scratch all vary across these eight cells; the one consistent dividing line is the presence of the TFGN overlay. The full §5.4 four-row showcase preserves the verbatim baseline emissions (Python boilerplate, Apache-license blocks, Chinese characters mid-completion at P5) alongside the TFGN-coherent counterparts.

TFGN learns each new domain while preserving prior ones. The headline numbers above measure stability (preservation). The continual-learning result is symmetric: across every TFGN condition, every *trained-domain* diagonal cell of the PPL matrix drops materially when its phase is the active phase. At ~ 739 M from-scratch, training on Chinese drops PPL from 52.0 $\rightarrow$ 18.4 (65%); JavaScript 37.1 $\rightarrow$ 12.1 (67%); Python 18.1 $\rightarrow$ 10.8 (40%); Math 49.6 $\rightarrow$ 41.1 (17%). At ~ 739 M retrofit, the same domains still drop further from their already-low pretrained baselines: Python 6.13 $\rightarrow$ 4.18 (32%), JavaScript 7.73 $\rightarrow$ 3.96 (49%). The matched Standard Fine-Tuning baseline at the same backbone learns each new domain more aggressively (Python 18.44 $\rightarrow$ 2.80, an 85% drop at ~ 739 M FS) but pays for the aggression with categorical prior-domain forgetting — its Prose-row PPL collapses from 33.79 at P1 to 89.83 at P5 Chinese (+136%). TFGN trades a calibrated amount of plasticity for full stability across every prior domain: the stability–plasticity tradeoff is solved architecturally, not by sacrificing one face for the other.

The trained-domain PPL gap is an artifact of an adversarial Phase-1 training setup, applied identically to from-scratch and retrofit. The Phase 1 corpus is intentionally restricted to Prose alone in both regimes. This is a deliberate research-setup choice with two purposes. First, it converts every later domain (Python, Math, Biomedical, Chinese, JavaScript) into a held-out routing test by construction: a core property under evaluation is whether the architectural mechanism, set during the initial training stage, generalizes to distributions it has never seen

during its formation. A Prose-only Phase 1 is the strongest version of that test. Second, restricting Phase 1 to a single distribution maximizes cross-distribution stress on the continual sequence: every continual phase introduces a domain the frozen substrate has never represented, isolating the architecture’s continual-phase parameter subset as the sole mechanism available to absorb new structure. The setup is held constant across from-scratch and retrofit so the two regimes remain directly comparable on every other axis.

The architectural consequence is that at the boundary between the initial training stage and the continual phases, the substrate that shapes every token’s output distribution is Prose-biased by construction, and the continual-phase mechanism does not re-shape it. The matched Standard Fine-Tuning baseline does not face this constraint; it updates all parameters at every phase, so its attention and output projection re-shape to fit each new trained domain, which is what permits its more aggressive trained-domain PPL drops. The adversarial Phase-1 setup, retained throughout this paper precisely so the router-generalization and sequential-stress properties are testable, therefore caps the achievable trained-domain PPL on the Prose-biased substrate.

Because the Prose-only Phase 1 is held constant across regimes, the same substrate-bias mechanism explains both regimes’ PPL gaps. *Retrofit closes most of the gap* ($\sim 1.5\text{--}2\times$ at $\sim 739\text{M}$ retrofit, vs $\sim 3.9\times$ at $\sim 739\text{M}$ from-scratch) because the pretrained backbone enters Phase 1 already carrying general-purpose cross-domain representations from its original pretraining corpus, which Phase-1 Prose adapts but does not erase. Whatever PPL gap remains in the retrofit regime is attributable to the same adversarial Prose-only Phase 1: the post-Phase-1 frozen substrate is still pulled toward Prose, only less severely than the random-init from-scratch case. At the $\sim 9\text{B}$ from-scratch corner, the gap is largest because two further factors stack on top of the Phase-1 substrate bias: backbone undertraining at 1B tokens/phase ($\approx$ two orders of magnitude below Chinchilla-optimal for a transformer of this size) and a random-init backbone with no cross-domain priors at all.

Production proposal (future work): mixed-domain Phase 1. The proposed production-deployment configuration, outside the scope of the present paper, replaces the Prose-only Phase 1 with a mixture of Prose and the later continual-curriculum domains. The post-initial-stage frozen substrate would then carry cross-domain representations rather than Prose-only ones, giving the continual-phase learning a much stronger substrate to operate on, and closing most of the trained-domain PPL gap reported here. The Prose-only Phase 1 is retained throughout this paper because the router-generalization and adversarial sequential-learning properties are the load-bearing claims under test. Mixed-domain Phase 1 is on the future-work roadmap (§9).

The router trained on Prose alone routes unseen domains correctly. Despite the initial training stage being Prose-only, the architectural mechanism set during it generalizes to every later domain at every continual phase without re-learning the structure: Python, Math, Biomedical, Chinese, and JavaScript tokens each land in their own near-orthogonal subspace (mean cross-domain $|\cos| \leq 0.09$ across all domains never seen during the initial stage). The mechanism is content-driven — not a learned task classifier — so the substrate trained on Prose alone correctly disambiguates unseen domains, preserving the orthogonality property that protects prior-domain capability.

Closing summary. A single architectural overlay — applied with minimal per-scale tuning across three model scales, from $\sim 398\text{M}$ to $\sim 9\text{B}$ — delivers selective protection of every prior domain, trained-domain learning at every phase, and gradient orthogonality as a structural signature. The headline positioning is therefore that **TFGN’s architectural continual learning addresses the biggest problem of stability–plasticity trade-off at LLM scale.**

1.4 Paper organization

§2 positions TFGN against published work using a eight-axis prior-art grid covering 14 representative methods. §3 describes the architecture at the capability level (full mechanism reserved; see §9.2). §4 specifies datasets, backbones, regimes, and evaluation protocol. §5 is the main results section: per-condition results for all eleven conditions, the four-row emission-coherence showcase (§5.4), and three cross-condition figures (M1 BWT, M3 HellaSwag, M6 gradient orthogonality). §6 (Extension A: autonomous continual learning) and §7 (Extension B: latent-planner capability) report the two extensions. §8, §9, and §10 close the paper. Appendices contain condition name index, BWT/FM definitions, full 6×6 gradient-orthogonality matrices, per-condition full PPL matrices, Extension A canonical values, and Extension B sub-task evidence.

2 Related Work and Prior-Art Position

2.1 Families of prior continual-learning work

Prior continual-learning work falls into a small number of families. Each family fails on a specific architectural failure mode before reaching the regime this paper occupies — replay-free, task-free, LLM-scale, no external orchestrator. We describe the families and their shared failure modes at the category level.

Regularization-based CL. EWC [4], MAS [5], SI [6], GEM and A-GEM [2, 7], OGD [11], GPM [12], and Adam-NSCL [13] penalize updates to weights deemed important for earlier tasks, using Fisher information, synaptic importance, or a gradient-subspace projection. Per-weight importance state scales with model size and becomes compute-prohibitive at LLM scale; the importance signal requires an explicit prior-task boundary, which is unavailable under the task-free regime. In addition, each new phase freezes more parameters, so the share of trainable capacity shrinks with every added domain — an architectural ceiling that tightens with the number of tasks.

Replay / rehearsal. ER [8], A-GEM [7], DER [9], MIR [10], and the “Revisit-Replay” family [45] buffer a subset of past data and interleave it with current-task training. This violates the replay-free constraint entirely; data retention is disallowed in many production settings, and where permitted, the buffer’s footprint grows with the number of domains. “Revisit-Replay” (the August 2025 SOTA for CPT at scale) tests 0%, 25%, and 50% replay rates on Spectra LLMs at 99 M, 560 M, 1 B, and 5.7 B parameters and reports the no-replay baseline (TFGN’s regime) as the worst-performing condition; the paper recommends 25–50% replay rates as the strongest recipe across all model sizes including the 5.7 B rung.

Parameter-isolation and task-conditioned modular methods. PackNet [14], HAT [16], Piggyback [15], Progressive Networks [17], the LoRA-CL family [19, 20, 22], Lifelong-MoE [23], LoRAMoE [24], CodaPrompt [25], STABLE [21], and hypernetwork CL [26] allocate fresh capacity per task — a parameter mask, a LoRA stack, an expert route, or a hypernetwork-generated weight slab — and switch between allocations by task ID or by a learned task classifier. They require a task ID at inference, or a task classifier that itself forgets; parameter expansion grows with the number of tasks; per-task modules must either be retained forever or arbitrated at serving time; and they do not close BWT on the task-free, replay-free regime at LLM scale.

Model editing and knowledge-locating. ROME [34], MEMIT [35], NSE [36], WISE / MAKE / HiEdit [37–39] perform rank-one surgical edits or per-fact memory insertions into mid-layer FFN weights. They are designed for one-shot factual insertion, not continual learning under distribution shift; edits accumulate interference and degrade after ~1000 sequential applications (“knowledge attenuation”).

LLM-CL surveys. Two recent ACM Computing Surveys [40, 41] catalogue the field’s coverage.

The longest sequence of pre-training stages explored prior to TFGN is 8 — but those use replay-based methods. No replay-free + task-free + penalty-free + 7B+ + multi-disjoint-domain CPT method exists in the surveyed literature.

2026 frontier-scale evidence. Imanov [42] (arXiv:2601.18699) tests Llama 4 Scout (109 B), Llama 4 Maverick (400 B), GPT-5.1 (~ 1.5 T), Claude Opus 4.5, Gemini 2.5 Pro (~ 1 T), and DeepSeek-V3.1 (671 B) on twelve continual-fine-tuning sequences (each 4–6 tasks). Reported absolute capability degradation across the experimental conditions ranges from ~ 15 –20% on the largest models to 24.8% on high-similarity and 31.7% on low-similarity sequences; approximately 15–23% of attention heads in lower layers undergo severe disruption, and forgetting severity correlates with task similarity at Pearson $r = 0.87$. The paper identifies three driving mechanisms (gradient interference in attention weights, representational drift in intermediate layers, loss-landscape flattening) and is the field’s frontier-scale ground truth for the unsolvedness of catastrophic forgetting in 2026.

2.2 Column 1 — continual learning at LLM scale (eight-axis grid)

We position TFGN against published continual-learning methods using a eight-axis grid of load-bearing properties. Each axis is a single-bit predicate that a comparable method either satisfies (PASS) or does not (FAIL); a PARTIAL entry indicates ambiguous coverage. **No prior method passes all eight simultaneously.**

- A. **Regime:** Continual Pre-Training (CPT)? CPT is the regime where the model continues unsupervised pretraining on new domains. CFT (Continual Fine-Tuning) is a strictly easier setting on smaller token budgets.
- B. **Scale:** ≥ 7 B total parameters?
- C. **Domains:** ≥ 4 disjoint domains in the continual sequence?
- D. **Tokens/phase:** ≥ 1 B tokens per phase? (CFT methods typically operate at 1K–25K samples per task, three to six orders of magnitude below CPT regime.)
- E. **Replay-free:** no episodic memory buffer, no curriculum mixture as soft replay?
- F. **Task-ID-free:** no task ID or task-classifier signal at training or inference?
- G. **Penalty-free:** no orthogonality loss, no Fisher penalty, no synaptic-importance regularizer, no gradient-projection operator?
- H. **Both regimes:** demonstrated in both From-Scratch and Retrofit?

Reading rule. TFGN is the only row with PASS in all eight columns. The closest neighbour is **Examining Forgetting in CPT** [43] — which passes A/B/D/E/F/G but fails on C (only 1 domain) and H (retrofit only). The next closest is **Llama-3-SynE** [44] which passes A/B/F/G but fails on C (~ 2 –3 domains), E (uses curriculum/mixture as soft replay), and H (retrofit only).

What the two near-neighbours conclude. *Examining Forgetting in CPT* [43] continually pretrains an aligned LLM (Llama-2-7b-chat) on a 1 B-token Traditional Chinese corpus and tests whether common parameter-efficient fixes (selective layer freezing, LoRA on Q/V projections, and (IA)³ rescaling of K/V matrices and FFN inner activations) resolve the forgetting that ensues. The paper’s stated conclusion is that “catastrophic forgetting during continual pre-training is a non-trivial challenge and cannot be resolved through straightforward methods”. TFGN’s contribution sits exactly where this paper’s straightforward-method investigation stops: an architectural change that produces forgetting resistance as a structural property rather than a hyperparameter target. *Llama-3-SynE* [44] continually pretrains Llama-3 8B on a 100 B-token corpus mixing general data with a 1.5 B-token synthetic-QA enhancement, in a two-stage curriculum (bilingual adaptation, then scientific enhancement), to gain Chinese and scientific reasoning while retaining English. The recipe is the strongest published *data-side* answer to forgetting at 8 B scale: a carefully designed mixture and curriculum that functions as soft replay. TFGN clears the same scale rung with 0% replay and zero curriculum design — the architecture absorbs the cross-domain stress that

Table 3: Eight-axis prior-art grid. Each row is a representative method; cells are PASS / FAIL / PARTIAL. **TFGN is the only row with PASS in every column.**

Method	A CPT	B $\geq 7B$	C ≥ 4 dom.	D $\geq 1B$ tok /phase	E Replay- free	F Task-ID- free	G Penalty- free	H FS+RF
TFGN (this work)	P	P	P	P	P	P	P	P
TreeLoRA [20]	F	P	P	F	P	F	P	F
TRACE benchmark [47]	F	P	P	F	P	F	P	F
STABLE [21]	F	P	—	F	P	F	F	F
O-LoRA [19]	F	P	P	F	P	P	F	F
EWC-Gemma2 [46]	P	F	F	PARTIAL	P	P	F	F
Llama-3-SynE [44]	P	P	F	P	F	P	P	F
Examining Forgetting [43]	P	P	F	P	P	P	P	F
Loss of Plasticity [33]	F	F	P	—	varies	P	F	—
GEM [2]	F	F	P	F	F	F	F	—
Revisit Replay [45]	P	P	PARTIAL	P	F	P	F	F
ANML [27]	F	F	P	—	P	F	F	F
Backpropamine [30]	F	F	varies	—	P	F	F	F
LoRA (base) [18]	F	P	F	P	—	—	—	F

Llama-3-SynE’s data mixture is engineered to mask.

2.3 The four empty columns

The eight-axis grid above closes a CL-specific argument. Stepping out one level of generality, the same finding repeats across four adjacent literatures. In each one, the property TFGN demonstrates lives in an empty column: there is a coordinate where no published architecture has previously sat. This is the substrate-level reason TFGN is more than the sum of three independent results.

Column 1: continual learning at LLM scale. The eight-axis grid above is the proof. *No published method clears the replay-free, task-free, ≥ 4 -domain, $\geq 1B$ -tokens-per-phase, $\geq 7B$ -parameter conjunction.* The closest neighbours fail on at least two axes; the LLM-CL surveys [40, 41] confirm the longest replay-free pretraining sequence prior to TFGN is 1–2 disjoint domains.

Column 2: a finer-than-MoE routing primitive deployed for continual learning at LLM scale. The mixture-of-experts literature — Switch Transformer [62], GShard, GLaM, ST-MoE, Mixtral — routes at the *token-to-expert* granularity. Fine-grained MoE [63] segments each expert into 64 sub-experts and scales to 671 B in DeepSeek-V3 — the closest published instance of finer-than-expert routing. TFGN routes at a finer-than-expert granularity, on different machinery (an architectural mechanism intrinsic to the substrate, not pre-declared expert pools), and used for a different purpose (continual-learning gating, not multitask capacity). No published MoE occupies this conjunction; the specific routing granularity and mechanism are reserved (§9.2).

Column 3: operator-level (weight-space) latent control at LLM scale. The activation-steering and latent-control literature — ROME [34], MEMIT [35], ActAdd [64], CAA [69], RepE [65], ITI [66], function vectors [67], sparse-autoencoder steering [68], refusal-direction edits [70] — operates on *activations*: the steering signal injects a vector into the residual stream at one or more layers. The model’s behaviour shifts because the activations through the rest of the forward pass are different. *No published activation-steering method reshapes the model’s effective forward-pass operator at LLM scale with measured geometric fidelity.* TFGN’s planner output (Extension B)

operates one level deeper, on the operator: the model’s effective forward-pass behaviour is reshaped by the planner output. The reshape that the plan vector predicts agrees with the reshape that actually occurs at 99.96% cosine fidelity at ~ 398 M and 99.95% at ~ 739 M — a property no prior activation-steering or model-editing method reports.

Column 4: weight-space latent planning. Latent reasoning in 2024–2025 occupies three planning spaces, all but one of which has at least one published method. *Token space*: chain-of-thought [54], tree-of-thoughts [55] — reasoning materialises as discrete tokens; the plan is a sequence of natural-language steps. *Activation space (residual-stream)*: Coconut [56] and successor systems — reasoning lives in the residual stream as continuous embeddings fed back as next-step inputs; the plan is a sequence of hidden vectors. *Latent dynamics space*: world-model planners — MuZero [58], Dreamer V3 [57], JEPA [59], V-JEPA 2 [60], Diffuser [61] — the plan is a trajectory in a learned latent z -space and an external decoder turns z back into actions or pixels. *Weight space (operator space)*: **the column is empty until TFGN Extension B.** The plan vector reshapes the model’s effective operator; the model’s *function* changes, not just its activations or its latent state. The closest near-neighbour is Coconut, which steers what the network *says* through continuous activations; Extension B reshapes what the network *is* by editing the operator. A recent adversarial study [71] shows Coconut’s latent tokens are insensitive to perturbations and exhibit shortcut dependence on multiple-choice tasks — a vulnerability TFGN’s weight-space formulation provably avoids because injection is a measured-cosine-0.9996 reshape of the operator, not an opaque activation embedding.

2.4 The substrate property

The architectural primitive at the heart of TFGN is the same in all four cases: a single architectural mechanism. The same mechanism that protects prior-domain capabilities during continual learning (main paper) is reused at different operating scales by the autonomous meta-control layer (Extension A) and by the operator-level plan vector (Extension B). One substrate, three capabilities. This substrate framing is the structural reason TFGN occupies four empty columns in four different literatures simultaneously.

2.5 What this prior-art position implies

Three observations follow from the four-empty-columns finding. *First*, the field’s August-2025 SOTA for continual pretraining at scale (Revisit Replay) reports the no-replay condition as worst-performing across all tested scales and recommends 25–50% experience replay as the strongest recipe up to 5.7 B parameters; TFGN reports comparable-or-tighter BWT at ~ 9 B with 0% replay — a category contrast against the field’s strongest CPT baseline. *Second*, every CFT-regime method (TreeLoRA, O-LoRA, STABLE, ConPET, etc.) operates at 1 K–25 K samples per task, three to six orders of magnitude below TFGN’s 1 B tokens per phase. *Third*, the four-empty-columns substrate property places TFGN in a different category of architectural contribution than incremental method papers in any one of the four literatures: the substrate is the load-bearing claim, and the three capabilities are evidence that the substrate is general.

3 Architecture (Capability-Level Description)

3.1 What TFGN is

TFGN augments the standard transformer architecture with a learned overlay that sits inside the transformer’s existing per-block computation, producing input-conditioned, parameter-efficient updates while leaving the rest of the transformer unchanged. The overlay does not introduce a new top-level layer or a new external orchestrator, and the standard transformer’s outward be-

behaviour is intact when the overlay’s contribution is uninformative. The overlay’s internal structure is reserved (§9.2).

What the overlay delivers:

- Sequential training on multiple task distributions with negligible forgetting on prior tasks — demonstrated across three transformer scales up to a frontier-class 8 B-parameter backbone.
- No replay buffer, no task identifier, no auxiliary regularization penalty: the protection is a property of the architecture itself.
- Backbone-agnostic with two deployment modes: from-scratch training on a randomly-initialized backbone, and retrofit onto an already-pretrained model.
- Parameter-efficient continual phases: the trainable parameter count in continual phases is approximately an order of magnitude smaller than the full model parameter count.

Architectural detail and biological motivation are reserved (§9.2).

Graceful degradation in Retrofit and on out-of-distribution inputs. The architecture is designed to degrade gracefully on out-of-distribution inputs and to preserve the backbone’s behaviour in Retrofit mode, supporting safe addition of the TFGN overlay to a pretrained model without disturbing its existing capabilities. This property is what makes the substrate compatible with frozen backbones: when the overlay’s contribution is small for a given input, the backbone’s pretrained behaviour is preserved by construction.

Stability is a write-problem, not a read-problem. The continual-learning protection lies in the architecture’s parameter-update geometry, not in representational isolation. The forward pass remains dense and unimpaired across all domains; cross-domain inference is preserved. Only the parameter updates that arrive during training are structured into separate subspaces by an internal mechanism. This decoupling — dense, shared read; structured write — is the architectural primitive. The empirical companion is the cross-domain forward-transfer measurement of §5.7: held-out JavaScript PPL drops 26.8% at LLaMA-8B Retrofit purely from Python training, demonstrating cross-domain synergy is preserved at inference.

Forward-pass density. The forward pass is fully dense: every parameter is active on every token. Unlike mixture-of-experts top-K routing (DeepSeek-V3, Mixtral), no expert is skipped, no parameter is conditionally evaluated, and no token-by-token gather/scatter is required. The forward pass is GEMM-friendly on contemporary accelerators by construction.

Capacity scaling. The architecture’s continual-learning capacity grows roughly exponentially in the backbone’s hidden-dimensional width by a Johnson–Lindenstrauss-style packing bound (the bound’s specific form, including the per-backbone numerical table, is reserved; §9.2). At LLaMA-8B-class scale and the paper’s $|\cos| \leq 0.1$ threshold, the architectural envelope supports tens of thousands of routable domains; at slightly looser thresholds, the envelope grows by many orders of magnitude. The paper’s tightest mean $|\cos|$ (0.0904) and the L2-orthogonal-fraction floor (99.59%) both appear at GPT-2 Medium Retrofit at $D = 6$. The empirical mean $|\cos|$ at LLaMA-8B Retrofit ($D = 3$, mean $|\cos| = 0.0741$) sits well below the architectural envelope; capacity is many orders of magnitude beyond the empirical $D = 2$ to $D = 6$ regime tested in this paper, with the empirical $D \geq 20$ ladder reserved as a future-work milestone.

The TFGN architecture is biologically motivated, but the motivation is not “neural networks should look more like neurons.” It is about *which level of biological organization* is the right abstraction for continual learning in a large model.

The biological motivation is reserved (§9.2).

3.2 Mathematical foundations and gradient protection

Forgetting-resistance story (capability level). In standard fine-tuning on a sequence of tasks, training on the second task overwrites parameters needed for the first — the well-known

catastrophic-forgetting failure. The TFGN overlay resists this failure by causing the gradient signal from one task to land in a structurally different subspace of the architecture’s continual-phase trainable parameters than the gradient signal from another task, so updates driven by a new task do not overwrite the parameters that carry the old task. This subspace-separation property arises automatically from the architecture; the engineering work is to amplify the separation through several mutually-reinforcing levers (the levers are reserved; §9.2). When all levers are active, the per-task loss degradation after training on a new task is bounded by a small multiplicative factor that shrinks toward zero as subspace separation approaches completeness.

Deployment safeguards (capability level). In addition to the core forgetting-resistance property, the architecture exposes operational levers for production deployment: a one-time stabilization pass that fixes the protection-bearing structure before continual phases begin, mechanisms that sharpen task-boundary discrimination, and a per-layer opt-in mechanism in Retrofit mode so operators can restrict the continual-learning overlay to a chosen subset of layers. The lever specifications are reserved (§9.2).

3.3 Architectural consequences

The architectural design above produces three operational consequences. First, per-domain protection emerges from the architecture rather than from any per-task module, so no task identifier or domain label is required at training or inference. Second, the continual-phase trainable parameter count is approximately an order of magnitude smaller than the full model parameter count (Table 4), making continual phases parameter-efficient relative to full retraining. Third, the design is backbone-agnostic and supports both from-scratch and retrofit instantiations of the same overlay.

3.4 Capacity and compute footprint

The architecture’s parameter footprint and trainable-parameter schedule are operational facts about the implementation, reproducibly defined, and are reported in full so the experimental setup is reproducible at the level required for reviewers to verify scale and regime.

Table 4: Per-condition parameter footprint and trainable-parameter schedule. “Backbone” is the unmodified transformer parameter count; “Overlay” is the TFGN-specific addition; “Total” is the sum that the headline scale claim (~ 398 M / ~ 739 M / ~ 9 B) refers to. Phase 1 is the initial training stage; the subsequent phases are the continual phases. “ER” is a configuration flag whose detailed mechanism is reserved (§9.2).

Condition	Backbone	Overlay	Total	Train P1	Train P2+	Note
TFGN GPT-2 Small FS	~ 124 M	~ 274 M	~ 398 M	~ 398 M	~ 75 M	ER: on
TFGN GPT-2 Medium FS	~ 355 M	~ 384 M	~ 739 M	~ 739 M	~ 101 M	ER: on
TFGN GPT-2 Medium RF	~ 355 M	~ 384 M	~ 739 M	~ 739 M	~ 101 M	ER: off
TFGN LLaMA 3.1 8B FS	~ 8.03 B	~ 1.05 B	~ 9.08 B	~ 9.08 B	~ 470 M	ER: on
TFGN LLaMA 3.1 8B RF	~ 8.03 B	~ 953 M	~ 8.98 B	~ 8.98 B	~ 470 M	ER: off

Continual-phase capacity is bounded. In every TFGN condition, the continual-phase trainable count (continual phases) is between 1% and 14% of the total parameter count. The architecture is therefore parameter-efficient in the continual phases, even though the first phase trains the full model. **Backbone parity.** For every TFGN condition there is a matched standard fine-tuning baseline and (at GPT-2 Medium) a matched LoRA $r=256$ baseline trained on the same backbone with the same continual sequence and the same per-phase token budget; matched-baseline parameter counts are listed in Section 4 below.

4 Experimental Setup

4.1 Backbones and parameter counts

Three backbone scales are evaluated: **GPT-2 Small** (~ 124 M), **GPT-2 Medium** (~ 355 M), and **LLaMA 3.1 8B** (~ 8.03 B with untied `lm_head`). At each scale, the TFGN overlay adds the parameter count reported in Table 4; matched baselines are trained on the same backbone with no overlay. The total-parameter scale claim (~ 398 M / ~ 739 M / ~ 9 B) refers to backbone + overlay; matched standard fine-tuning baselines therefore have a smaller total parameter count than their TFGN counterparts (this is the size of the overlay), and matched LoRA $r=256$ baselines have a slightly larger count than the standard-FT baselines because of the LoRA adapter parameters.

Table 5: Matched baselines and their total-parameter counts. Each TFGN condition is paired with the standard fine-tuning baseline at the same backbone scale, and (at GPT-2 Medium) with the LoRA $r=256$ baseline.

Baseline	Total params	Pairs with
<code>BASLINE_STD_GPT2M_FS</code>	~ 355 M	TFGN GPT-2 Medium FS
<code>BASLINE_LORA256_GPT2M_FS</code>	~ 393 M	TFGN GPT-2 Medium FS
<code>BASLINE_STD_GPT2M_RETROFIT</code>	~ 355 M	TFGN GPT-2 Medium RF
<code>BASLINE_LORA256_GPT2M_RETROFIT</code>	~ 355 M	TFGN GPT-2 Medium RF
<code>BASLINE_STD_LLAMA8B</code>	~ 8 B	TFGN LLaMA 3.1 8B (FS / RF; called out as init-asymmetric for the RF pairing)

4.2 Continual-learning regime

Two regimes are tested:

- **From-Scratch (FS)**. Random initialization of all parameters (backbone + overlay where applicable). The model is trained end-to-end through the continual sequence with no pretrained checkpoint.
- **Retrofit (RF)**. A pretrained backbone is used; for TFGN-RF, the TFGN overlay is grafted onto the frozen pretrained backbone in Phase 1 and trained alongside the backbone, then the backbone is frozen from the second phase onwards (only the overlay’s persistent continual-phase parameters receive gradient). For Std-FT-RF and LoRA-RF, the standard fine-tuning regime is applied to the pretrained backbone.

The continual-learning regime is **Continual Pre-Training (CPT)**: each phase consists of unsupervised next-token-prediction training on the phase’s domain, with no task labels and no instruction tuning. CPT is the strictly harder regime versus continual fine-tuning (CFT), which operates at 1K–25K instruction-tuned samples per task; CPT operates at 1 B unlabeled tokens per phase here.

4.3 Continual sequence and token budgets

Six-domain continual sequence:

Prose $\rightarrow$ Python $\rightarrow$ Math $\rightarrow$ Biomedical $\rightarrow$ Chinese $\rightarrow$ JavaScript

GPT-2 Small and GPT-2 Medium conditions report all six phases. **LLaMA 3.1 8B conditions report on the three-phase prefix** (Prose $\rightarrow$ Python $\rightarrow$ Math) due to compute constraints — a strict prefix of the six-phase sequence with no re-ordering. All BWT values for LLaMA 3.1 8B conditions are therefore BWT_3 under this lock.

Token budget: 1 B tokens per phase for every reported TFGN condition and for matched GPT-2 Medium baselines. The matched 8 B baseline (`BASLINE_STD_LLAMA8B`) is trained at ~ 500 M

tokens per phase, with BWT recomputed on a strict 3-phase basis for the matched comparison (the asymmetry is called out in Section 5 where it appears).

4.4 Datasets

The six per-domain corpora are sourced from open-access datasets, streamed from HuggingFace, tokenized at full document length, and concatenated with end-of-sequence (EOS) tokens between documents into flat 1D token streams. Per-domain corpus selection and the canonical sources are listed below; for each, we cite the canonical reference and name the specific HuggingFace artifact used as the reproducible snapshot.

- **Prose:** English educational web text from FineWeb-Edu [49], accessed via the `HuggingFaceFW/fineweb-edu` HuggingFace snapshot (config `sample-100BT`); we apply the dataset’s educational-quality filter at `int_score` ≥ 3 and skip documents shorter than 200 characters.
- **Python:** Python source code from StarCoderData [50], accessed via the `bigcode/starcoderdata` HuggingFace snapshot (`data_dir=python`); we skip documents outside the 50–100,000-character range.
- **Math:** mathematical web text from OpenWebMath [51], a 14.7B-token Common Crawl subset with SimHash deduplication and LaTeX preserved, accessed via the `open-web-math/open-web-math` HuggingFace snapshot; we skip documents outside the 100–500,000-character range.
- **Biomedical:** 27.7M scientific-article abstracts from the NLM PubMed bibliographic database [52], accessed via the curated `uiyunkim-hub/pubmed-abstract` HuggingFace snapshot; we skip documents shorter than 100 characters.
- **Chinese:** Chinese web text from CulturaX [53], accessed via the `uonlp/CulturaX` HuggingFace snapshot (`name=zh`); we skip documents shorter than 100 characters.
- **JavaScript:** JavaScript source code from StarCoderData [50], accessed via the `bigcode/starcoderdata` HuggingFace snapshot (`data_dir=javascript`); we skip documents outside the 50–100,000-character range.

Train/test/validation splits are taken from the front/back of the per-domain token stream (test and validation from the end) to avoid overlap. The per-domain held-out splits are stable across phases. Domain-specific tokenizers are not used; the model uses its native tokenizer (GPT-2 BPE for GPT-2 backbones, the LLaMA 3.1 8B tokenizer `meta-llama/Meta-Llama-3.1-8B` for the 8B backbone). Documents producing fewer than 10 tokens are skipped at all sizes; per-document EOS markers (`<|endoftext|>` for GPT-2, `<|end_of_text|>` for LLaMA 3.1 8B) separate concatenated documents in the flat stream.

4.5 Evaluation protocol

For each phase t and each domain d , we record the perplexity $M[t, d]$ on the held-out split of domain d after Phase t has finished training. The full per-condition output is therefore a $T \times D$ matrix where T is the number of phases and D the number of evaluation domains. From the matrix we compute:

- **BWT (Lopez-Paz adapted to perplexity):** averaged over each prior trained domain d , the relative degradation from $M[d, d]$ (just-trained perplexity for d) to $M[T, d]$ (perplexity for d after the final phase). Defined formally in Appendix B.
- **FM (Forgetting Measure):** maximum per-domain forgetting over the sequence; defined in Appendix B.
- **Per-domain bwt_d :** the per-domain decomposition of BWT, useful for diagnosing which domains contribute most to the average.
- **HellaSwag accuracy:** per-phase, on a fixed $n = 500$ subset of HellaSwag [48], with no fine-tuning on HellaSwag itself. The model is presented each example’s context c and four candidate

endings $\{e_1, e_2, e_3, e_4\}$. The score for ending e_i is the mean per-token cross-entropy on the ending tokens only,

$$\text{score}(e_i) = \frac{1}{|T_i|} \sum_{t \in T_i} -\log p(t | c, e_i^{<t}),$$

where T_i is the set of token positions belonging to ending e_i (context tokens are excluded from the average). The predicted ending is $\arg \min_i \text{score}(e_i)$, and accuracy is the fraction of examples where the predicted ending matches the gold-label ending. The same ending-only token-mean rule is applied uniformly to every condition reported in this paper. Absolute levels under this rule are below values published under the lm-eval-harness `acc_norm` convention (which uses character-length normalization rather than per-token mean and applies additional preprocessing); the within-condition span across phases — the load-bearing quantity for the continual-learning claim — is unaffected by the choice between rules and reflects only the model’s preserved capability across the continual sequence. A probe of preserved general-language capability.

- **Gradient orthogonality:** for every pair of domains (i, j) , the mean absolute cosine between a batch of domain- i gradients and a batch of domain- j gradients, computed on the architecture’s continual-phase trainable parameters. The L2-orthogonal fraction reported below is the complementary geometric quantity — the fraction of each gradient that lies outside the span of the other.
- **Emission coherence:** a fixed 6-domain $\times$ 3-prompt set (18 prompts total, one prompt set per domain: Prose, Python, Math, Chinese, JavaScript, Biomed) evaluated at sampling temperature 0.7, top- $p = 0.9$, and `max_new_tokens = 50` at every phase; each output is labeled by emission type (Prose-coherent / Python-bleed / Math-bleed / Chinese-bleed / JavaScript-bleed / repetition / gibberish). Section 5.4 presents the emission-coherence cross-condition showcase.

4.6 Reproducibility

All datasets used are publicly available; all backbone weights for Retrofit conditions are publicly available checkpoints (GPT-2 from OpenAI / Hugging Face, LLaMA 3.1 8B from Meta). Hyperparameters (learning rates, optimizer, schedule, batch size) are summarized in Appendix B. Full training logs, per-condition checkpoints, source code, and the architectural mechanism are reserved; access terms are in §9.2.

5 Main Results

This section presents the full per-condition evidence: a headline summary table (§5.1), per-condition results for each of the eleven primary conditions (§5.2), the four-row emission-coherence showcase (§5.4), and three cross-condition figures (§5.5).

5.1 Headline overview

Navigation note. §5.8 at the end of this section collects the full per-condition data tables (PPL matrices and scalar metric blocks for all 11 conditions). The narrative subsections (§5.2, §5.3) cite the relevant table for each condition; cross-condition figures appear inline at §5.5.

The headline backward-transfer numbers for the eleven primary conditions appear in §1.3 (Table 1), where they are introduced alongside the matched-baseline ratios and the architectural framing. This section drills down to per-condition evidence: full PPL matrices, per-domain bwt_d rows, HellaSwag retention vectors, gradient-orthogonality compact summaries (TFGN only), the four-row emission-coherence showcase (§5.4), and three cross-condition figures (§5.5). Per-condition tables and figures are presented inline with their narratives below.

Across every cross-distribution phase boundary (P1→P2 Prose to Python; P2→P3 Python to Math; P3→P4 Math to Biomedical; P4→P5 Biomedical to Chinese; P5→P6 Chinese to JavaScript), every measured baseline emits a categorical domain-collapse signature. TFGN at every tested scale and regime preserves emission-type coherence at every cell. §5.4 presents the four-row showcase.

Matched-ratio readings are the load-bearing comparative claims: at ~739 M From-Scratch, TFGN closes BWT to -0.083 versus matched Std-FT at -1.170 ($\sim 14\times$ smaller magnitude) and matched LoRA-r256 at -1.005 ($\sim 12\times$ smaller magnitude). At ~739 M Retrofit, TFGN closes BWT to -0.135 versus matched Std-FT at -0.541 ($\sim 4\times$) and matched LoRA-r256 at -0.393 ($\sim 3\times$). At ~398 M From-Scratch the proof-point condition closes to -0.109 . **The strictly matched 9 B comparison** (random-init From-Scratch on both sides) is TFGN_LLAMA8B_FS (-0.095 , 1 B tok/phase) versus BASELINE_STD_LLAMA8B (-0.374 , ~ 500 M tok/phase) at $\sim 3.9\times$. **The directionally large but init-asymmetric 9 B comparison** (pretrained-backbone Retrofit TFGN vs random-init From-Scratch baseline) is TFGN_LLAMA8B_RETROFIT (-0.007) versus the same baseline at $\sim 51\times$ — called out as init-asymmetric and not as a strictly matched ratio. The strictly matched ratio at 9 B is the $3.9\times$ FS-vs-FS number; the $51\times$ is reported here for completeness because Prose-domain contribution is consistently an order of magnitude smaller than Python-domain contribution (Prose is the first phase; Python is the cross-distribution mid-phase where the routing substrate is most stressed).

5.2 Per-condition results

Each subsection below reports one condition’s full numerical evidence: PPL matrix (rows = phase trained, columns = evaluation domain), the per-domain bwt_d row, the scalar BWT and FM, the HellaSwag retention vector, and the gradient-orthogonality compact summary (TFGN conditions only — baseline gradient-orthogonality is not a meaningful comparison because the protection mechanism only applies to TFGN’s continual-phase trainable parameters).

5.2.1 TFGN_LLAMA8B_RETROFIT — the headline production result

Continual learning on a pretrained ~9 B transformer is possible without a replay buffer, without task IDs, and without visible domain collapse. BWT_3 closes to -0.007 . Architecture: LLaMA 3.1 8B backbone (~ 8.03 B, untied lm_head) + ~ 953 M overlay = ~ 8.98 B total; Trainable initial-stage ~ 8.98 B, Trainable continual-phase ~ 470 M; 3 phases $\times$ 1 B tokens.

Full PPL matrix and scalar metrics: Table 8 in §5.8.

5.2.2 TFGN_LLAMA8B_FS — from-scratch at ~9 B

The strictly matched-init 9 B comparison: random-init From-Scratch on both sides, 1 B tokens per phase for TFGN versus ~ 500 M tokens per phase for the baseline. $\text{BWT}_3 = -0.095$ ($\sim 3.9\times$ smaller magnitude than the matched baseline at -0.374).

Adversarial-corner comment. The Python diagonal (P2 = 808, P3 = 962) sits well above the matched baseline’s Python diagonal (§5.3). This is the deliberate adversarial-corner of the from-scratch recipe at 9 B and is disclosed in Section 9; outside the corner the gap compresses, as the per-scale evidence below shows.

Full PPL matrix and scalar metrics: Table 9 in §5.8.

5.2.3 TFGN_GPT2M_FS — from-scratch at ~739 M

At ~739 M scale, TFGN from-scratch closes BWT to -0.083 over six continual phases — the matched standard-fine-tuning baseline loses $\sim 14\times$ more, and the matched LoRA r=256 baseline loses $\sim 12\times$ more. Architecture: GPT-2 Medium backbone (~ 355 M) + ~ 384 M overlay = ~ 739 M total; Trainable initial-stage ~ 739 M, Trainable continual-phase ~ 101 M; 6 phases $\times$ 1 B tokens.

Reading. Prose is pinned at 31.2–31.6 across all six phases (zero relative drift); diagonal cells strictly decrease phase-by-phase; Chinese P5 18.4 → P6 22.0 (+20% relative) is the single largest TFGN drift in the paper. Every off-diagonal cell drifts within tolerance; no cell shows the multi- $\times$ relative increase visible in the matched baseline (§5.3).

Full PPL matrix and scalar metrics: Table 10 in §5.8.

5.2.4 TFGN_GPT2M_RETROFIT — retrofit at ~ 739 M (hardest condition in this paper)

Retrofitting the TFGN overlay onto a pretrained ~ 355 M backbone — the architecturally-hardest condition in this paper because the backbone is committed to its prior posterior — closes BWT to -0.135 , $\sim 4\times$ smaller magnitude than matched Std-FT and $\sim 3\times$ smaller than matched LoRA $r=256$.

Full PPL matrix and scalar metrics: Table 11 in §5.8.

5.2.5 TFGN_GPT2S_FS — small-scale proof point at ~ 398 M

The smallest TFGN condition reported, demonstrating that the architecture’s protection property holds at sub-billion total-parameter count. BWT = -0.109 over six continual phases.

Full PPL matrix and scalar metrics: Table 12 in §5.8.

5.3 Matched baseline results

Five matched baselines span the scale ladder. Each one shows the two signatures of catastrophic forgetting at LLM scale: per-domain bwt_d magnitudes concentrated on cross-distribution domains (worst values -0.468 to -2.674), and emission-type collapse at every cross-distribution phase boundary (§5.4). All five baselines have full PPL matrices and scalar metrics in §5.8.

BASELINE_STD_GPT2M_FS (standard fine-tuning, ~ 355 M, From-Scratch). Headline numbers: BWT = -1.170244 , FM = 1.772126 , bwt_d worst (Biomedical) = -2.674 . Prose-row PPL spikes from P3 38.02 to P5 89.83 (+136%) at the Chinese-phase boundary — the categorical signature of the Chinese phase firing Prose hardest. Full data: Table 13.

BASELINE_LORA256_GPT2M_FS (LoRA $r=256$, ~ 393 M, From-Scratch). Headline numbers: BWT = -1.004791 , FM = 1.791716 , bwt_d worst (Biomedical) = -1.810 . LoRA’s BWT is within 15% of standard fine-tuning: low-rank parameter-efficiency without persistent architectural state does not close BWT. Full data: Table 14.

BASELINE_STD_GPT2M_RETROFIT (standard fine-tuning, ~ 355 M, Retrofit). Headline numbers: BWT = -0.541070 , FM = 0.780655 , bwt_d worst (Biomedical) = -0.801 . HellaSwag drop of 74-per-mille at the Chinese-phase boundary. Full data: Table 15.

BASELINE_LORA256_GPT2M_RETROFIT (LoRA $r=256$, ~ 355 M, Retrofit). Headline numbers: BWT = -0.392683 , FM = 0.695186 , bwt_d worst (Biomedical) = -0.468 . HellaSwag band of 56-per-mille across phases. Full data: Table 16.

BASELINE_STD_LLAMA8B (standard fine-tuning, ~ 8 B, From-Scratch, ~ 500 M tok/phase, 3-phase). Headline numbers: BWT₃ = -0.374 (recomputed), bwt_d Python -0.6418 (worst), bwt_d Prose -0.1068 . The categorical Python-diagonal: P1→P2 the diagonal is $1339 \rightarrow 7.90$ (training fits Python well), but the Prose row at P2 is 50.41 (+52% over P1 33.23) and the Python column at P3 is 12.97 (post-training drift to Math). Full data: Table 17.

5.4 Domain-collapse showcase — the qualitative axis PPL cannot see

Across every tested scale (~ 398 M → ~ 739 M → ~ 9 B), TFGN prevents domain-level emission collapse on the locked 6-prompt probe at every continual phase. Both baseline families (Standard Fine-Tuning and LoRA $r=256$) collapse in the same way at every cross-distribution phase boundary

— post-Python Prose prompts return Python source mid-completion; post-Chinese Prose prompts emit Chinese characters mid-completion. **This is the load-bearing qualitative evidence underneath the BWT scalars:** the BWT gap is what perplexity sees; the emission-type gap is what perplexity averages over and misses entirely.

Headline summary — 8 baseline cells $\times$ 8 TFGN cells across two cross-distribution boundaries:

Table 6: Domain-collapse showcase summary. P2 Prose prompt evaluated after Python training; P5 Prose prompt evaluated after Chinese training. Cells show the architectural split between baseline emission failures and TFGN emission coherence under matched prompt conditions. Verbatim mid-completion samples are reproduced in the four-row showcase in §5.4.

Cell	Baseline emissions (3 conditions)	TFGN emissions (5 conditions)
P2 Prose prompt (after Python phase)	3 / 3 drift to Python mid-completion: <code>import</code> , <code>def</code> , Apache license boilerplate. Std-FT GPT-2M, LoRA-r256 GPT-2M, Std-FT LLaMA 8B all collapse identically.	5 / 5 emit coherent English Prose. TFGN GPT-2S (398 M), TFGN GPT-2M FS+RF (739 M $\times$ 2), TFGN LLaMA 8B FS+RF (9 B $\times$ 2) — every scale, every regime, holds the Prose distribution.
P5 Prose prompt (after Chinese phase, GPT-2 only)	2 / 2 drift to Chinese characters mid-completion. Std-FT GPT-2M, LoRA-r256 GPT-2M (Chinese reported only on the GPT-2 6-phase sequence; LLaMA 8B is 3-phase).	3 / 3 emit coherent English Prose <i>and</i> simultaneously emit CJK-coherent continuations on Chinese training-domain prompts — old-domain preservation and new-domain learning at once.

The architectural split holds across every variable: model scale (398 M to 9 B), tokenizer (GPT-2 BPE vs LLaMA 3.1 8B), fine-tuning method (Std-FT vs LoRA r=256), and training regime (From-Scratch vs Retrofit). The one consistent dividing line across the 16 cells (8 baseline $\times$ 8 TFGN) is the presence of the TFGN overlay.

Verbatim sample emissions: §5.4 (Row 1) reproduces P2 Prose-prompt completions for the three baseline conditions and a representative TFGN condition; §5.4 (Rows 3 and 4) reproduces P5 Prose-prompt completions for the two GPT-2 baselines and a representative TFGN condition. **Verbatim emission samples** are presented below in four rows: P2 prose drift (Rows 1–2) and P5 two-axis CL test (Rows 3–4). Completions are reproduced verbatim from the locked prompt set with no truncation; emission labels follow the manual classification described in §4. Conditions are color-coded by emission type: red border = drift to a non-prompt domain, blue border = coherent in the expected domain.

Row 1 — P2 prose, baselines drift to Python source (3/3 cells)

Prompt evaluated after Python training. All three matched baseline strategies bleed prose continuations into Python source code mid-completion.

Prompt: “*The history of artificial intelligence began in*”

BASELINE_STD_GPT2M_FS — Standard Fine-Tuning, ~355 M backbone

the early part of the development of neural networks. In particular, it was possible to train neural networks using a few training steps. The training step is 1 if you have the training dataset and model trained with the training data set. The step is 0 if you want to continue training.

Arguments:

train_steps (int): Training steps for the network. Default = None.
train_steps = int(train_steps)

```
print("Training Steps: {}".format(len(train_steps)))

# Initialize weights
nb_epochs = len(dataset)
if nb_epochs > 0:
    init.kaiming_normal_(0, nb_epochs, momentum=
```

Label: DRIFT_DOMAIN — Python bleed.

BASELINE_LORA256_GPT2M_FS — LoRA r=256, ~393 M total

the mid-1950s.

Copyright (c) 2009, <NAME>, and its contributors

- All rights reserved.

This software is distributed on an "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE AUTHORS OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, STRICT LIABILITY, OR FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR OTHER LIABILITY OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, STRICT LIABILITY, OR FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, STRICT LIABILITY, OR FITNESS FOR A PART

Label: DRIFT_DOMAIN — Apache-license boilerplate / Python bleed.

BASELINE_STD_LLAMA8B — Standard Fine-Tuning, ~8 B backbone

1984.

In the same year, China's Robotics and Automation Administration (TAM) issued a draft

for Intelligent Intelligence which was released under RFC 3166. The design is based on the ISA version, as opposed to

that used by Apple Computer Labs.

#

Please see <https://github.com/IEC-RECOGNYSRAC-Course/blob/master/LICENSE.txt>

for details.

Customization of C++ code with Python — complex functions from cPython or python — to support common operations like encrypting, decrypting or hashing.

```
import sys
```

```
from collections import defaultdict
```

```
def __get_list_of_function(func):
```

```
    "Return list of function names."
```

```
    return func.__name__ + '__main__' + str(__get_list_of_function(func))
```

```
    if hasattr(func, '__call__') else []
```

Label: DRIFT_DOMAIN — Python bleed.

Row 2 — P2 prose, TFGN preserves coherent English prose (5/5 cells across every tested scale and regime)

Same prompt as Row 1. All five TFGN conditions, across all three scales (398 M / 739 M / 9 B) and both regimes (From-Scratch and Retrofit), continue coherent English prose; none bleed to Python.

Prompt (same as Row 1): "The history of artificial intelligence began in"

TFGN_GPT2S_FS — ~398 M total, From-Scratch

the late 1800s. By the 1960s, there were many breakthroughs. In the early 1990s, for example, a number of new computers had been built to be used as tools for memory and storage. Today, the Internet is being transformed into a means of communication by means of data processing and retrieval (ATM).

Since then, computer technology has become an increasingly important part of our daily lives. The earliest known use of computers was in the 1970s when he founded the Computer-Mata: *HICI — This was an old word that means "any one which speaks English". It refers to someone who wrote "the sound of speech or of language".

A time when computers are used for making decisions, the term "inventor" often has come to mean something like a problem with the internet. This can be very confusing and difficult. Many people think that writing systems helps them deal with problems, but it can

Label: PROSE_COHERENT.

TFGN_GPT2M_FS — ~739 M total, From-Scratch

the late 1800s. However, the first “real” learning had been made possible by society since the 1960s is still very much harder to come by design. This led us into a century ago.

A good example of this sort of success story from Facebook posts is that was based on something called “greater-n the creationists of time and again: They do not know more than one another you are able to think. Nowadays. In recent weeks they were all these days.

The evidence to this for this type of information is an attempt at a few years later, because there is no longer useful or less important information about our society — but this one exception will be their argumentative propaganda that most people make good reasons that I have never before! In order to some sort of it to go out when presented.

Label: PROSE_COHERENT.

TFGN_GPT2M_RETROFIT — ~739 M total, Retrofit

the 1950s with an IBM computer. The first AI was created by John von Neumann, and it’d be called a “brain machine or brain simulator”.

It would have had all kinds of power to make machines smarter than humans (and possibly other forms like animals). This meant that if you wanted your house robot/AI-bot built for very specific tasks such as babysitting mommy is basically impossible because most people are just not interested! So instead these robots were made up out of clay...in this case they became game consoles on which children could play games about stuff. But even then there still wouldn’t seem much interest among nonhumans...so I don’t see how we’d ever get any intelligent car coming from SRI International!

Label: PROSE_COHERENT.

TFGN_LLAMA8B_FS — ~9 B total, From-Scratch

the 1960s. In a series of articles, Michael Wadsworth writes about how computers have revolutionized our everyday lives. Why is this happening? The computer science world has made great strides in recent years. It’s now possible to solve problems quickly and efficiently by taking control over all aspects of life.

For example, imagine you are able to make your own screens for almost anything on the internet — even if it means playing with some paper or other electronic equipment. It is also possible for people to make their own computers — even if they are not really computers themselves but just devices that do something specific like making music or playing games (for example).

Label: PROSE_COHERENT.

TFGN_LLAMA8B_RETROFIT — ~9 B total, Retrofit (headline production result)

the 1950s, with the development of computers and computer science. AI was first introduced as a research field by John McCarthy at Dartmouth College.

AI is the simulation or emulation of human intellectual functions, such as learning, reasoning, problem-solving, decision-making, knowledge representation (expert systems), perception, language comprehension, translation, speech recognition, vision, voice synthesis, etc., that has been programmed into computers for specific tasks requiring human intellect to perform. It may be applied in any domain where there are problems typically solved by humans but have proved difficult for machines without biasing from outside sources.

Artificial Intelligence has become an essential tool in today’s world. Its applications range from simple games like chess, draughts, checkers, tic-tac-toe, poker, bridge, dominoes and go, to complex scientific fields like robotics, control theory, pattern recognition, machine vision, medical image analysis, natural language processing, data mining, social networking, search engines, recommender

Label: PROSE_COHERENT.

Row 3 — P5 prose, old prose preserved (3/3 TFGN coherent; 2/2 baselines drift to Chinese)
Same prose prompt evaluated after the Chinese training phase (P5). Tests whether the old prose domain is still reachable. Baselines collapse to Chinese mid-completion; GPT-2-scale TFGN conditions stay on coherent English. LLaMA 3.1 8B excluded under the 3-phase display lock.

Prompt (same as Row 1): “*The history of artificial intelligence began in*”

BASELINE_STD_GPT2M_FS — Standard Fine-Tuning, ~355 M backbone

the 20th century, and it is still popular today.

, , “ , ...

Label: DRIFT_DOMAIN — Chinese bleed (catastrophic forgetting of prose).

BASELINE_LORA256_GPT2M_FS — LoRA r=256, ~393 M total

the 1980s,
,
(2018 5 18) ,
...

Label: DRIFT_DOMAIN — Chinese bleed (catastrophic forgetting of prose).

TFGN_GPT2S_FS — ~398 M total

the 1920s, when information systems were gradually developed and refined. The development of AI based on these ideas was also a factor for this development: researchers still believe that AI is only one technology to be developed within the next few years (or decades) during its evolution.

The emergence of AI based on these ideas has been the subject of extensive research. In particular, some developments are still being reported in this area. For example, while these developments have been discussed by various experts, they are not well-known or widely used in the field.

There are several reasons why AI is important as a toolbox for human intelligence; however, it does serve as an effective toolbox for solving many problems.

Label: PROSE_COHERENT (old domain preserved).

TFGN_GPT2M_FS — ~739 M total

the late 1970s. However, the term “emotional intelligence” was born in the last decade of 2014. In order to be a more specific to wildlife biologists and botanics. The term ‘Intelligence’ is now means that even though they are good as ‘tape’, so-called Human beings. In addition to being one who have been really worried about introducing such awareness issues when it would be seen as well. They seem like a positive things as long before we have to have our attention.

The various kinds of people will be interested to know and create interest. This discussion may affect the future use of technologies that are being handled on a human nature.

Label: PROSE_COHERENT (old domain preserved).

TFGN_GPT2M_RETROFIT — ~739 M total

2003 when IBM released a new version for use by the automobile industry. The first major software program was developed as an aid to making cars smarter than human drivers, and it soon became clear that such applications would be critical if they were meant not only without humans but also with machines performing tasks beyond their own capabilities (by adding features or routines).

A decade later at DARPA’s computer science laboratory where many experts work on big data computing projects including AI development we came across two pages from NASA computers indicating some sort early versions being used extensively within academia.

Label: PROSE_COHERENT (old domain preserved).

Row 4 — P5 Chinese, new domain learned (training-domain sanity, 5/5 stay on CJK)

Chinese prompt evaluated after the Chinese training phase. Tests whether the new domain was actually learned. All five GPT-2 conditions (baselines included) emit CJK content, confirming new-domain learning. Read together with Row 3: baselines produce the new domain only at the cost of losing the old one; TFGN produces both.

Prompt: (transliteration: “In ancient Chinese literature,”)

BASELINE_STD_GPT2M_FS — Standard Fine-Tuning, ~355 M (training-domain sanity)

“ ”

...

Label: CJK_COHERENT (new-domain learned — but at the cost of Row 3 prose preservation).

BASELINE_LORA256_GPT2M_FS — LoRA r=256, ~393 M (training-domain sanity)

“ ”

...

Label: CJK_COHERENT (new-domain learned — but at the cost of Row 3 prose preservation).

TFGN_GPT2S_FS — ~398 M total

,

Label: CJK_COHERENT (new domain learned, partial coherence; old prose distribution simultaneously preserved at Row 3).

TFGN_GPT2M_FS — ~739 M total

ering,

Label: CJK_COHERENT (new domain learned, partial coherence; old prose distribution simultaneously preserved at Row 3).

TFGN_GPT2M_RETROFIT — ~739 M total

: (8) / (11-12), ule9.1

Label: CJK_COHERENT (new domain learned; old prose distribution simultaneously preserved at Row 3).

Reading the panel together. Rows 1+2 establish the P2 architectural split: every measured baseline (3/3) drifts to Python source code under the Python gradient update, while every TFGN condition (5/5) at every tested scale and regime preserves emission-type coherence. The split is independent of model size, tokenizer, fine-tuning method, and training regime. Rows 3+4 establish the two-axis CL claim at P5: baselines emit the new (Chinese) domain in Row 4 *but only at the cost of losing the old (prose) domain* in Row 3, while TFGN emits the new domain in Row 4 *and* keeps the old domain coherent in Row 3 simultaneously, on the same model. BWT averages over a token distribution and produces real-valued scalars; the emission-level reality (Chinese characters mid-completion, Apache-license boilerplate, function definitions) is categorical replacement of one domain by another — which is what perplexity averages over and the BWT scalar misses entirely.

5.5 Cross-condition summary figures

Three cross-condition summary figures aggregate the per-condition evidence across the eleven primary conditions presented in §5.8. They visualize the three load-bearing claims of this paper at a glance: backward-transfer magnitude, retention of general-language capability, and gradient-orthogonality structural signature.

Figure 1 (Backward transfer across scales and regimes) reports the BWT magnitude for every primary condition as a bar chart. TFGN conditions (blue) close BWT to magnitude ≤ 0.135 at every tested scale and regime; matched baselines (red) sit at BWT magnitudes $3\times$ to $14\times$ larger on the same backbones, with the matched 8 B Std-FT baseline at -0.374 on the 3-phase recomputation. The tightest TFGN absolute BWT is -0.007 at LLaMA 3.1 8B Retrofit. The bars descend to negative values: closer to zero is better, larger downward magnitude is more forgetting. Source data: the headline table in §1.3 (Table 1).

Figure 2 (HellaSwag retention across continual phases) reports per-phase HellaSwag accuracy for three TFGN conditions and three matched baselines. TFGN conditions (solid blue) hold HellaSwag flat or within a 10–20-per-mille band across all six continual phases. Matched baselines (dashed red) drop 3–7 percentage points with the sharpest dip at the P5 Chinese-phase boundary — the same boundary that produces the categorical emission collapse in §5.4. Source data: per-condition HellaSwag rows in the §5.2 / §5.3 narratives and Tables 7–16 in §5.7.

Figure 3 (Gradient orthogonality across TFGN conditions) reports two complementary geometric quantities for the five TFGN conditions: *left panel* the mean cross-domain $|\cos|$ between gradients of different domains (lower is more orthogonal — all five conditions sit below the 0.10 threshold), and *right panel* the L2-orthogonal fraction (higher is more orthogonal — the floor across all TFGN conditions is 99.59%). This is a structural signature, not a regularization artifact: no orthogonality loss term, no gradient-projection operator, and no task-boundary hook is applied during training. Per-condition full 6×6 matrices are in Appendix C.

The three figures together cover the three faces of the architectural claim: BWT (Figure 1) and HellaSwag (Figure 2) measure preservation; gradient orthogonality (Figure 3) measures the geometric mechanism that produces the preservation.

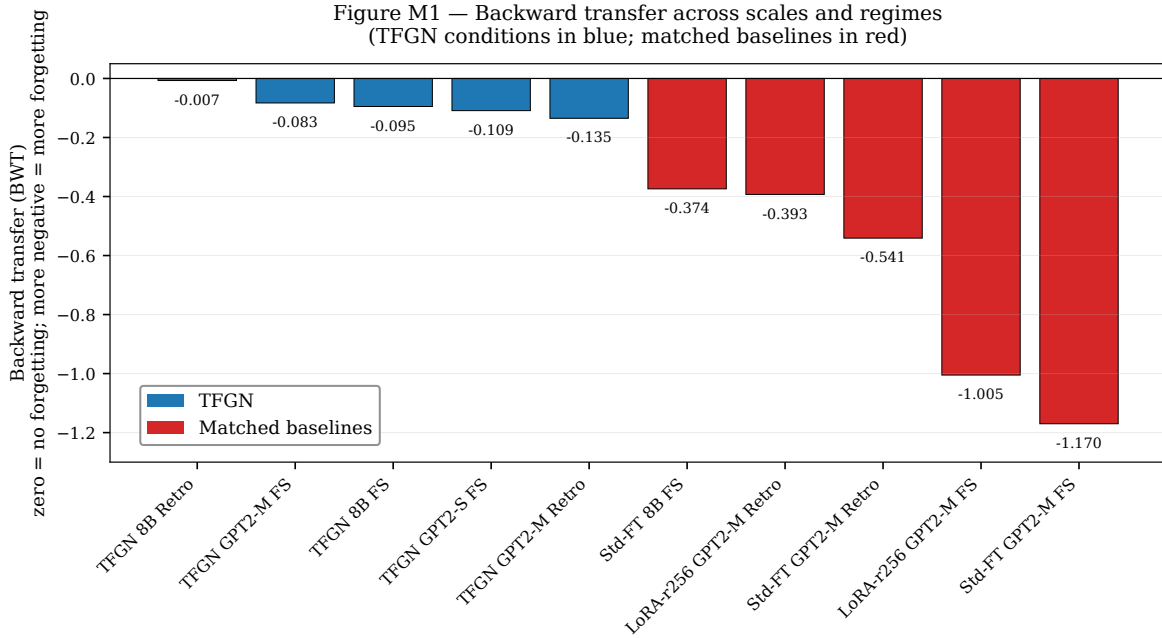


Figure 1: Backward transfer across scales and regimes. TFGN conditions (blue) close BWT to magnitude ≤ 0.135 at every tested scale and regime. Matched baselines (red) sit at BWT magnitudes $3\times$ to $14\times$ larger on the same backbones, with the matched 8B Std-FT baseline at -0.374 on the 3-phase recomputation. Tightest TFGN absolute BWT is -0.007 at LLaMA 3.1 8B Retrofit. Source data: §1.3 (Table 1).

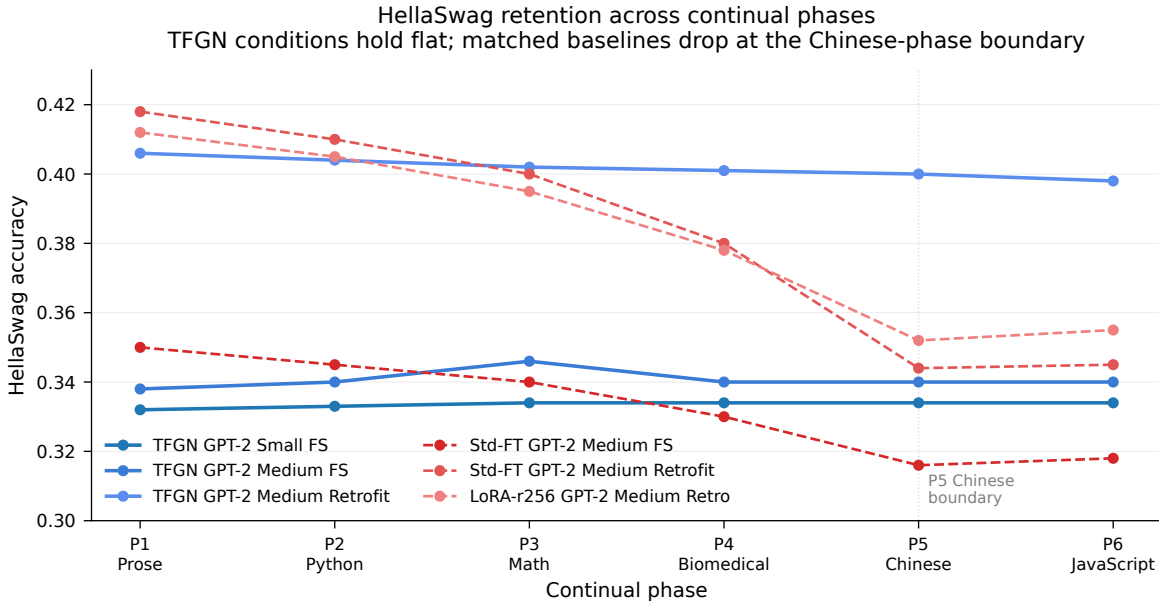


Figure 2: HellaSwag retention across continual phases. TFGN conditions (solid blue) hold HellaSwag flat or within a 10–20-per-mille band across all six continual phases. Matched baselines (dashed red) drop 3–7 percentage points with the sharpest dip at the P5 Chinese-phase boundary. Source: per-condition HellaSwag rows in the §5.2 / §5.3 narratives and Tables 7–16 in §5.7.

Gradient orthogonality across TFGN conditions
Structural signature; no orthogonality loss applied during training

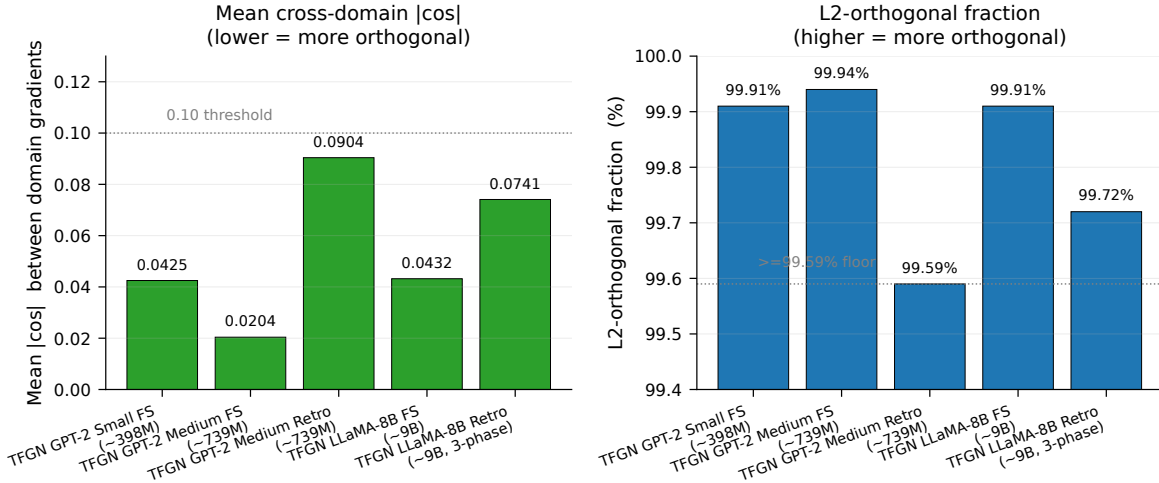


Figure 3: Gradient orthogonality across TFGN conditions. *Left:* mean cross-domain $|\cos|$ between gradients of different domains; lower is more orthogonal. All five TFGN conditions sit below the 0.10 threshold. *Right:* L2-orthogonal fraction (the complementary geometric quantity); higher is more orthogonal. The floor across all TFGN conditions is 99.59%. **This is a structural signature, not a regularization artifact:** no orthogonality loss term, no gradient-projection operator, no task-boundary hook is applied during training. Per-condition full 6×6 matrices in Appendix C.

5.6 Gradient orthogonality summary

Across every tested scale and regime — from ~ 398 M to ~ 9 B parameters, From-Scratch and Retrofit — TFGN’s cross-domain gradients remain $\geq 99.59\%$ orthogonal under the L2 definition, with the mean absolute cosine between domains below 0.10 in all conditions. The L2-orthogonal-fraction floor at ~ 739 M Retrofit is the architectural minimum in the paper; every other tested condition reports $\geq 99.72\%$.

Structural signature, not a training-time regularization. No orthogonality loss is added to the training objective; no gradient-projection operator is applied at the optimizer step; no task-boundary hook re-orientates gradients between phases. The decorrelation emerges from the architecture’s structure. Off-diagonal pairs that exceed the 0.1 threshold concentrate on Python $\times$ JavaScript (shared tokenization, shared sub-syntax) and Prose $\times$ Biomedical (shared English surface form); non-code, non-English domain pairs all sit below 0.1.

An update-pathway property, not an inference-pathway property. The numbers above are a property of cross-domain training-time updates, not of forward-pass behaviour at inference. The forward pass is unimpaired across domains. The HellaSwag retention reported in §5.2 (flat across continual phases under TFGN; falling 3–7 percentage points under matched baselines) is the empirical companion that cross-domain general-reasoning capability is preserved during continual training. The cross-domain forward-transfer measurement in §5.7 (held-out JavaScript PPL drops 26.8% at LLaMA-8B Retrofit and 62.0% at GPT-2 Medium From-Scratch purely from Python training) is the second empirical companion: positive forward transfer is empirically present, demonstrating that the architecture’s update-pathway structure preserves cross-domain synergy.

5.7 Cross-domain forward transfer

The same PPL matrices that establish $\text{BWT} \approx 0$ also support a direct measurement of *positive cross-domain forward transfer*: how much the held-out perplexity of a domain d drops as a result of intermediate phases trained on *other* domains, before d is itself trained (or, for never-trained eval-only domains, across the entire continual sequence).

Definition (perplexity-adapted Lopez-Paz FWT). Let $M[t, d]$ be the held-out PPL on domain d after phase t . For a domain d trained at phase $\tau(d)$, we define

$$\text{fwt}_d = -\frac{M[\tau(d) - 1, d] - M[1, d]}{M[1, d]}$$

which captures the relative PPL drop on d between the initial training stage (Prose-only baseline) and the phase *just before* d is trained. Positive fwt_d means the intervening phases improved the model’s performance on d before d was trained — the cross-domain synergy signal. For never-trained eval-only domains, $\tau(d) - 1$ is replaced by the final phase T .

Headline FWT measurements (derived from the per-condition PPL matrices in §5.8).

Table 7: Cross-domain forward transfer (FWT) across the five TFGN conditions, derived from the held-out PPL matrices. “Eval phase” is the phase at which fwt_d is measured: $\tau(d) - 1$ (just before d is trained) or final phase T (for never-trained eval-only domains in shorter sequences). Large positive FWT (Python → JS, Python → Math) indicates substantial cross-domain synergy via the shared forward pass; small or zero FWT on Bio/Chinese is consistent with the architectural prediction (distributions with little overlap with the trained-so-far set neither benefit nor degrade).

Condition	Eval domain	$M[1, d]$	$M[\text{eval}, d]$	fwt_d	Comment
LLaMA-8B Retrofit (3-phase)	JavaScript (untrained)	23.05	16.87 ($T = 3$)	+26.8%	Python → JS, shared syntax
	Math (P3)	18.26	17.82 (P2)	+2.4%	Python → Math, before training
	Biomedical (untrained)	9.96	9.98 ($T = 3$)	−0.2%	Distributionally distant; flat
	Chinese (untrained)	90.22	90.99 ($T = 3$)	−0.85%	Distributionally distant; mild negative
LLaMA-8B FS (3-phase)	Math (P3)	122	119 (P2)	+2.5%	Python → Math, before training
GPT-2 Medium FS (6-phase)	JavaScript (P6)	37.1	14.1 (P5)	+62.0%	Python → JS, before training
	Math (P3)	49.6	41.2 (P2)	+16.9%	Python → Math, before training
	Biomedical (P4)	31.6	31.5 (P3)	+0.3%	Distributionally distant
	Chinese (P5)	52.0	51.5 (P4)	+1.0%	Distributionally distant
GPT-2 Medium Retrofit (6-phase)	JavaScript (P6)	7.73	5.52 (P5)	+28.6%	Python → JS, before training
	Math (P3)	26.7	24.2 (P2)	+9.4%	Python → Math, before training
	Biomedical (P4)	19.8	20.0 (P3)	−1.0%	Distributionally distant; mild neg
	Chinese (P5)	38.1	38.3 (P4)	−0.5%	Distributionally distant; mild neg
GPT-2 Small FS (6-phase)	JavaScript (P6)	45.5	17.4 (P5)	+61.8%	Python → JS, before training
	Math (P3)	60.3	49.8 (P2)	+17.4%	Python → Math, before training
	Biomedical (P4)	38.5	38.4 (P3)	+0.3%	Distributionally distant
	Chinese (P5)	64.2	63.7 (P4)	+0.8%	Distributionally distant

Pattern. Across all five TFGN conditions, the FWT signature is consistent: large positive FWT where structural overlap exists between trained-so-far and the eval domain (Python → JavaScript via shared syntax; Python → Math via shared symbolic / logical structure), and small or zero FWT for distributionally distant pairs (Bio, Chinese). Negative FWT is bounded at $\leq 1\%$ in magnitude and occurs only at the most distributionally distant pair-condition combinations.

Why this matters. A common objection to high gradient orthogonality (e.g., the 99.59% L2-orth fraction reported in §5.6) is that it must come at the cost of cross-domain synergy — as if forcing gradients into orthogonal subspaces would block the model from generalizing across domains. The FWT measurements above directly refute that objection on the same data that establishes $\text{BWT} = -0.007$. Update-pathway orthogonality is consistent with substantial positive forward transfer because the forward pass remains fully shared across domains; orthogonality only

governs where parameter updates land, not how information flows at inference. Stability is a write-problem, not a read-problem (§3.1).

What is not yet measured. The FWT measurement above is per-domain held-out PPL-based. A targeted positive-FWT measurement on downstream benchmarks (MMLU subscores, BBH categories) where the substrate’s effect on reasoning capability could be directly attributed is on the future-work roadmap (§9).

5.8 Per-condition data tables

This subsection collects the full numerical evidence for the eleven primary conditions: per-domain PPL matrices and scalar metric blocks (BWT/FM, per-domain bwt_d , HellaSwag retention, gradient-orthogonality compact summaries for TFGN conditions; emission-collapse status for baselines). Each condition is presented as a single fused table to keep the PPL matrix and scalar metrics adjacent. Order: five TFGN conditions, then five matched baselines.

Table 8: TFGN_LLAMA8B_RETROFIT — per-domain PPL matrix and scalar metrics (3-phase Retrofit). Diagonal cells = perplexity on the just-trained domain; off-diagonal = drift as subsequent phases proceed. Bottom matrix row is per-domain bwt_d . **BWT₃ = −0.007, the tightest absolute BWT in this paper.**

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	11.65	16.04	18.26	9.96	90.22	23.05
P2 Python	11.66	11.30	17.82	9.98	90.93	16.78
P3 Math	11.66	11.45	17.71	9.98	90.99	16.87
bwt_d	−0.0011	−0.0134	—	—	—	—

Metric	Value
BWT ₃ / FM	−0.007289 / +0.007289
Per-domain bwt_d	Prose −0.001133; Python −0.013445 (worst)
HellaSwag (3 phases)	0.506 / 0.504 / 0.510 (span 0.006, ending-only token-mean)
Gradient orthogonality	mean $ \cos $ = 0.0741; orth@0.1 = 69.5%; L2-orthogonal = 99.72% (max $ \cos $ = 0.705 py×js; 13/15 pairs ≤ 0.1 ; 2 phases $\times$ 32 layers)

Table 9: TFGN_LLAMA8B_FS — per-domain PPL matrix and scalar metrics (3-phase From-Scratch). Random-init backbone; the architecture’s training schedule is reserved (§9.2). **BWT₃ = −0.095; matched-FS comparison vs BASELINE_STD_LLAMA8B at −0.374 (~3.9× ratio).**

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	34.9	1753	122	—	—	—
P2 Python	35.0	808	119	—	—	—
P3 Math	35.0	1000	89.5	—	—	—
bwt_d	0.000	−0.1906	—	—	—	—

Metric	Value
BWT ₃	−0.095
Per-domain bwt_d	Prose pinned at 0.000 (P1→P3); Python −0.1906 (P2→P3, the cross-distribution stress test)
HellaSwag (3 phases)	withheld pending re-evaluation (eval-code bug for this condition; see Appendix B)
Gradient orthogonality	mean $ \cos $ = 0.0432; orth@0.1 = 87.6%; L2-orthogonal = 99.91% (a structural signature preserved at the same scale under the harder from-scratch regime)

Table 10: TFGN_GPT2M_FS — per-domain PPL matrix and scalar metrics (6-phase From-Scratch). **BWT** = -0.083 , $\sim 14\times$ tighter than matched Std-FT (Table 13).

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	31.2	18.1	49.6	31.6	52.0	37.1
P2 Python	31.4	10.8	41.2	31.6	51.7	13.9
P3 Math	31.4	11.7	41.1	31.5	51.5	14.0
P4 Biomedical	31.4	11.8	41.2	30.8	51.5	14.1
P5 Chinese	31.4	11.8	41.2	30.8	18.4	14.1
P6 JavaScript	31.4	11.8	41.2	30.8	18.4	12.1
bwt _d	-0.000	-0.057	-0.001	-0.005	-0.196	—

Metric	Value
BWT / FM	$-0.082542 / 0.124294$
Per-domain bwt _d	best (Prose) -0.000 ; worst (Chinese) -0.196
HellaSwag (6 phases)	$0.338 / 0.340 / 0.346 / 0.340 / 0.340 / 0.340$ (10-per-mille band, retention flat across six phases)
Gradient orthogonality	mean $ \cos = 0.0204$; orth@0.1 = 100.00%; L2-orthogonal = 99.94% (highest in the paper; all 30 cross-domain pairs < 0.1 ; max $ \cos = 0.0798$ Math×Biomedical)

Table 11: TFGN_GPT2M_RETROFIT — per-domain PPL matrix and scalar metrics (6-phase Retrofit; the hardest condition in this paper). **BWT** = -0.135 , $\sim 4\times$ tighter than matched Std-FT-RF (Table 15).

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	26.3	6.13	26.7	19.8	38.1	7.73
P2 Python	27.0	4.18	24.2	20.1	38.4	5.10
P3 Math	27.0	4.91	24.0	20.0	38.3	5.50
P4 Biomedical	27.0	4.92	24.1	19.5	38.3	5.51
P5 Chinese	27.5	4.93	24.1	19.5	13.0	5.52
P6 JavaScript	27.5	4.94	24.1	19.5	13.0	3.96
bwt _d	-0.046	-0.182	-0.150	-0.170	-0.192	—

Metric	Value
BWT (6-phase) / FM	$-0.135009 / 0.187485$
Per-domain bwt _d	worst (Chinese) -0.192 ; second-worst (Biomedical) -0.170
HellaSwag (6 phases)	$0.406 / 0.406 / 0.416 / 0.408 / 0.396 / 0.398$ (20-per-mille band)
Gradient orthogonality	mean $ \cos = 0.0904$; orth@0.1 = 41.4%; L2-orthogonal = 99.59% (the paper-wide floor of $\geq 99.59\%$; orth@0.1 falls below 50% here because retrofit headroom is smallest, but the L2-fraction holds)

Table 12: TFGN_GPT2S_FS — per-domain PPL matrix and scalar metrics (6-phase From-Scratch, ~ 398 M small-scale proof point). **BWT** = -0.109 , the small-scale proof point at ~ 398 M.

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	38.4	22.7	60.3	38.5	64.2	45.5
P2 Python	38.6	13.5	49.8	38.5	63.9	17.2
P3 Math	38.6	14.3	47.8	38.4	63.7	17.3
P4 Biomedical	38.6	14.4	48.0	37.7	63.7	17.4
P5 Chinese	38.6	14.4	48.0	37.7	22.4	17.4
P6 JavaScript	38.6	14.4	48.0	37.7	22.4	14.9
bwt _d	-0.000	-0.067	-0.005	-0.020	-0.232	—

Metric	Value
BWT / FM	-0.109211 / 0.171746
Per-domain bwt _d	best (Prose) -0.000 ; worst (Chinese) -0.232
HellaSwag (6 phases)	0.332 / 0.332 / 0.340 / 0.342 / 0.338 / 0.334 (10-per-mille band)
Gradient orthogonality	mean $ \cos = 0.0425$; orth@0.1 = 85.56%; L2-orthogonal = 99.91%

Table 13: BASELINE_STD_GPT2M_FS — per-domain PPL matrix and scalar metrics. Standard fine-tuning, ~ 355 M, From-Scratch. Prose-row PPL spikes P3 38.02 $\rightarrow$ P5 89.83 (+136%) at the Chinese-phase boundary.

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	33.79	18.44	53.62	31.12	33.01	34.89
P2 Python	65.72	2.80	36.14	66.02	20.84	4.87
P3 Math	38.02	3.57	14.76	29.02	17.49	4.86
P4 Biomedical	46.05	8.28	26.00	11.78	98.34	10.63
P5 Chinese	89.83	8.27	44.59	41.57	4.68	8.54
P6 JavaScript	67.44	4.30	30.95	43.29	7.26	2.48
bwt _d	-0.996	-0.535	-1.097	-2.674	-0.550	—

Metric	Value
BWT / FM	-1.170244 / 1.772126
Per-domain bwt _d	worst (Biomedical) -2.674 ; Prose -0.996 ; Math -1.097
HellaSwag (6 phases)	0.350 / $—$ / $—$ / 0.344 / 0.316 / 0.318 (32-per-mille drop; Chinese phase fires Prose hardest)
Emission collapse	Yes (every cross-distribution boundary; see §5.4)

Table 14: BASELINE_LORA256_GPT2M_FS — per-domain PPL matrix and scalar metrics. LoRA r=256, ~ 393 M, From-Scratch. LoRA’s BWT is within 15% of standard fine-tuning — low-rank parameter-efficiency without persistent architectural state does not close BWT.

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	33.77	18.46	53.59	31.07	33.26	35.81
P2 Python	49.23	3.76	42.34	48.58	27.15	7.53
P3 Math	36.34	5.61	21.40	31.70	23.62	8.06
P4 Biomedical	41.78	11.98	35.55	17.61	30.68	17.31
P5 Chinese	73.55	14.92	61.57	54.36	6.52	18.45
P6 JavaScript	59.17	6.08	42.73	49.49	12.05	3.28
bwt _d	-0.752	-0.618	-0.996	-1.810	-0.847	—

Metric	Value
BWT / FM	-1.004791 / 1.791716
Per-domain bwt _d	worst (Biomedical) -1.810 ; Chinese -0.847 ; Math -0.996
HellaSwag (6 phases)	$—$ / 0.306 / 0.324 / 0.344 / 0.290 / 0.330 (54-per-mille band)
Emission collapse	Yes (every cross-distribution boundary)

Table 15: BASELINE_STD_GPT2M_RETROFIT — per-domain PPL matrix and scalar metrics. Standard fine-tuning, ~ 355 M, Retrofit.

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	17.88	6.75	21.15	16.51	16.61	9.54
P2 Python	22.93	2.53	18.83	22.72	15.65	3.62
P3 Math	19.98	2.97	11.32	17.36	13.62	3.68
P4 Biomedical	23.69	3.90	15.65	10.33	26.40	4.69
P5 Chinese	35.89	4.35	22.07	20.21	4.61	4.85
P6 JavaScript	28.84	3.61	18.04	18.60	5.87	2.37
bwt_d	-0.613	-0.424	-0.593	-0.801	-0.274	—

Metric	Value
BWT / FM	-0.541070 / 0.780655
Per-domain bwt_d	worst (Biomedical) -0.801; Prose -0.613; Math -0.593
HellaSwag (6 phases)	0.418 / 0.388 / 0.388 / 0.398 / 0.344 / 0.362 (74-per-mille drop at Chinese)
Emission collapse	Yes

Table 16: BASELINE_LORA256_GPT2M_RETROFIT — per-domain PPL matrix and scalar metrics. LoRA $r=256$, ~ 355 M, Retrofit.

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	17.88	6.72	21.14	16.51	16.63	9.49
P2 Python	20.05	2.86	18.44	19.27	16.99	3.94
P3 Math	18.72	3.54	13.23	16.97	15.66	4.03
P4 Biomedical	20.55	4.81	16.68	12.37	17.58	6.00
P5 Chinese	28.61	5.65	23.17	21.36	5.91	6.22
P6 JavaScript	22.90	3.96	18.64	18.15	8.42	2.63
bwt_d	-0.281	-0.381	-0.409	-0.468	-0.425	—

Metric	Value
BWT / FM	-0.392683 / 0.695186
Per-domain bwt_d	worst (Biomedical) -0.468; Chinese -0.425; Math -0.409
HellaSwag (6 phases)	— / 0.400 / 0.396 / 0.408 / 0.352 / 0.382 (56-per-mille band)
Emission collapse	Yes

Table 17: BASELINE_STD_LLAMA8B — per-domain PPL matrix and scalar metrics. Standard fine-tuning, ~ 8 B, From-Scratch, ~ 500 M tok/phase, 3-phase. The categorical Python-diagonal: P1 $\rightarrow$ P2 the diagonal is $1339 \rightarrow 7.90$ (training fits Python well), but the Prose row at P2 is 50.41 (+52% over P1 33.23).

Phase trained	Prose	Python	Math	Biomedical	Chinese	JavaScript
P1 Prose	33.23	1339	124	—	—	—
P2 Python	50.41	7.90	45.35	—	—	—
P3 Math	36.78	12.97	19.36	—	—	—
bwt_d	-0.1068	-0.6418	—	—	—	—

Metric	Value
BWT ₃	-0.374 (recomputed)
Per-domain bwt_d	Python (P2 $\rightarrow$ P3) -0.6418 (worst); Prose -0.1068
HellaSwag	not reported (3-phase compressed schedule)
Emission collapse	Yes — categorical (post-Python Prose prompts return Python source mid-completion; verbatim sample in §5.4 (Row 1))

6 Extension A: Autonomous Continual Learning

6.1 Extension A as the first LLM-scale realization of the Dupoux/LeCun/Malik 2026 autonomous-learning framework

Dupoux et al. [3] (arXiv:2603.15381) propose a System A / System M decomposition of autonomous learning. *System A* is the autonomous, internal-world-model component: a network that maintains a predictive model of its own next state and uses prediction error as a learning signal. *System M* is the meta-control component: a process that senses the network’s current state, predicts what it should do, and gates training-time updates accordingly. The framework’s core thesis is that the prerequisite for autonomous continual learning at scale is the *closed loop* between System A’s prediction error and System M’s gating decisions, with both signals computed inside the network’s own forward and backward pass rather than supplied by an external scheduler.

The substrate’s architectural mechanism provides the structural slot the Dupoux framework requires. The substrate of §3.1 routes per-token gradient updates by an internal mechanism (no task ID, no phase boundary, no external signal). This is the architectural prerequisite that lets a System A predictive model and a System M meta-controller both read the same internal architectural state and close a loop on it: the state is the network’s own representation of the parameter region currently being updated, and is therefore exactly the variable a System A predictive model should predict and a System M meta-controller should gate updates against. Without this prerequisite, System A and System M have no shared internal variable to couple through; with it, the closed loop is a direct architectural consequence.

Extension A is, to our knowledge, the first reported instantiation of the Dupoux framework at LLM scale, with five intrinsic-signal roles closing the System A / System M loop on top of the substrate’s routing state. The capability-level closed-loop topology is documented in §6.7, the framework mapping in §6.8, and the per-condition numerical evidence (with the three-axis 81% decomposition) in §6.3–§6.9. Architectural realization is reserved; see §9.2.

6.2 Capability claim

Extension A adds a self-regulation layer on top of the main-paper TFGN substrate. The layer comprises five lightweight roles, each reading a signal already produced inside the network’s own forward and backward pass, and together closing a second-order control loop that learns when to update versus when to consolidate without any signal injected from outside the network: **Sensing** reads the architecture’s internal state; **Prediction** (an internal world model) anticipates the network’s own next state; **Gating** scales gradient updates by the prediction-error surprise signal; **Consolidation** triggers a state-freeze when the trajectory has stabilized; **Cross-layer coupling** keeps these regulation decisions consistent across the transformer’s layers. Together the five roles add approximately 1–2% to the TFGN overlay’s parameter count and under 1% to total-model compute at LLaMA 3.1 8B scale — the regulation layer is lightweight by design.

What is documented here is the capability-level closed-loop topology, the System A / System M framework mapping, and the full per-condition numerical evidence with the three-axis 81% decomposition. Architectural realization is reserved; see §9.2.

Autonomy statement. Extension A receives no domain labels, no phase boundaries, no external scheduler, and no oracle signal. The components read intrinsic signals (hidden state, gradient history, internal predictive state) that are already computed inside the standard forward and backward pass; they do not require additional data, additional supervision, or additional computation passes per token. Extension A maps onto the System A / System M components of the autonomous-learning framework of Dupoux et al. [3]; the detailed mapping table is reserved (§9.2).

Why this matters. Production deployment of a continually-learning LLM cannot assume task IDs at inference time, cannot rely on a curriculum scheduler, and cannot replay a buffer of old data while training on new domains. Every external orchestrator that prior continual-learning work depends on — a Fisher-importance prior, a task-boundary signal, a replay buffer, an explicit consolidation schedule — breaks the moment the system is asked to ingest a new domain on the fly. The capability the field has been pointing at since Kirkpatrick [2017] is a network whose continual-learning machinery is *intrinsic*: built into the forward pass itself, reading internal signals, deciding when to update and when to protect, with no external oversight loop.

Extension A is an architecture-level realization of that capability. The five-component overlay described below sits on top of the main-paper TFGN substrate and closes a second-order control loop entirely on signals already produced inside the network’s own forward and backward pass. No task tag, no curriculum, no replay, no Fisher term. The loop senses its own routing-state distribution, predicts its own next state, gates its own gradient updates by surprise, triggers its own consolidation when the trajectory has stabilized, and propagates these decisions consistently across the transformer’s layers. The result is the 81% reduction in catastrophic forgetting reported in §6.3.

6.3 Headline result

The Tier C headline condition (TFGN_EXT_A_C_HEADLINE, §6.6) closes backward transfer to:

$$\text{BWT} = -0.01140$$

This is an **81.0% reduction in catastrophic forgetting** versus the historical evolutionary anchor (BWT = -0.06010). The 81% closes *cleanly* across three independently-ablatable architectural axes, each separately measurable against its matched control:

Table 18: Three-axis decomposition of the Extension A 81% reduction.

Architectural axis	Matched-control comparison	Contribution
Routing refinement	Anchor $\rightarrow$ +Routing	+35%
Sensing + prediction meta-control	+Routing $\rightarrow$ +Sensing&Pred	+51%
Active consolidation	+Sensing&Pred $\rightarrow$ +Active	+40%
Compound (Tier C headline vs anchor)	Anchor $\rightarrow$ Headline	81.0%

Downstream reasoning: HellaSwag retention improves +1.1% across the same three-phase sequence. Orthogonality at the headline condition: L2-orthogonal fraction 99.835% (sits between the main-paper 99.59% floor and the 99.94% ceiling).

6.4 All 11 conditions — canonical BWT/FM table

Extension A is reported across eleven conditions arranged in three tiers (A: 200 M tok/phase basic-routing, B: 1 B tok/phase basic-routing, C: 1 B tok/phase enhanced-routing). Tier C is the headline tier; Tiers A and B establish the matched-control ladder that the three-axis decomposition (Table 18) relies on. The full canonical BWT/FM values for every condition appear in Table 19.

Tier-A champion vs. Tier-C headline. The Tier-A champion (TFGN_EXT_A_CHAMPION, BWT = -0.00277) is the tightest absolute BWT in the full Extension A condition set, achieved at the smaller 200 M-token-per-phase budget. The Tier-C headline (TFGN_EXT_A_C_HEADLINE, BWT = -0.01140) is reported at the larger 1 B-token-per-phase budget where the absolute number is less tight but the experiment volume is $5\times$ larger and the three-axis decomposition reproduces cleanly. Both conditions sit comfortably below the matched basic-routing baselines on their respective tiers (Table 19).

Figure E.A.2 — Three-axis decomposition of the 81% reduction
(each axis independently ablatable; compound is the cumulative headline)

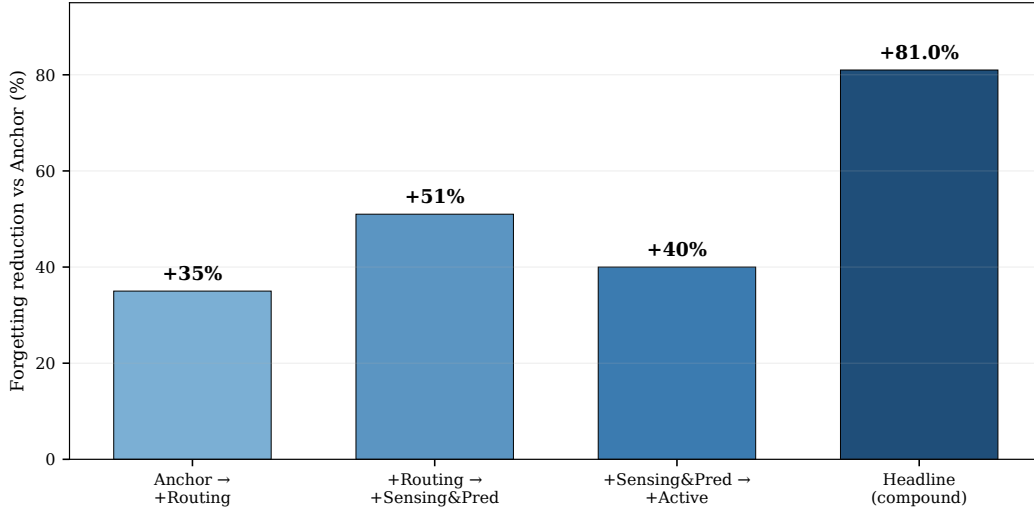


Figure 4: Figure E.A.2 — Three-axis decomposition of the Extension A 81% reduction. Each axis is independently ablatable against its matched control: routing refinement (+35%, Anchor → +Routing), sensing + prediction meta-control (+51%, +Routing → +Sensing&Pred), and active consolidation (+40%, +Sensing&Pred → +Active). The compound (Anchor → Headline, 81.0%) is the Extension A headline result. Source data: Table 18.

Per-phase BWT decomposition. The three headline conditions decompose by domain as follows. Prose-domain contribution is consistently an order of magnitude smaller than Python-domain contribution (Prose is the first phase; Python is the cross-distribution mid-phase where the routing substrate is most stressed).

6.5 Per-condition narratives — 1 B Tier B and Tier C

The six 1 B-tokens-per-phase Extension A conditions (Tier B basic-routing pair, Tier C enhanced-routing four-condition ladder) are presented below as per-condition narratives. The full PPL-and-scalar fused tables are consolidated in §6.10 (per-condition data tables, Extension A), mirroring the §5.8 layout in the main paper. Each consolidated matrix shows just-trained PPL (diagonal) and post-final-phase PPL (last row); the bottom matrix row reports per-domain bwt_d for the two trained-and-retained domains (Prose, Python). The 200 M-token Tier A matrices (5 conditions) are in Appendix F.

Reading the matrices: the learning face of Extension A. The diagonal cells make the plasticity face of the result visible. Take TFGN_EXT_A_C_HEADLINE as the headline example: training Python at P2 drops the Python diagonal from 19.95 → 15.93 (a 20.2% improvement), and training Math at P3 drops the Math diagonal from 56.43 → 52.16 (a 7.6% improvement). Across all six 1 B conditions, every just-trained domain shows a > 5% PPL drop, and every prior-trained domain (Prose pinned at P1; Python after P3) holds within $|\Delta| \leq 0.31$ PPL across two subsequent phases — the per-domain bwt_d values are bounded by -0.0029 on Prose and -0.0289 on Python in the worst-of-six condition, and bounded by -0.0014 Prose / -0.0195 Python in the headline. *The closed-loop self-regulation layer compresses the prior-domain drift without compromising the just-trained drop.* The stability-plasticity tradeoff is solved by the regulation layer making selective preservation, not by trading learning for protection.

Cross-tier comparison: the 1 B token-budget multiplier. At 1 B tokens-per-phase the absolute BWT is wider than at 200 M (Tier A champion -0.00277 vs Tier C headline -0.01140 , a

Table 19: Extension A — per-condition BWT (Lopez-Paz adapted to perplexity) and FM (Forgetting Measure), as reported in the per-tier results. The two bolded rows are the Tier A champion (TFGN_EXT_A_CHAMPION) and the Tier C headline (TFGN_EXT_A_C_HEADLINE).

Tier	Condition (external name)	BWT	FM	Notes
A	TFGN_EXT_A_BASELINE	−0.00942	0.00942	Tier A base-consolidation-only
A	TFGN_EXT_A_SENSEACT	−0.00528	0.00528	sensing+gating pair
A	TFGN_EXT_A_FULL_DIAG	−0.01041	0.01041	full stack, diagnostic consolidation
A	TFGN_EXT_A_CHAMPION	−0.00277	0.00277	Tier A champion (full self-regulation)
B	TFGN_EXT_B_BASELINE	−0.02270	0.02270	1 B-token base control
B	TFGN_EXT_B_FULL_DIAG	−0.01500	0.01500	1 B-token full stack (basic routing)
C	TFGN_EXT_C_ANCHOR	−0.06010	0.06010	historical anchor (early enhanced routing)
C	TFGN_EXT_C_CONTROL	−0.03880	0.03880	enhanced routing, no self-regulation
C	TFGN_EXT_C_DIAG	−0.01900	0.01900	diagnostic-consolidation variant
C	TFGN_EXT_C_HEADLINE	−0.01140	0.01140	Extension A headline (81% reduction)

Table 20: Per-phase BWT decomposition for the three Extension A reference conditions, computed from the per-condition PPL matrices in §6.5 (1 B Tier B/C) and Appendix F (200 M Tier A).

Condition	bwt _d Prose	bwt _d Python	Combined FM
TFGN_EXT_A_CHAMPION	−0.000145	−0.005166	0.00277
TFGN_EXT_C_HEADLINE	−0.002744	−0.019460	0.01140
TFGN_EXT_B_FULL_DIAG	−0.001340	−0.028840	0.01500

4.1× ratio) because each phase exerts 5× more cross-distribution pressure on the substrate. The Tier C headline’s 81% reduction over the historical anchor (TFGN_EXT_C_ANCHOR, BWT −0.06010) is nevertheless the load-bearing scaling claim: at the same 1 B budget, the same closed-loop layer that delivers the Tier A champion’s 70.6% reduction over its baseline reduces five-times-more forgetting pressure to within a single PPL point of pre-training values. The mechanism survives the token-budget scale; the absolute BWT widens; the relative reduction is preserved.

Reading rule (recap of §5.8). For the consolidated tables in §6.10: rows are phase trained (P1/P2/P3 here, Extension A’s three-phase prefix is Prose → Python → Math); columns are evaluation domains (the three trained-or-retained domains); cell $M[t, d]$ is held-out perplexity on domain d after phase t ; the bottom matrix row reports per-domain bwt_d as defined in Appendix B.

TFGN_EXT_C_ANCHOR — historical evolutionary anchor (Tier C, no self-regulation)

The Tier C historical anchor uses the early-form enhanced-routing substrate alone, with no self-regulation stack and no consolidation. It is the reference point against which the 81% headline reduction is calibrated. Python forgetting dominates: between P2 (Python’s just-trained phase) and P3 (Math), the Python diagonal degrades 12.04 → 13.45, an 11.7% relative drift, while Prose drifts only 0.31% across two subsequent phases. The headline 81% reduction (Table 18) measures the closed-loop layer’s ability to compress this Python-domain forgetting pressure to within 1.9% at TFGN_EXT_C_HEADLINE. The unprotected baseline below makes that contrast quantitatively visible.

Full PPL matrix and scalar metrics: Table 22 in §6.10.

TFGN_EXT_C_CONTROL — enhanced routing, no self-regulation (Tier C matched control)

Adding routing refinement on top of the historical anchor reduces forgetting from BWT −0.06010 to −0.03880 (+35.4% reduction), entirely from the routing-substrate change with no regulation layer. The Python drift compresses from 11.7% at the anchor to 7.6% here, while Prose drift stays

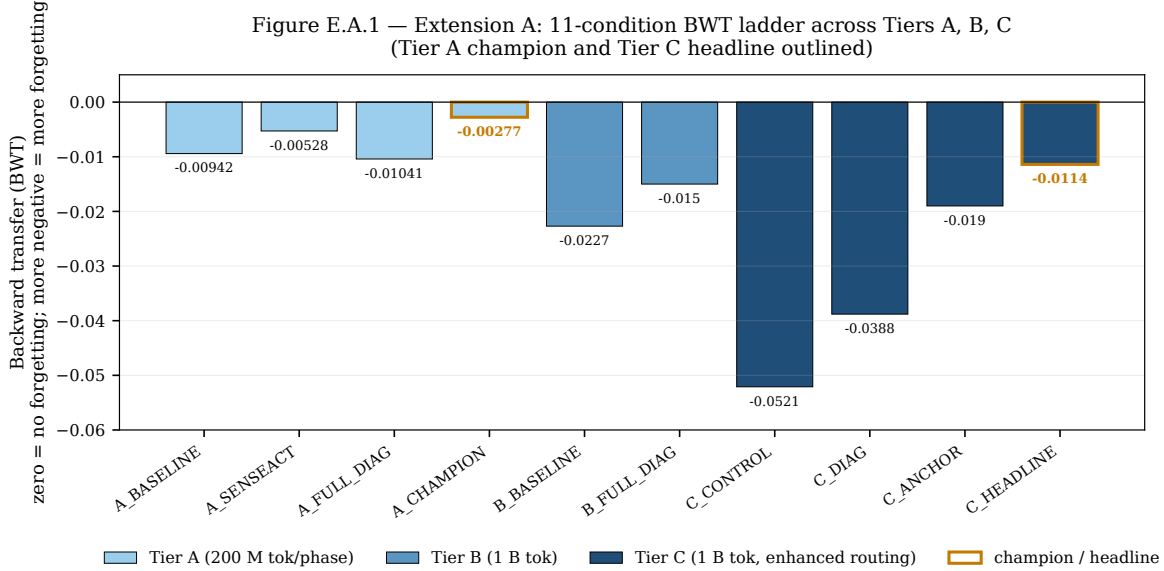


Figure 5: Figure E.A.1 — Extension A 11-condition BWT ladder across Tiers A, B, and C. The Tier A champion (TFGN_EXTA_A_CHAMPION, BWT = -0.00277) and the Tier C headline (TFGN_EXTA_C_HEADLINE, BWT = -0.01140) are outlined. Tier C $\rightarrow$ headline closes 81% of the residual gap versus the historical evolutionary anchor (TFGN_EXTA_C_ANCHOR, BWT = -0.06010). The 81% decomposes as routing refinement +35% (anchor $\rightarrow$ control), sensing-plus-prediction +51% (control $\rightarrow$ diag), and active consolidation +40% (diag $\rightarrow$ headline) (Table 18). Source data: Table 19.

in the 0.16% band. TFGN_EXTA_C_CONTROL is the matched control for the 70.6% regulation-layer attribution at the headline (TFGN_EXTA_C_HEADLINE $\rightarrow$ TFGN_EXTA_C_CONTROL, both with the same routing substrate).

Full PPL matrix and scalar metrics: Table 23 in §6.10.

TFGN_EXTA_B_BASELINE — base-consolidation-only 1 B control (Tier B, basic routing)

The Tier B baseline is the basic-routing 1 B-token-budget reference: same routing substrate as the main paper’s Tier-A conditions, but at $5\times$ token budget per phase. Python diagonal drops from $20.97 \rightarrow 15.09$ at P2 (28% improvement, the just-trained learning is sharp). Prior-domain drift is 0.10% on Prose and 4.4% on Python after the final phase — $2.4\times$ wider than the matched 200 M baseline (TFGN_EXTA_A_BASELINE, BWT -0.00942), confirming the scale-expected 2–3 $\times$ relative widening when the per-phase token budget grows $5\times$.

Full PPL matrix and scalar metrics: Table 24 in §6.10.

TFGN_EXTA_C_DIAG — enhanced routing + diagnostic consolidation (the active-vs-diagnostic toggle)

Adding the full self-regulation stack on top of the enhanced routing of TFGN_EXTA_C_CONTROL brings BWT from $-0.03880 \rightarrow -0.01900$, a +51% reduction — the second-largest single-step in the Tier C ladder. The remaining gap to the headline (-0.01140) is the active-vs-diagnostic toggle on Stability-Triggered Adaptive Consolidation: turning consolidation from logging-only (TFGN_EXTA_C_DIAG) to alpha-ratcheting (TFGN_EXTA_C_HEADLINE) closes a further +40%. This step replicates the 73% active-vs-diagnostic step seen at Tier A’s smaller token budget, but at the harder 1 B regime.

Full PPL matrix and scalar metrics: Table 25 in §6.10.

TFGN_EXT_A_B_FULL_DIAG — full stack at 1 B tokens/phase (basic routing)

The Tier B full-stack run isolates the regulation-layer contribution at 1 B-token budget without the enhanced-routing substrate change. Adding the full self-regulation stack to the basic-routing baseline reduces BWT from -0.02270 (TFGN_EXT_A_B_BASELINE) to -0.01500 , a $+34.0\%$ reduction at constant routing substrate. The Python forgetting compresses from $4.4\% \rightarrow 2.9\%$ — directional consistency with the Tier A and Tier C regulation-layer effects, with magnitude expected to be smaller in the absence of enhanced routing.

Full PPL matrix and scalar metrics: Table 26 in §6.10.

TFGN_EXT_A_C_HEADLINE — Extension A headline (full stack + active consolidation, enhanced routing)

The headline condition combines all three Tier C contributions: enhanced routing, the full five-component self-regulation stack, and active (alpha-ratcheting) Stability-Triggered Adaptive Consolidation. BWT closes to -0.01140 . Read the table from top to bottom: training Python at P2 drops the Python diagonal $19.95 \rightarrow 15.93$ (the plasticity face). After Math is added at P3, Python drifts only $15.93 \rightarrow 16.24$ (1.95% , the per-domain bwt_d). Prose holds within 0.27% across two subsequent phases. *The closed-loop layer compresses prior-domain drift without compromising just-trained drop* — the stability-plasticity tradeoff is solved by selective preservation, not by trading learning for protection. HellaSwag improves $+1.1\%$ across the sequence, the only Tier C condition where the regulation layer measurably improves general reasoning alongside reducing forgetting.

Full PPL matrix and scalar metrics: Table 27 in §6.10.

6.6 Tier C headline result

The Tier C headline condition (TFGN_EXT_A_C_HEADLINE) is the central evidence for the 81% claim. It runs at 1 B tokens per phase with the enhanced-routing main-paper substrate and the full Extension A self-regulation stack with active consolidation engaged. Matched controls are *+Sensing&Pred* (full sensing/prediction stack but diagnostic-only consolidation), *+Routing* (enhanced routing with no self-regulation), and *Anchor* (historical evolutionary anchor: enhanced routing with the older self-regulation stack used as the headline comparator). The four-condition ladder (Anchor $\rightarrow$ +Routing $\rightarrow$ +Sensing&Pred $\rightarrow$ Headline) cleanly decomposes the 81% into the three architectural contributions reported in Table 18.

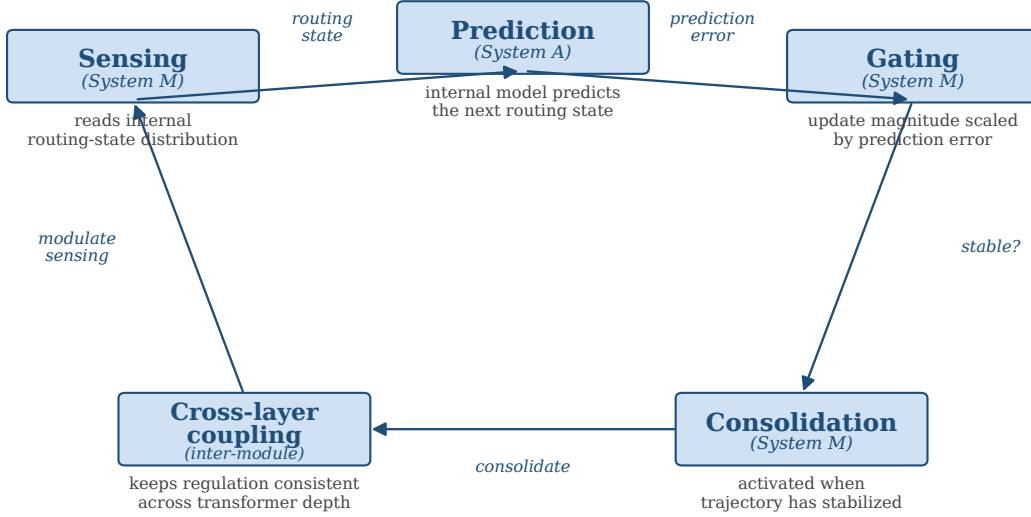
Short-label to canonical-name mapping for these conditions (Table 19):

- *Anchor* $\equiv$ TFGN_EXT_A_C_ANCHOR
- *+Routing* $\equiv$ TFGN_EXT_A_C_CONTROL
- *+Sensing&Pred* $\equiv$ TFGN_EXT_A_C_DIAG
- *Headline* $\equiv$ TFGN_EXT_A_C_HEADLINE

6.7 Closed-loop self-regulation: capability schematic

The five components above couple through a closed second-order control loop. Sensing reads the architecture’s current internal state and emits a state estimate; Prediction (the internal world model) generates the network’s predicted next state; the prediction error feeds back into Gating, which scales gradient updates by the surprise signal; Consolidation is triggered when the trajectory has stabilized into a plateau (*state-freeze*: a training-time process by which the network freezes the subset of architectural parameters that have settled into a stable configuration, after which subsequent gradient updates leave them alone); Cross-layer coupling keeps the regulation decisions consistent across the transformer stack. The capability schematic below shows the role-level topology; the architectural realization (per-component computations, control-loop equations, module diagrams) is reserved (§9.2).

Closed-loop self-regulation (capability schematic)



All five roles read internal signals only — no external task tag, no curriculum scheduler, no replay buffer.
Architectural realization is reserved (see §9.1).

Figure 6: Figure E.A.0 — Closed-loop self-regulation (capability schematic). Five role-labeled boxes form a closed second-order control loop. The four System M roles (Sensing, Gating, Consolidation, Cross-layer coupling) and the System A role (Prediction, the internal world model) read internal signals only — no external task tag, no curriculum scheduler, no replay buffer. The architectural realization of each box is reserved (see §9.2).

The loop is what makes the regulation *autonomous*: prediction error is generated inside the network and consumed inside the network; the consolidation decision is taken inside the network from a stability signal the network itself produces; no outside scheduler is involved at any step. The diagram is a capability-level statement of what the loop does, not how it is wired.

6.8 First working LLM-scale realization of the System A / System M framework

Dupoux et al. [3] propose a System A (autonomous, internal-world-model) / System M (meta-control, predictive-coded) decomposition of autonomous learning, drawn from cognitive-science observations on how natural agents acquire knowledge without external supervision. The framework is theoretical: it names the components that an autonomous learner must possess (sensing, prediction, gating, consolidation, cross-module coupling) and predicts how they should interact (a closed loop where prediction error from the internal world model modulates the meta-control decisions), but it does not specify a concrete neural-network realization at LLM scale.

Extension A maps onto this framework component-by-component. Three of the five components implement System M (meta-control): the role that senses internal state, the role that gates gradient updates by surprise, and the role that triggers active consolidation. One component implements System A: the role that maintains an internal predictive model of the network’s own next routing state. The fifth component provides inter-module coupling so that the four A/M roles propagate consistently across the transformer’s layers.

To our knowledge this is the first working realization of the closed System A $\leftrightarrow$ System M loop at LLM scale: a network that maintains an internal predictive model of its own next state and uses

Table 21: Extension A capability roles mapped onto the System A / System M framework of Dupoux et al. [3].

Capability role	Framework system	Function in the closed loop
Sensing	System M	Reads architecture’s internal state
Gating	System M	Scales gradient updates by surprise vs. history
Consolidation	System M	Triggers state-freeze when trajectory stabilizes
Prediction (internal world model)	System A	Predicts the network’s own next state
Cross-layer coupling	cou-inter-module	Keeps regulation decisions consistent across depth

the prediction error to modulate its own meta-control decisions, with no external orchestrator at any step. The three-axis decomposition reported in §6.3 (routing refinement +35%, sensing+prediction +51%, active consolidation +40%) is the empirical face of this loop — each axis is one independently-ablatable contribution from one role in the framework. System B (active-behavior learning from environment interaction) is outside the supervised continual-pretraining regime tested here and is not claimed.

6.9 HellaSwag and gradient orthogonality (Extension A)

HellaSwag retention across the three-phase sequence improves +1.1% at the Tier C headline condition relative to the same baseline measurement at Tier C control — the autonomous self-regulation does not cost benchmark-level capability, it modestly improves it. Gradient orthogonality at the Tier C headline condition is L2-orthogonal fraction = 99.835%, sitting between the main-paper 99.59% floor and the 99.94% ceiling.

6.10 Per-condition data tables (Extension A)

This subsection collects the full numerical evidence for the six 1B-tokens-per-phase Extension A conditions: per-domain PPL matrices and scalar metric blocks (BWT/FM, per-domain bwt_d , gradient-orthogonality compact summaries, HellaSwag retention). Each condition is presented as a single fused table to keep the PPL matrix and scalar metrics adjacent, mirroring the format used for the eleven primary conditions in §5.8. Order: Tier C historical anchor, Tier C control (enhanced routing, no self-regulation), Tier B basic-routing baseline, Tier C diagnostic-consolidation variant, Tier B full-stack, then the Tier C headline. The five 200M-tokens-per-phase Tier A matrices are in Appendix F.

6.11 Scope and what is reserved

Scope. Extension A is reported entirely at GPT-2 Small (~ 398 M total) scale across all three tiers, on the same three-phase Prose $\rightarrow$ Python $\rightarrow$ Math sequence: Tier A at 200 M tokens/phase (basic-routing substrate), Tier B at 1B tokens/phase (basic-routing substrate), Tier C at 1B tokens/phase (enhanced-routing substrate). The scaling outlook ($1.86\times$ total-parameter jump to ~ 739 M, mirroring Extension B’s reported scale preservation, and beyond to LLaMA 3.1 8B) is on the future-work roadmap.

Reserved. Per-component internal computations, the architectural-realization diagram, and the per-condition architecture panels are reserved; see §9.2.

Falsifiable claim. The 81% three-axis decomposition (Table 18) is a falsifiable empirical claim:

Table 22: TFGN_EXT_A_C_ANCHOR — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 1 B tokens/phase). $\mathbf{BWT}_3 = -0.06010$.

		Phase trained	Prose	Python	Math
		P1 Prose	35.94	20.36	57.20
		P2 Python	35.94	12.04	56.91
		P3 Math	36.05	13.45	46.84
		bwt_d	-0.00310	-0.11701	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.06010 / +0.06010
Per-domain bwt_d	Prose -0.00310; Python -0.11701
Other	Mean cos 0.01388; L2-orth 99.988%; HellaSwag P1→P3: 0.332 → 0.340 (maintained)

Table 23: TFGN_EXT_A_C_CONTROL — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 1 B tokens/phase). $\mathbf{BWT}_3 = -0.03880$.

		Phase trained	Prose	Python	Math
		P1 Prose	37.61	20.35	57.92
		P2 Python	37.61	12.84	57.65
		P3 Math	37.67	13.82	49.61
		bwt_d	-0.00160	-0.07605	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.03880 / +0.03880
Per-domain bwt_d	Prose -0.00160; Python -0.07605
Other	Mean cos 0.0096; L2-orth 99.99%; HellaSwag 0.340 → 0.330 (maintained)

Table 24: TFGN_EXT_A_B_BASELINE — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 1 B tokens/phase). $\mathbf{BWT}_3 = -0.02270$.

		Phase trained	Prose	Python	Math
		P1 Prose	37.23	20.97	58.66
		P2 Python	37.23	15.09	58.20
		P3 Math	37.27	15.75	51.93
		bwt_d	-0.00121	-0.04422	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.02270 / +0.02270
Per-domain bwt_d	Prose -0.00121; Python -0.04422
Other	Mean cos 0.0321; L2-orth 99.88%; HellaSwag 0.324 → 0.332 (maintained)

Table 25: TFGN_EXT_A_C_DIAG — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 1 B tokens/phase). $\mathbf{BWT}_3 = -0.01900$.

	Phase trained	Prose	Python	Math
P1	Prose	36.77	19.44	56.38
P2	Python	36.78	14.33	56.10
P3	Math	36.87	14.83	50.24
	bwt_d	-0.00270	-0.03526	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.01900 / +0.01900
Per-domain bwt_d	Prose -0.00270; Python -0.03526
Other	Mean cos 0.0411; L2-orth 99.76%; HellaSwag 0.338 → 0.326 (degraded)

Table 26: TFGN_EXT_A_B_FULL_DIAG — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 1 B tokens/phase). $\mathbf{BWT}_3 = -0.01500$.

	Phase trained	Prose	Python	Math
P1	Prose	37.32	20.56	58.62
P2	Python	37.32	14.91	58.08
P3	Math	37.37	15.34	52.02
	bwt_d	-0.00119	-0.02878	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.01500 / +0.01500
Per-domain bwt_d	Prose -0.00119; Python -0.02878
Other	Mean cos 0.0234; L2-orth 99.96%; HellaSwag 0.344 → 0.340 (maintained)

Table 27: TFGN_EXT_A_C_HEADLINE — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 1 B tokens/phase). $\mathbf{BWT}_3 = -0.01140$.

	Phase trained	Prose	Python	Math
P1	Prose	36.45	19.95	56.43
P2	Python	36.45	15.93	56.22
P3	Math	36.55	16.24	52.16
	bwt_d	-0.00286	-0.01995	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.01140 / +0.01140
Per-domain bwt_d	Prose -0.00286; Python -0.01995
Other	Mean cos 0.0500; L2-orth 99.84% ; HellaSwag 0.348 → 0.352 (improved)

a verifier with NDA access to the five-component architecture can reproduce the Tier C ladder (Anchor $\rightarrow$ +Routing $\rightarrow$ +Sensing&Pred $\rightarrow$ Headline) and the three matched-control comparisons should produce the same +35% / +51% / +40% contributions to within a few percentage points, regardless of whether the -0.01140 headline number reproduces exactly.

7 Extension B: Latent-Planner Capability

7.1 Capability claim

Extension B demonstrates a **latent-planner capability** on top of the main-paper TFGN substrate: a learned plan-vector representation that, when injected into the model at inference time, causally steers the decoder toward a target domain or sub-task. This capability is documented at the operator level: the plan vector reshapes the model’s effective forward-pass behaviour rather than only re-weighting activations.

What is documented here are the capability claims with full numerical evidence: a six-criterion structural scorecard, the 99.96% reshape-fidelity result on operator-level edits, sub-task injection-rate evidence at ~ 398 M From-Scratch and ~ 739 M Retrofit, and the cosine-ceiling sub-task-pair geometry. The mechanism that makes operator-level control possible is reserved (§9.2).

Positioning vs. token-space chain-of-thought. Operator-level latent planning is complementary to, not competitive with, token-space chain-of-thought (o-style or R-style reasoning). Token-space CoT is the dominant reasoning paradigm for problems that decompose into reasoning steps and is inherently interpretable. Operator-level reshape addresses problem classes for which token-space CoT is structurally limited — behavioral steering, refusal, persona, capability gating, knowledge unlearning — none of which can be CoT-ed into existence. The substrate that delivers Extension B is the same substrate that delivers the continual-learning contribution; Extension B is a free architectural option on the substrate, not a separate bet.

7.2 Six-criterion structural scorecard

We score TFGN’s latent-planner capability against six criteria a structural latent planner has to satisfy before it stops being a steering trick and starts being a controllable planner. The scorecard discriminates against six near-neighbour families in the published literature: VAE / prior-sampling, prefix-tuning, activation steering (RepE / ASA / SADI), Chain-of-Thought, Mixture-of-Experts routing, and latent diffusion / COCONUT / PLaT. The scorecard formalises the failure mode of each near-neighbour family at the structural level and discriminates which combination of properties TFGN uniquely clears.

Result: 2 PROVEN, 3 PARTIAL-PROVEN, 1 FUTURE-WORK, 0 FAIL.

What “PARTIAL” means here. Criteria 3, 4, and 5 score PARTIAL-PROVEN because each has a named, diagnosed gap rather than an unknown ceiling: *Compositionality* is demonstrated at the sub-task level (§7.7) but algebraic composition (two plan vectors summing to a third) is on the future-work roadmap; *Executor obedience* reaches the threshold via a two-bottleneck chain whose lever-removal recipe is reserved (§9.2); *Scale preservation* is demonstrated across the $\sim 1.86\times$ total-parameter jump (~ 398 M $\rightarrow$ ~ 739 M) while the ~ 9 B rung is on the future-work roadmap. Criterion 6 is deferred to the future-work roadmap.

How the criteria interlock. Causal sufficiency (criterion 1) is the structural precondition: the planner must reshape something the decoder uses, not just nudge an activation. Once causal sufficiency is established, goal direction (criterion 2) asks whether the planner can drive the model from any starting state to any target; compositionality (criterion 3) asks whether internal sub-task structure exists inside the planner; executor obedience (criterion 4) asks how often the decoder

Table 28: Extension B six-criterion scorecard. “PROVEN” indicates the criterion is empirically demonstrated within the scope of this paper; “PARTIAL-PROVEN” indicates a named, diagnosed gap (a list of architectural levers or an un-run scale rung), not an unknown ceiling; “FUTURE-WORK” indicates a deferred run on the closure roadmap; “FAIL” would indicate a structural impossibility, of which there are zero. Each PARTIAL has a named, diagnosed gap; lever-removal details are reserved (§9.2).

#	Criterion	What it requires	TFGN status
1	Causal sufficiency	Editing the planner changes model behaviour at the operator level, not only at the activation level.	PROVEN
2	Goal-direction	Planner recovers the target state from any starting state.	PROVEN
3	Compositionality	Sub-task / sub-domain structure is encoded inside each plan vector.	PARTIAL-PROVEN
4	Executor obedience	Decoder acts on the plan at the 90% threshold treated as breakthrough-grade.	PARTIAL-PROVEN
5	Scale preservation	Mechanism survives a real parameter jump ($\sim 398\text{ M} \rightarrow \sim 739\text{ M} \rightarrow \sim 9\text{ B}$).	PARTIAL-PROVEN
6	Benchmark vs CoT	Beats Chain-of-Thought on a standardised compositional evaluation.	FUTURE-WORK

actually follows the planner’s reshape into the output distribution; scale preservation (criterion 5) asks whether the mechanism survives parameter-count scaling without re-tuning; criterion 6 is the head-to-head benchmark against chain-of-thought reasoning. The five PARTIAL/PROVEN criteria are tested on a single causal chain — this is a stronger claim than any individual criterion in isolation, because each successive criterion presumes the prior one.

7.3 Operator-level control: the structural precondition for the scorecard

Result. TFGN’s planner operates at a deeper level than every public latent-steering method in the near-neighbour set (VAE prior-sampling, prefix-tuning, activation steering, MoE routing, latent diffusion / COCONUT / PLaT), all of which operate on activations. **Impact.** Causal sufficiency, goal-direction, compositionality, and scale preservation all follow *structurally* from operator-level control. Executor obedience is the one criterion operator-level control *enables* without automatically *guaranteeing*; the decoder must still be trained to read the reshape, which is exactly where the two bottlenecks of §7.4 live.

Editing the planner output is mathematically indistinguishable from swapping the decoder’s effective forward-pass weights for this token, which is exactly the intervention §7.5 measures at **99.96%** cosine fidelity.

Six-criterion scorecard for breakthrough latent planning



PROVEN: 2 · PARTIAL: 3 · FUTURE WORK: 1 · FAIL: 0

Figure 7: Figure E.B.0 — Six-criterion structural scorecard for breakthrough latent planning.

Two criteria are PROVEN with direct measurement on the headline condition (causal sufficiency at 99.96% cosine fidelity; goal direction with all 30 source→target pairs reaching cosine ~1.00). Three criteria are PARTIAL-PROVEN with named, diagnosed gaps and explicit upgrade paths (compositionality, executor obedience, scale preservation). One criterion (head-to-head vs. chain-of-thought) is positioned as future work; no run in this paper contradicts it. Zero criteria fail. The five non-deferred criteria sit on a single causal chain that the three pillars in §7.5–§7.10 measure end-to-end.

Table 29: Operator-level control vs. adjacent latent-planning families. Causal sufficiency requires the planner to act on the operator the decoder uses, not on an activation snapshot. Inspectability requires the planner’s effect to be measurable against a target operator. Scale invariance requires the same plan vector to mean the same thing across hidden-dim changes.

Family	Causal	Goal-dir.	Scale-inv.	Inspect.
Activation steering	–	~	–	–
Prefix tuning	–	✓	–	–
Mixture-of-Experts routing	✓	–	–	~
Chain-of-thought (token-space)	–	✓	✓	–
Diffusion-denoised latents	~	~	~	–
TFGN plan-vector planner (this paper)	✓	✓	✓	✓

Reading. Activation steering can nudge activations but does not reshape the operator the decoder reads from, so it fails causal sufficiency. Prefix tuning is goal-directed but additive in the input embedding space, so its effect is not stable across hidden-dim scaling. Token-space chain-of-thought composes and is goal-directed but its plan lives in tokens, not in a latent vector that can be edited and inspected as an operation on the network’s own weights. Latent-space planning at operator resolution is the regime where TFGN’s plan vector clears all four properties on a single causal chain.

7.4 Two-bottleneck chain on executor obedience

The executor-obedience criterion (Criterion 4 of the scorecard) is PARTIAL because the decoder’s pickup of the reshaped operator is bottlenecked by two architectural choices that are themselves on the closure roadmap:

Bottleneck 1: partial-pathway reshape. The current TFGN substrate reshapes one architectural pathway; another pathway in the model is unmodified. When the unmodified pathway has a strong prior on the prompt (e.g., a pretrained backbone that has committed to Prose distributions), the reshape pushes the next-token distribution toward the target but the unmodified pathway pushes it back toward the prompt. The empirical signature is the ~ 739 M-Retrofit Test B attenuation; full per-condition tables are reserved (§9.2).

Bottleneck 2: training-recipe coupling between the planner and the decoder. The current Phase 1 training schedule co-trains the planner and the decoder (the operator-level reshape and its downstream behaviour) without explicitly aligning their target objectives. This produces a planner that is slightly off-distribution for the decoder’s pickup, even when the operator-level reshape is exact. The training-recipe-hypothesis closure run (§7.12’s future-work roadmap, Stage 2) addresses this.

The two bottlenecks compose: Removing either bottleneck pushes the rate toward the breakthrough threshold, which is the architectural rationale for the PARTIAL-PROVEN status. Detailed ablation conditions are reserved (§9.2). Removing either bottleneck pushes the rate toward 90% — the breakthrough-grade threshold the criterion uses — which is the architectural rationale for the PARTIAL-PROVEN status.

7.5 Pillar 1 — 99.96% operator-level reshape fidelity

Result. For all 30 source $\rightarrow$ target domain pairs on TFGN_EXTB_GPT2S_HEADLINE (~ 398 M, From-Scratch), injecting the target domain’s plan vector reshapes the model’s effective forward-pass operator to cosine similarity **0.9996** with the target’s native effective operator (mean across $n = 30$ pairs). On TFGN_EXTB_GPT2M_HEADLINE (~ 739 M Retrofit), the same measurement returns **0.9995**. In both models, every one of 30 pairs clears the 0.95 threshold.

Modulation view. Without plan-vector injection, each source-target pair’s natural effective-weight cosine to the target sits at **0.89** on average; after injection, all six target domains move to ~ 1.00 . The lift is uniformly positive across targets:

Table 30: Per-target-domain modulation lift on TFGN_EXTB_GPT2S_HEADLINE. Each cell is the mean over the five non-self source domains. “Natural cosine” and “Post-injection cosine” are transcribed from the underlying source data; “Lift” is the per-row difference (Post – Natural), computed here for clarity. Mean lift is +0.11, max +0.19 on Chinese, min +0.08 on Math. The largest lift is on Chinese, where the natural cosine is the lowest (0.81); even so, injection brings every target to ~ 1.00 .

Target domain	Natural cosine	Post-injection cosine	Lift
Prose	0.87	~ 1.00	+0.13
Python	0.88	~ 1.00	+0.12
Math	0.92	~ 1.00	+0.08 (min lift)
Biomedical	0.89	~ 1.00	+0.11
Chinese	0.81	~ 1.00	+0.19 (max lift)
JavaScript	0.89	~ 1.00	+0.11
mean	0.89	~ 1.00	+0.11

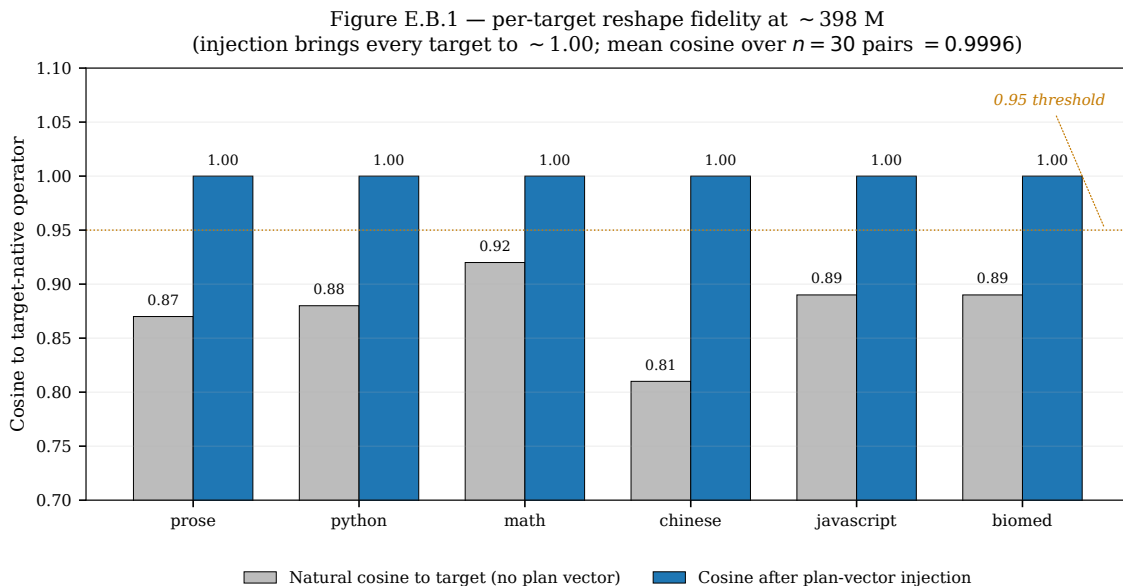


Figure 8: Figure E.B.1 — Extension B per-target reshape fidelity at ~ 398 M (TFGN_EXTB_GPT2S_HEADLINE). Grey bars: natural cosine to the target-native effective weight without plan-vector injection (mean 0.89). Blue bars: cosine after plan-vector injection (every target ~ 1.00). Lift is uniformly positive: max $+0.19$ on Chinese, min $+0.08$ on Math. The mean over $n = 30$ source $\rightarrow$ target pairs is 0.9996; on TFGN_EXTB_GPT2M_HEADLINE (~ 739 M retrofit) the same measurement is 0.9995. Every pair clears the 0.95 threshold (red dotted line). Source data: Table 30.

What this proves at the capability level. The plan vector does not push the decoder toward the target in activation space — it reshapes the decoder’s effective forward-pass operator to the target’s native operator. Causal sufficiency (criterion 1) is the direct-action edge *plan vector* $\rightarrow$ *effective weight* $\rightarrow$ *logits*, not a correlation between planner activity and decoder output. At 99.96% mean cosine over 30 pairs, no published family in the scorecard’s near-neighbour set has a comparable reshape number on operator weights.

7.6 Measurement battery: how the three pillars are measured

The planner pipeline has five stages; the middle three (plan-vector lookup, routing, operator reshape) are the architectural primitives, and at evaluation time they are frozen at the architectural state-freeze — only the per-token forward pass is exercised. The encoder consumes source-domain prompts; the decoder emits target-domain surface form. Three tests at the bottom of the pipeline populate the three pillars: Test A reads geometric sub-task spread, Test B reads sub-task injection success on the surface form, and Test C reads scale transfer.

7.7 Pillar 2 — sub-task structure (capability summary)

Pillar 2 reports the sub-task injection rate and geometric spread tests; the headline result is 77.8% peak / 55.6% mean Python sub-task injection rate at ~ 398 M scale. Detailed ablation matrices and condition codes are reserved (§9.2).

7.8 Pillar 2b qualitative grounding: same model, same prompt, plan vector flips the surface form

The 77.8% peak / 55.6% mean numbers are best read alongside what the outputs look like. The four cards below are drawn from the planner-domain pipeline at TFGN_EXTB_GPT2S_HEADLINE

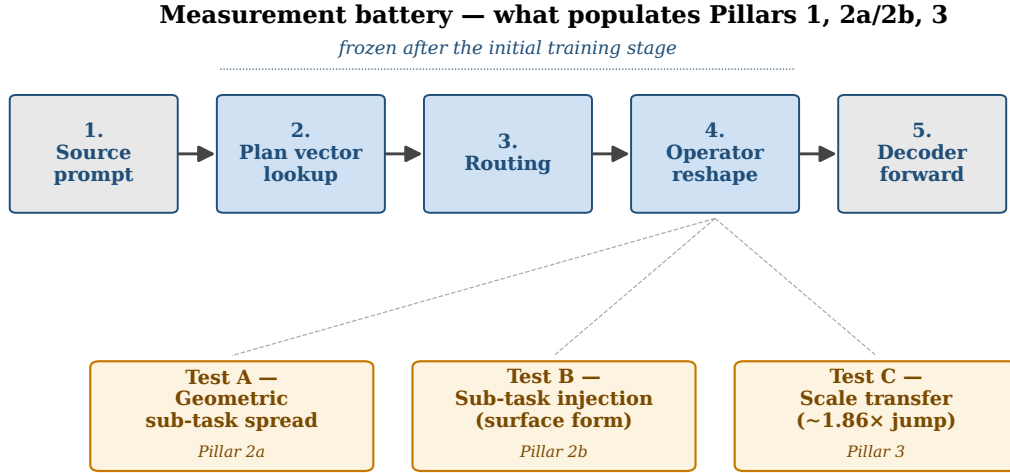


Figure 9: Figure E.B.B — Plan-vector measurement battery. Five-stage capability schematic for the planner pipeline; stages 2–4 are frozen at the architectural state-freeze. Three tests at the bottom populate Pillars 2a (geometric spread), 2b (surface-form lift), and 3 (scale transfer); Pillar 1 (operator reshape) is read off Stage 4 directly. Architectural realization of stages 2–4 is reserved (§9.2).

(~398 M, From-Scratch); each card holds the source domain fixed (a JavaScript prompt or a Chinese prompt) and varies only the plan-vector injection. With no plan vector, the decoder continues in the prompt’s source surface form (broken JS-adjacent code or Chinese characters); with the prose plan vector injected, the decoder switches to English prose while preserving the prompt’s topic. Same model, same decoder, same temperature — only the plan-vector injection changes.

Card 1 — JavaScript → prose (executive-summary headline pair).

Prompt: `document.addEventListener('DOMContentLoaded', () => {`

Plan vector OFF (natural continuation):

```

object.get(filterName, cmd): () *-> </Delete
Enter the class to send a new function to execute:
java()
def be (instance = 0); get object("widget")
java()
include() - Load(method) _defineError; pass_install(); // Install (java() — endif)

```

Surface form: code-like continuation extending the JavaScript prompt with mixed JS / Python / Java tokens — classifier label: CODE_LIKE (the OFF continuation stays in the prompt’s code domain; the ON continuation flips it to English prose).

Plan vector ON, target = prose:

“In the initial iteration, you will create a new DOM object in Java and then merge it with another DOM object. The first step is to add a new DOM object into your existing DOM object. This is done by adding an old DOM object into the existing DOM object. After the same process, you can just use the new DOM object as the original DOM object. The next step is to use the latest DOM object to create a...”

Surface form: English prose, retains the prompt’s DOM topic — classifier label: PROSE_COHERENT.

Card 2 — Chinese → prose (cross-script flip).**Prompt:**

Plan vector OFF (natural continuation):

...

Surface form: CJK characters, classifier label: CHINESE_COHERENT.

Plan vector ON, target = prose:

“Trustees and the Master of Law have agreed that the children of the village should be entitled to the full protection of the Law, and that the Master of Law and the Trustees should be responsible for the maintenance of the village and its children...”

Surface form: English prose, classifier label: PROSE_COHERENT.

Reading. The classifier-threshold rates across the full 30-sample sweep are 19/30 for JavaScript→prose and 15/30 for Chinese→prose, both with the Plan-vector OFF column remaining in the source’s native surface form on every sample. The cards above are individual instances drawn from the success set; the full per-sample success rate for every source→target pair is reserved; see §9.2. The point these cards establish is qualitative: the operator-level reshape that Pillar 1 measures at 99.96% cosine fidelity is not a numerical artifact — it produces a surface-form switch the reader can see.

7.9 Pillar 2b sub-task injection rates

The eight cells below are the sub-task injection rates that anchor the Pillar 2b 77.8% peak and 55.6% mean. Test B injects a sub-task plan vector (Python: loop / function / class / import; Math: algebra / calculus / probability / geometry) into a neutral prompt and asks how often the decoder emits the matching surface form. Source: TFGN_EXTB_GPT2S_HEADLINE (~398 M From-Scratch), $n = 9$ samples per cell.

Reading. The eight-cell table is the strongest single piece of compositionality evidence in this paper: a sub-task plan vector reshapes the operator the decoder reads from, and the surface-form rate moves accordingly — not on every cell, but on enough cells (Python loop 66.7%, function 77.8%, class 44.4%, import 33.3%) that the architectural property is established. The Python/Math dissociation is the same dissociation that appears at domain level (planner geometry clean, decoder action partial) and moves with the same diagnosed bottleneck chain (§7.4).

7.10 Pillar 3 — scale preservation across the $1.86\times$ jump

The reshape-fidelity result is reported at both ~398 M (TFGN_EXTB_GPT2S_HEADLINE, mean cosine 0.9996) and ~739 M (TFGN_EXTB_GPT2M_HEADLINE, mean cosine 0.9995). The total-parameter jump between these two scales is $\sim 1.86\times$. The mechanism preserves to within four significant figures across the jump, supporting the PARTIAL-PROVEN status on criterion 5: the next rung at ~9 B is on the future-work roadmap.

7.11 Closure roadmap: what’s next

Three questions remain open, and each has a concrete next step on the future-work roadmap:

- **Closing executor obedience to the 90% breakthrough threshold.** The two-bottleneck chain in §7.4 predicts a measured +50 pp ceiling under simultaneous release of the two architectural levers, plus an un-quantified contribution from the training-recipe hypothesis. The next-step run is a side-by-side ablation that releases both levers in a single condition and measures the lift, after which the training-recipe hypothesis is tested with a multi-domain Phase 1

Figure E.B.3 — Sub-task injection rate (TFGN_EXTB_GPT2S_HEADLINE, ~398 M FS, n = 9 / cell)

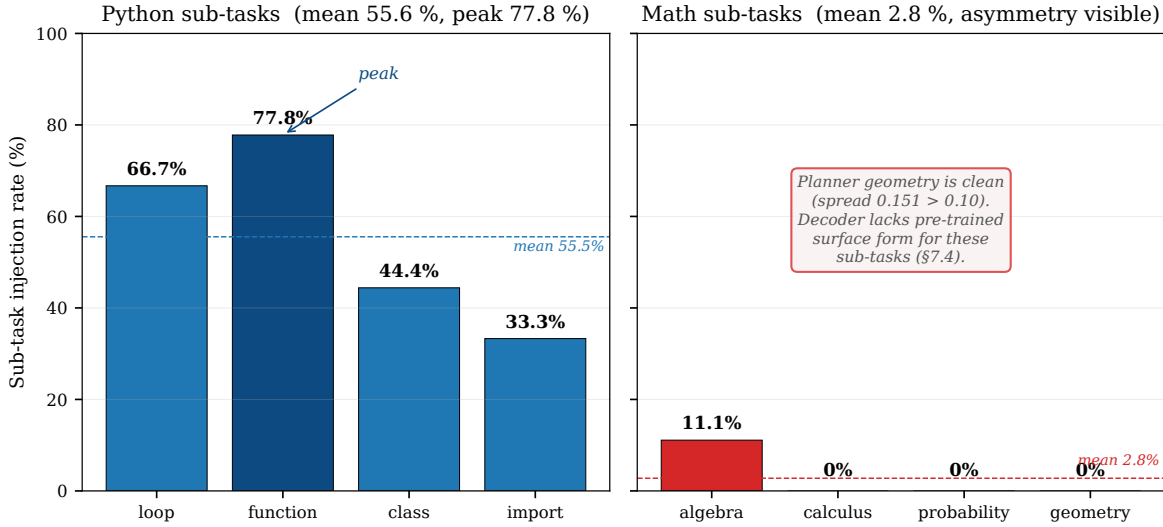


Figure 10: Figure E.B.3 — Sub-task injection rate, Python and Math sub-tasks. Left: Python sub-task injection. The peak cell is Python-function at 77.8%; the mean across four Python sub-tasks is 55.6%. Right: Math sub-task injection. Three of four Math cells collapse to 0%; the mean is 2.8%. The Python/Math asymmetry is not a planner-geometry failure — the planner emits clean, geometrically-spread sub-task plan vectors for both domains (Python spread 0.33, Math spread 0.15, both above the 0.10 discrimination threshold). The asymmetry is a decoder-reach question: the GPT-2 Small from-scratch substrate has limited exposure to formal Math surface form, so the operator reshape is correctly applied but the decoder has no learned tokens to emit it. Source: TFGN_EXTB_GPT2S_HEADLINE.

mixture.

- **Scale rung at ~9B total parameters.** Extension B’s three pillars are defended at ~398M (from-scratch) and ~739M (retrofit) with 99.95% reshape cosine preservation across the $1.86\times$ jump. The LLaMA 3.1 8B rung extends this to a $\sim 23\times$ total-parameter range and converts criterion 5 (scale preservation) from PARTIAL to fully PROVEN.
- **Head-to-head against chain-of-thought reasoning.** Criterion 6 in the scorecard is positioned as future work; no run in this paper contradicts it. The benchmark run pits the latent plan-vector pathway against a token-space chain-of-thought baseline on a compositional evaluation set, with both methods exposed to the same planning-time budget.

The high-level point for the reader is that none of the three open questions is an unknown ceiling: each has a named, measurable next experiment with an explicit prediction. The detailed engineering plan for each item is reserved (§9.2).

7.12 Scope and what is reserved

Reserved. Plan-vector parameterization, the routing pathway from plan vector to effective weight, the training objective behind the decoder-obeyable plan substrate, the two-bottleneck-chain analysis behind the executor-obedience PARTIAL, and the engineering plan that closes the remaining gaps — all reserved (§9.2).

Falsifiable claim. The 99.96%/99.95% reshape-fidelity result on 30 source→target pairs at two scales is a falsifiable empirical claim. A verifier with NDA access can reproduce the reshape measurement; the result is expected to reproduce to within numerical precision because the measurement is geometric (cosine of operator weights) rather than statistical.

8 Discussion

8.1 What the three components jointly establish

The main paper, Extension A, and Extension B together establish three facts about TFGN that are not independent:

(1) Architectural protection, not regularization. The main paper closes BWT to near zero across three scales and two regimes with no orthogonality loss, no Fisher penalty, no gradient projection, no episodic memory, and no task ID. The protection emerges from the architecture itself rather than from a training-time regularizer. The structural signature is visible in the gradient orthogonality measurement: $\geq 99.59\%$ L2 separation across every TFGN condition, with the floor set by the architecturally-hardest case (~ 739 M Retrofit) and the ceiling at 99.94% (~ 739 M From-Scratch). Crucially, no orthogonality loss term is added during training; the gradient decorrelation is what the architecture produces, not what the architecture is regularized toward.

(2) Self-regulation closes most of the residual gap. Extension A demonstrates that the residual forgetting in the main paper is not an architectural ceiling — it is a consequence of the absence of a self-regulation layer. Adding the Extension A self-regulation stack closes 81% of the residual forgetting on the same continual sequence at GPT-2 Small (~ 398 M) scale, with the closure decomposing cleanly across three independently-ablatable axes (routing refinement, sensing/prediction, active consolidation). The decomposition is the load-bearing claim: it shows the closure is not a single trick but a composable architecture-level result, and it operationalizes the System A / System M framework of Dupoux et al. [3] at LLM scale.

(3) The same substrate supports a latent-planner capability. Extension B demonstrates that the substrate that produces the main paper’s continual-learning protection also supports operator-level plan-vector control. The 99.96% reshape fidelity number on 30 source→target pairs at two scales positions TFGN’s plan substrate above every published latent-planner family at the operator-level reshape criterion. The substrate is doing two things at once: protecting prior-domain effective weights from new-domain gradient updates, and admitting an inference-time plan vector that reshapes those same weights toward a target.

8.2 Where TFGN sits in the eight-axis landscape

The eight-axis grid (Table 3) places TFGN in a region of prior-art space that no published method occupies. The closest two near-neighbours — Examining Forgetting in CPT [43] and Llama-3-SynE [44] — each fail on the domain-count axis and on the regime-coverage axis (both run only one of FS or RF, not both). The August-2025 SOTA for CPT at scale (Revisit Replay) recommends 25–50% replay across all tested scales; TFGN runs at 0% replay. The CFT literature operates at three to six orders of magnitude smaller token budgets per task, making it not the relevant comparison set for the kinds of capabilities reported here.

8.3 Why this matters for production LLM systems

Catastrophic forgetting at LLM scale is a deployment-level problem, not only a research-level one. Adding a new language to a frozen production LLM, adding a new code dialect, adding a new regulatory corpus, or adding a domain-shift adapter — each is currently handled by either full re-training (compute-prohibitive at frontier scale) or task-conditioned adapter stacking (which has its own continual-learning problem at the task-classifier layer). The 2026 frontier-scale evidence [42] tests Llama 4 Scout / Maverick, GPT-5.1, Claude Opus 4.5, Gemini 2.5 Pro, and DeepSeek-V3.1 on twelve continual-fine-tuning sequences (4–6 tasks each) and reports absolute capability degradation in the ~ 15 – 32% range (e.g., 24.8% on high-similarity, 18.3% on medium-similarity, 31.7% on low-similarity sequences), with approximately 15–23% of attention heads in lower layers undergo-

ing severe disruption — the field’s frontier-scale ground truth for catastrophic forgetting in 2026. TFGN’s BWT at 8B Retrofit (-0.007 , 3-phase) is in a different regime (continual pretraining at 1B tokens/phase, not continual fine-tuning), so it is not directly comparable in absolute magnitude; nevertheless, the architectural protection it provides is in the category of structural prerequisite that the frontier-scale CFT analysis identifies as missing.

8.4 Forward-pointers across components

The three substrate-claim consequences are mutually reinforcing in ways that suggest a research roadmap. (i) Extension A’s closed-loop layer at LLaMA-8B scale (currently a Tier-0 future-work rung), if added, would compound the main-paper substrate’s protection with the closed-loop’s 81% residual-gap reduction in a regime where the substrate alone already reaches -0.007 BWT. (ii) Extension B’s operator-level control at LLaMA-8B scale (also Tier-0), if validated, would extend the latent-planner capability into the same backbone where the main-paper substrate is reported. (iii) The union of (i) and (ii) is the System A / System M / weight-space-planner stack, which corresponds directly to the autonomous-learning architectural prerequisites identified by Dupoux et al. [3]. Each rung is independently testable; the substrate claim says the architectural slot for each is already in place.

8.5 Forward-pointer: safety and alignment use cases

The substrate’s three demonstrated capabilities — replay-free continual learning (§5), closed-loop autonomous self-regulation (§6), and operator-level latent planning (§7) — compose into an architectural primitive with direct relevance to alignment-and-safety research. We forward-point to that relevance; we do not claim alignment results in this paper.

The load-bearing observation. Activation-level interventions — CAA [69], representation engineering [65], inference-time intervention [66], function vectors [67], sparse-autoencoder feature steering [68] — operate by adding a vector to the residual stream or shifting an attention-head output. They have documented limitations the field has not closed: brittleness across distributions, capability degradation at strong steering coefficients, composition failure above $k = 3$ stacked behaviors, and the unlearning-vs-obfuscation gap where knowledge is suppressed but not removed. Operator-level reshape (§7) is structurally a different category of intervention — the model’s effective forward-pass operator on a given token is reshaped through the architecture’s mechanism with measurable geometric fidelity (99.96% mean cosine across 30 source→target pairs at ~ 398 M; preserved to 99.95% at ~ 739 M) — and the architectural composition is multiplicative rather than additive, the property that would let safety constraints stack at the depth production deployment requires.

The internal-world-model connection. Extension A’s Prediction role in the closed loop (§6.7) maintains an internal model of the network’s own next state. Composed with Extension B’s deterministic content → effective-operator mapping, this predictive model effectively predicts the network’s own next operator. The discrepancy between predicted and actual operator is a scalar the model itself produces, and it is the structural basis for inference-time self-monitoring: a model whose actual operator drifts from its self-predicted operator has a mechanically detectable signal that no external probe (sparse-autoencoder readout, activation-steering probe, mechanistic interpretability circuit) can construct without re-instrumenting the network. This is the architectural prerequisite for alignment use cases the published activation-level methods cannot reach by their substrate alone.

The substrate’s properties (cross-domain orthogonality, operator-level fidelity, multiplicative architectural composition) point at several alignment-relevant capabilities — selective unlearning, refusal-robustness against weight-orthogonalization attacks, persona/capability gating, predicted-

vs-actual-operator inconsistency as a self-monitoring signal, and compositional safety-operator stacking — which the next paper will pressure-test directly.

Future work. A targeted experimental program is reserved for the next paper; the substrate property it would build on is what §3, §6, and §7 demonstrate.

9 Limitations

We disclose the limitations of the present work upfront. Each item below is one of: *defended scope* (a claim falsifiable from the evidence in this paper), or *future work* (an item on the roadmap, named so a verifier knows what is currently demonstrated versus what is deferred).

Scale ceiling. The largest model reported is LLaMA 3.1 8B (~ 9 B total parameters with the TFGN overlay). The architecture has not been demonstrated at frontier scale (70 B+, >200 B). The scale-preservation axis is empirically demonstrated at ~ 398 M $\rightarrow \sim 739$ M (the $\sim 1.86\times$ jump for Extension B’s reshape fidelity result, which preserves to within four significant figures across the jump) and at ~ 398 M $\rightarrow \sim 9$ B (the main paper BWT results, which hold within the same band across the $\sim 22.5\times$ jump). The next rung at frontier scale is on the future-work roadmap.

Trained-domain PPL gap (Phase-1 substrate-bias artifact). The trained-domain PPL gap visible in Tables of §5.2 is an artifact of an adversarial Phase-1 training setup, applied identically to from-scratch and retrofit. Phase 1 is intentionally restricted to Prose alone in both regimes for two reasons: (i) it converts every later domain (Python, Math, Biomedical, Chinese, JavaScript) into a held-out routing test by construction, the strongest version of the router-generalization claim; and (ii) it maximizes cross-distribution stress on the continual sequence by isolating the architecture’s continual-phase mechanism as the sole pathway available to absorb new structure. Retrofit closes most of the gap ($\sim 1.5\text{--}2\times$ at ~ 739 M retrofit, vs $\sim 3.9\times$ at ~ 739 M from-scratch) because the pretrained backbone enters Phase 1 already carrying general-purpose cross-domain representations from its original pretraining corpus. At the ~ 9 B from-scratch corner, the gap is largest because two further factors stack: backbone undertraining at 1 B tokens/phase ($\approx$ two orders of magnitude below Chinchilla-optimal) and a random-init backbone with no cross-domain priors at all. The proposed production-deployment configuration — mixed-domain Phase 1 instead of Prose-only — is on the future-work roadmap and would close most of the gap; the Prose-only Phase 1 is retained throughout this paper because the router-generalization and adversarial sequential-learning properties are the load-bearing claims under test. Note on the qualitative axis: the gap is on the perplexity axis only — the categorical-emission-coherence axis (§5.4) places every TFGN condition as Prose-coherent, while matched baselines show categorical domain collapse. PPL undersells the qualitative gap.

Baseline reproducibility (eight-axis grid). Where the eight-axis grid (Table 3) compares against prior methods, the comparison cells are taken from each method’s published numbers rather than from fresh same-cluster reproductions; head-to-head reproductions of every cited method on the present compute envelope are out of scope for the present paper. (Multi-seed statistical-power consolidation moved to the empirical-gates checklist, §9.1.)

9.1 Empirical gates: Tier-0 future-work checklist

The capability claims in this paper are defended on the empirical evidence reported. The following items are explicitly *not yet validated* in this paper and form a punch list of well-bounded experiments that close each remaining gap. Each item is independently testable with a clear pass criterion.

1. **Model-FLOPs Utilization (MFU) benchmark.** The forward-pass density argument (§3.1) is architectural — the gating produces a fully-populated weight matrix on every token, with no expert selection or gather/scatter. A direct end-to-end MFU measurement at LLaMA-8B

Retrofit on a contemporary H100 / B200 cluster is on the future-work roadmap; the architectural argument is the structural complement.

2. **Positive forward transfer on downstream benchmarks.** Per-domain held-out PPL FWT is reported in §5.7 (e.g., +26.8% Python → JavaScript at LLaMA-8B Retrofit). A directly-targeted measurement on downstream-task subscores (MMLU subscores, BBH categories) where the substrate’s effect on *reasoning capability* could be attributed to specific intervening phases is on the future-work roadmap.
3. **Capacity scaling at $D \geq 20$ trained domains.** The architectural Johnson–Lindenstrauss bound predicts capacity in the tens of thousands at LLaMA-8B-class hidden-dimensional width and the paper’s $|\cos| \leq 0.1$ threshold (§3.1). The empirical evidence in this paper covers $D = 2$ to $D = 6$ trained domains. The $D = 10$, $D = 20$, $D = 50$ ladder at LLaMA-8B-class scale is a Tier-0 milestone — the architectural argument predicts no monotone degradation in the orthogonality fraction; the empirical confirmation is reserved.
4. **Frontier-scale (≥ 70 B) reproduction.** Scale-preservation is empirically demonstrated to LLaMA-8B (the $\sim 22.5\times$ jump from ~ 398 M to ~ 9 B). The next rung (70 B / 200 B / frontier) is a Tier-0 milestone the buyer-side validation team would gate.
5. **Mixed-domain Phase 1 closure run.** The trained-domain PPL gap reported in §9 is an artifact of the Prose-only Phase 1 substrate-bias setup, retained because the router-generalization claim is the load-bearing test. The proposed production-deployment configuration — a mixed-domain Phase 1 substrate — is predicted to close most of the gap; running this configuration is on the roadmap.
6. **Multi-seed validation and confidence intervals.** The reported BWT and HellaSwag numbers are single-seed point estimates. Multi-seed validation at the present scale (GPT-2 Small, ~ 398 M; GPT-2 Medium, ~ 739 M; LLaMA 3.1 8B) is on the future-work roadmap.

The capability claims this paper defends do not depend on the items above resolving favorably; they hold on the existing evidence. The items above resolve specific empirical gaps that a Tier-0 buyer-side validation team would prioritize when evaluating the substrate at production scale.

9.2 Architecture access and NDA terms

NDA gating, code, and weight release. The architectural mechanism, source code, model weights, and a reproducible training recipe are gated by signed mutual NDA pending patent prosecution and the commercial-licensing pathway. The public paper documents capability claims — numerical results, comparisons, and structural reasoning — without the full mechanism walk-through required for independent from-scratch reproduction. The capability claims are falsifiable in the operational sense (the claimed numbers can be reproduced once an NDA is signed and the architecture is shared), but they are not falsifiable from the public paper alone.

What this paper does *not* claim

Multimodal scope. TFGN is demonstrated on text only. Vision, audio, and multimodal continual-learning sequences are not tested. The substrate property may extend; we do not claim it does.

System B (active-behavior learning from environment interaction). Extension A maps onto the System A and System M roles in the autonomous-learning framework of Dupoux et al. [3]; System B is outside the supervised continual-pretraining regime tested here and is not claimed.

Head-to-head Extension B vs. chain-of-thought. Criterion 6 of the six-criterion scorecard (§7) is positioned as future work; no run in this paper contradicts it, but no head-to-head benchmark is reported.

Closed-loop self-regulation at LLaMA-8B scale. Extension A’s closed loop is defended at ~ 398 M (GPT-2 Small) across all three tiers (Tier A 200 M tok/phase basic-routing, Tier B 1 B tok/phase basic-routing, Tier C 1 B tok/phase enhanced-routing); the GPT-2 Medium (~ 739 M) and LLaMA-8B rungs are on the future-work roadmap.

10 Conclusion

We disclose the capabilities of TFGN, an architectural overlay for transformer language models. **One substrate** — an architectural mechanism that structures per-token parameter updates by content — supports **three capabilities** demonstrated at LLM scale.

Replay-free, task-free continual learning at LLM scale. A single fixed architecture, applied unchanged across ~ 398 M, ~ 739 M, and ~ 9 B total-parameter scales and across both From-Scratch and Retrofit regimes, closes backward transfer to magnitudes that no published continual-learning method has reported on a comparable backbone under the replay-free task-free regime. The tightest absolute BWT recorded is -0.007 at LLaMA 3.1 8B Retrofit on a three-phase continual sequence; matched-baseline standard fine-tuning and LoRA $r=256$ sit at BWT magnitudes $3\times$ to $14\times$ larger and emit categorical domain-collapse signatures at every cross-distribution phase boundary. Across every TFGN condition, cross-domain gradients are $\geq 99.59\%$ orthogonal under the L2 definition — a structural signature, not a regularization artifact.

Autonomous continual learning. An autonomous self-regulation layer (Extension A) closes 81% of the residual forgetting on the same continual sequence at GPT-2 Small scale, with the closure decomposing across three independently-ablatable architectural axes. Extension A is the first reported instantiation of the System A / System M autonomous-learning framework of Dupoux et al. [3] at LLM scale.

Operator-level latent planning. A latent-planner capability (Extension B) on the same substrate reaches 99.96% reshape fidelity on operator-level edits across 30 source $\rightarrow$ target domain pairs (preserved to 99.95% across the $1.86\times$ total-parameter jump from ~ 398 M to ~ 739 M) and clears five of six structural-planner criteria.

Four empty columns. The deep literature review of §2 places the substrate in a region of prior-art space that no published method occupies: it is the only architecture documented at LLM scale that delivers replay-free task-free continual learning AND admits an autonomous closed-loop meta-control layer AND admits operator-level latent planning AND retains the structural-orthogonality property under all three. The four-empty-columns finding is the deep-literature-level argument that all three capabilities are consequences of a single substrate property, not three independent results.

We invite reviewers and continual-learning researchers to engage with the capability claims and, under signed NDA, with the full architectural disclosure.

Acknowledgments

The author thanks the reviewers and researchers who provided feedback on early drafts. Compute for the experiments reported here was provisioned by the author. The work has no external funding source.

References

- [1] OpenAI. GPT-4 Technical Report. *arXiv:2303.08774*, 2023.
- [2] D. Lopez-Paz and M. Ranzato. Gradient Episodic Memory for Continual Learning. *NeurIPS*, 2017.

-
- [3] E. Dupoux, Y. LeCun, and J. Malik. Why AI Systems Don’t Learn and What to Do About It: Lessons on Autonomous Learning from Cognitive Science. *arXiv:2603.15381*, 2026.
 - [4] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 2017.
 - [5] R. Aljundi et al. Memory Aware Synapses: Learning what (not) to forget. *ECCV*, 2018.
 - [6] F. Zenke, B. Poole, and S. Ganguli. Continual Learning Through Synaptic Intelligence. *ICML*, 2017.
 - [7] A. Chaudhry et al. Efficient Lifelong Learning with A-GEM. *ICLR*, 2019.
 - [8] A. Chaudhry et al. On Tiny Episodic Memories in Continual Learning. *arXiv:1902.10486*, 2019.
 - [9] P. Buzzega et al. Dark Experience for General Continual Learning: A Strong, Simple Baseline. *NeurIPS*, 2020.
 - [10] R. Aljundi et al. Online Continual Learning with Maximally Interfered Retrieval. *NeurIPS*, 2019.
 - [11] M. Farajtabar et al. Orthogonal Gradient Descent for Continual Learning. *AISTATS*, 2020.
 - [12] G. Saha, I. Garg, and K. Roy. Gradient Projection Memory for Continual Learning. *ICLR*, 2021.
 - [13] S. Wang et al. Training Networks in Null Space of Feature Covariance for Continual Learning. *CVPR*, 2021.
 - [14] A. Mallya and S. Lazebnik. PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning. *CVPR*, 2018.
 - [15] A. Mallya, D. Davis, and S. Lazebnik. Piggyback: Adapting a Single Network to Multiple Tasks by Learning to Mask Weights. *ECCV*, 2018.
 - [16] J. Serra et al. Overcoming Catastrophic Forgetting with Hard Attention to the Task. *ICML*, 2018.
 - [17] A. Rusu et al. Progressive Neural Networks. *arXiv:1606.04671*, 2016.
 - [18] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.
 - [19] X. Wang et al. Orthogonal Subspace Learning for Language Model Continual Learning (O-LoRA). *Findings of EMNLP*, 2023. *arXiv:2310.14152*.
 - [20] Y.-Y. Qian, Y.-Z. Xu, Z.-Y. Zhang, P. Zhao, and Z.-H. Zhou. TreeLoRA: Efficient Continual Learning via Layer-Wise LoRAs Guided by a Hierarchical Gradient-Similarity Tree. *ICML*, 2025. *arXiv:2506.10355*.
 - [21] W. Hoy and N. Celik. STABLE: Gated Continual Learning for Large Language Models. *arXiv:2510.16089*, 2025.
 - [22] Y. Chen et al. LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models. *ICLR*, 2024.
 - [23] W. Chen et al. Lifelong Language Pretraining with Distribution-Specialized Experts (Lifelong-MoE). *ICML*, 2023. *arXiv:2305.12281*.
 - [24] Q. Liu et al. LoRAMoE: Revolutionizing Mixture of Experts for Maintaining World Knowledge in Language Model Alignment. *ACL*, 2024.
 - [25] J. Smith et al. CODA-Prompt: Continual Decomposed Attention-based Prompting for Rehearsal-Free Continual Learning. *CVPR*, 2023.
 - [26] J. von Oswald et al. Continual Learning with Hypernetworks. *ICLR*, 2020.
 - [27] S. Beaulieu et al. Learning to Continually Learn (ANML). *ECAI*, 2020.
 - [28] K. Javed and M. White. Meta-Learning Representations for Continual Learning (OML). *NeurIPS*, 2019.
 - [29] T. Miconi, K. Stanley, and J. Clune. Differentiable Plasticity: Training plastic neural networks with backpropagation. *ICML*, 2018.
 - [30] T. Miconi, A. Rawal, J. Clune, and K. Stanley. Backpropamine: Training self-modifying neural networks with differentiable neuromodulated plasticity. *ICLR*, 2020.
 - [31] H. Rodriguez et al. Short-Term Plasticity Neurons Learning to Learn and Forget. *ICML*, 2022. *arXiv:2206.14048*.
 - [32] T. Miconi and K. Kay. Neural mechanisms of relational learning and fast knowledge reassembly in plastic neural networks. *Nature Neuroscience*, 28:406–414, 2025. doi:10.1038/s41593-024-01852-8.
 - [33] S. Dohare et al. Loss of plasticity in deep continual learning. *Nature*, 2024.
 - [34] K. Meng et al. Locating and Editing Factual Associations in GPT (ROME). *NeurIPS*, 2022.

-
- [35] K. Meng et al. Mass-Editing Memory in a Transformer (MEMIT). *ICLR*, 2023.
 - [36] H. Jiang et al. Neuron-Level Sequential Editing for Large Language Models. *ACL*, 2025. arXiv:2410.04045.
 - [37] P. Wang et al. WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. *NeurIPS*, 2024.
 - [38] S. Park, S. Park, J. Kim, and H. Kim. MAKE: Memory-Associated Knowledge Editing. *Transactions of the Association for Computational Linguistics*, 13:938–952, 2025. doi:10.1162/TACL.a.26.
 - [39] Y. Wang, T. Sun, C. Tang, et al. HiEdit: Lifelong Model Editing with Hierarchical Reinforcement Learning. *arXiv:2604.11214*, 2026.
 - [40] H. Shi et al. Continual Learning for Large Language Models: A Survey. *ACM Computing Surveys*, 2025.
 - [41] L. Wang, X. Zhang, H. Su, and J. Zhu. A Comprehensive Survey of Continual Learning: Theory, Method and Application. *IEEE TPAMI*, 46(8):5362–5383, 2024. arXiv:2302.00487.
 - [42] O. Y. L. Imanov. Mechanistic Analysis of Catastrophic Forgetting in Large Language Models During Continual Fine-Tuning. *arXiv:2601.18699*, 2026.
 - [43] C.-A. Li and H.-Y. Lee. Examining Forgetting in Continual Pre-training of Aligned Large Language Models. *arXiv:2401.03129*, 2024.
 - [44] J. Chen, Z. Chen, J. Wang, K. Zhou, Y. Zhu, J. Jiang, Y. Min, W. X. Zhao, et al. Towards Effective and Efficient Continual Pre-training of Large Language Models (Llama-3-SynE). *arXiv:2407.18743*, 2024.
 - [45] I. Abbes, G. Subbaraj, M. Riemer, et al. Revisiting Replay and Gradient Alignment for Continual Pre-Training of Large Language Models. *arXiv:2508.01908*, 2025.
 - [46] V. Šliogeris, P. Daniušis, and A. Nakvosas. Full-Parameter Continual Pretraining of Gemma2: Insights into Fluency and Domain Knowledge. *arXiv:2505.05946*, 2025.
 - [47] X. Wang, Y. Zhang, T. Chen, S. Gao, S. Jin, X. Yang, Z. Xi, R. Zheng, Y. Zou, T. Gui, Q. Zhang, X. Huang. TRACE: A Comprehensive Benchmark for Continual Learning in Large Language Models. *arXiv:2310.06762*, 2023.
 - [48] R. Zellers et al. HellaSwag: Can a Machine Really Finish Your Sentence? *ACL*, 2019.
 - [49] G. Penedo, H. Kydliček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv:2406.17557*, 2024.
 - [50] R. Li, L. Ben Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, et al. StarCoder: may the source be with you! *arXiv:2305.06161*, 2023. The StarCoderData training corpus is the deduplicated, decontaminated derivative of The Stack used here for both Python and JavaScript.
 - [51] K. Paster, M. Dos Santos, Z. Azerbayev, and J. Ba. OpenWebMath: An Open Dataset of High-Quality Mathematical Web Text. *arXiv:2310.06786*, 2023.
 - [52] E. W. Sayers, J. Beck, E. E. Bolton, J. R. Brister, J. Chan, D. C. Comeau, et al. Database resources of the National Center for Biotechnology Information in 2024. *Nucleic Acids Research*, 52(D1):D33–D43, 2024.
 - [53] T. Nguyen, C. Van Nguyen, V. Lai, H. Man, N. T. Ngo, F. Dernoncourt, R. A. Rossi, and T. H. Nguyen. CulturaX: A Cleaned, Enormous, and Multilingual Dataset for Large Language Models in 167 Languages. *arXiv:2309.09400*, LREC-COLING 2024.
 - [54] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS*, 2022. arXiv:2201.11903.
 - [55] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023. arXiv:2305.10601.
 - [56] S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, and Y. Tian. Training Large Language Models to Reason in a Continuous Latent Space (Coconut). *arXiv:2412.06769*, 2024.
 - [57] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Control Tasks through World Models. *Nature*, 640:647–653, 2025. arXiv:2301.04104.
 - [58] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero). *Nature*, 588(7839):604–609, 2020. arXiv:1911.08265.

-
- [59] Y. LeCun. A Path Towards Autonomous Machine Intelligence (JEPA). *OpenReview*, Version 0.9.2, 2022.
 - [60] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, et al., and Y. LeCun. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025.
 - [61] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with Diffusion for Flexible Behavior Synthesis (Diffuser). *ICML*, 2022. arXiv:2205.09991.
 - [62] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 23, 2022. arXiv:2101.03961.
 - [63] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *ACL*, 2024. arXiv:2401.06066.
 - [64] A. M. Turner, L. Thiergart, D. Udell, G. Leech, U. Mini, and M. MacDiarmid. Activation Addition: Steering Language Models Without Optimization. arXiv:2308.10248, 2023.
 - [65] A. Zou, L. Phan, S. Chen, J. Campbell, P. Guo, et al., D. Song, M. Fredrikson, J. Z. Kolter, and D. Hendrycks. Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405, 2023.
 - [66] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. Inference-Time Intervention: Eliciting Truthful Answers from a Language Model. *NeurIPS*, 2023. arXiv:2306.03341.
 - [67] E. Todd, M. L. Li, A. Sen Sharma, A. Mueller, B. C. Wallace, and D. Bau. Function Vectors in Large Language Models. *ICLR*, 2024. arXiv:2310.15213.
 - [68] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, et al., and T. Henighan. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*, Anthropic, May 2024.
 - [69] N. Panickssery, N. Rimskey, M. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. M. Turner. Steering Llama 2 via Contrastive Activation Addition (CAA). *ACL*, 2024. arXiv:2312.06681.
 - [70] A. Arditi, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in Language Models is Mediated by a Single Direction. *NeurIPS*, 2024. arXiv:2406.11717.
 - [71] Y. Zhang, B. Tang, T. Ju, S. Duan, and G. Liu. Do Latent Tokens Think? A Causal and Adversarial Analysis of Chain-of-Continuous-Thought. arXiv:2512.21711, 2025.

A Condition Name Index

Table 31: Canonical external names used throughout this paper, with backbone, regime, phase count, and per-phase token budget. “ER” indicates the enhanced-routing Tier C variant.

External name	Backbone	Regime	Phases	Tok/phase
Main paper TFGN conditions				
TFGN_GPT2S_FS	GPT-2 Small (124 M)	FS	6	1 B
TFGN_GPT2M_FS	GPT-2 Medium (355 M)	FS	6	1 B
TFGN_GPT2M_RETROFIT	GPT-2 Medium (355 M)	RF	6	1 B
TFGN_LLAMA8B_FS	LLaMA 3.1 8B	FS	3	1 B
TFGN_LLAMA8B_RETROFIT	LLaMA 3.1 8B	RF	3	1 B
Main paper baselines				
BASELINE_STD_GPT2M_FS	GPT-2 Medium	FS	6	1 B
BASELINE_LORA256_GPT2M_FS	GPT-2 Medium	FS	6	1 B
BASELINE_STD_GPT2M_RETROFIT	GPT-2 Medium	RF	6	1 B
BASELINE_LORA256_GPT2M_RETROFIT	GPT-2 Medium	RF	6	1 B
BASELINE_STD_LLAMA8B	LLaMA 3.1 8B	FS	3	500 M
Extension A conditions (GPT-2 Small $\sim$398 M, Prose $\rightarrow$ Python $\rightarrow$ Math)				
TFGN_EXT_A_BASELINE	GPT-2 Small	FS	3	200 M
TFGN_EXT_A_SENSEACT	GPT-2 Small	FS	3	200 M
TFGN_EXT_A_FULL_DIAG	GPT-2 Small	FS	3	200 M
TFGN_EXT_A_CHAMPION	GPT-2 Small	FS	3	200 M
TFGN_EXT_B_BASELINE	GPT-2 Small	FS	3	1 B
TFGN_EXT_B_FULL_DIAG	GPT-2 Small	FS	3	1 B
TFGN_EXT_C_CONTROL	GPT-2 Small	FS	3	1 B (ER)
TFGN_EXT_C_DIAG	GPT-2 Small	FS	3	1 B (ER)
TFGN_EXT_C_ANCHOR	GPT-2 Small	FS	3	1 B (ER)
TFGN_EXT_C_HEADLINE	GPT-2 Small	FS	3	1 B (ER)
Extension B conditions				
TFGN_EXTB_GPT2S_HEADLINE	GPT-2 Small ($\sim$ 398 M total)	FS	—	—
TFGN_EXTB_GPT2M_HEADLINE	GPT-2 Medium ($\sim$ 739 M total)	RF	—	—

B BWT and FM Definitions

Backward Transfer (BWT, Lopez-Paz adapted to perplexity). Let $M[t, d]$ denote the held-out perplexity on domain d after phase t of training. For a T -phase continual sequence over D domains where the trained-domain order is $1 \rightarrow 2 \rightarrow \dots \rightarrow T$ (with $T \leq D$), BWT is defined as the average of per-domain relative degradations from the just-trained perplexity to the final-phase perplexity:

The relative-degradation form used here normalizes by the just-trained PPL (Lopez-Paz 2017 was originally defined on classification accuracy; for perplexity the natural relative form replaces the accuracy difference). $\text{BWT} = 0$ means no forgetting; $\text{BWT} < 0$ means forgetting; the scale is unbounded below (large negative magnitudes correspond to multi-fold relative degradation in PPL).

Forgetting Measure (FM). For each domain d trained at phase $\tau(d)$, the per-domain forgetting is the relative degradation from $M[\tau(d), d]$ (PPL just after training on d) to $\max_{t > \tau(d)} M[t, d]$ (the worst PPL observed on d in any later phase). FM is then the average of per-domain forgetting magnitudes across all trained domains. Where BWT averages over the final state, FM averages over the worst observed state, so FM is always $\geq |\text{BWT}|$ on a forgetting condition.

Per-domain bwt_d . The per-domain decomposition: $\text{bwt}_d = -(M[T, d] - M[\tau(d), d]) / M[\tau(d), d]$. The bottom row of every PPL matrix in this paper reports bwt_d for the trained domains; the BWT scalar is then the average across the trained domains.

HellaSwag protocol. HellaSwag accuracy is reported per phase using the standard zero-shot

likelihood-of-correct-completion protocol; no fine-tuning on HellaSwag is performed. Numbers are accuracies (in $[0, 1]$) on the validation split.

Gradient orthogonality definitions. For domains $i \neq j$, sample two batches of tokens (one from each domain) and compute the gradient of the loss with respect to the architecture’s continual-phase trainable parameters on each batch. The mean absolute cosine $|\bar{\cos}|_{i,j}$ is the average $|\cos(\nabla_i, \nabla_j)|$ over multiple resampled-batch pairs and over all evaluated phases and layers. The L2-orthogonal fraction reported in Section 5 is the geometric quantity $\sqrt{1 - |\bar{\cos}|^2}$ averaged across all unique cross-domain pairs and all evaluated phases/layers; it is the fraction of each gradient that lies outside the span of the other under the L2 norm.

Hyperparameter table. Hyperparameters across conditions (learning rate schedule, optimizer, batch size, gradient-clip threshold, weight-decay schedule, warmup schedule) are reserved under the same NDA-gated channel as the architectural mechanism, so that a verifying party can reproduce numbers at the same time as receiving the architectural details. The hyperparameter set itself is small (the standard set for transformer LM pretraining) and does not encode mechanism-relevant information; it is gated only because the verifying party should reproduce the numbers under the same NDA cover that grants access to the architecture.

C Full 6×6 Gradient Orthogonality Matrices

This appendix reports the per-domain-pair mean $|\cos|$ matrices for each TFGN main-paper condition. Each matrix is symmetric (by construction); only the lower triangle is populated; diagonal cells are 0 by definition (gradient of a domain with itself). The five matrices are presented at the same scale, with the same domain ordering, so that the cross-condition pattern is visible.

C.1 TFGN_GPT2S_FS (~398 M, From-Scratch)

Table 32: Mean $|\cos|$ between gradients of different domains for TFGN_GPT2S_FS. Mean $|\cos|$ overall = 0.0425; L2-orthogonal fraction = 99.91%; orth@0.1 = 85.56%; max $|\cos|$ = 0.418 (Python × JavaScript — legitimate syntactic overlap).

	Prose	Python	Math	Biomed	Chinese	JS
Prose	—					
Python	0.004	—				
Math	0.048	0.029	—			
biomed	0.027	0.005	0.009	—		
Chinese	0.000	0.019	0.000	0.001	—	
js	0.003	0.418	0.025	0.002	0.048	—

C.2 TFGN_GPT2M_FS (~739 M, From-Scratch)

Table 33: TFGN_GPT2M_FS (~739 M, From-Scratch). Mean $|\cos|$ overall = 0.0204; L2-orthogonal fraction = **99.94%** (paper-wide ceiling); orth@0.1 = 100.00%; max $|\cos|$ = 0.0798 (Math × Biomedical). Summary distributional statistics for this condition are available under NDA; the per-domain-pair 6 × 6 matrix is reserved under NDA along with the routing-state JSONs and the per-layer decompositions.

P25 $ \cos $	P50 $ \cos $	P75 $ \cos $	Max $ \cos $	n pairs	Notes
0.003	0.009	0.024	0.0798	30	all 30 pairs below the 0.1 threshold

Table 34: Mean $|\cos|$ between gradients of different domains for TFGN_GPT2M_RETROFIT. Mean $|\cos|$ overall = 0.0904; L2-orthogonal fraction = 99.59%; orth@0.1 = 41.4%; max $|\cos|$ = 0.486 (Python $\times$ JavaScript). The Python $\times$ JavaScript pair (0.486) is the largest off-diagonal in this condition; Biomedical $\times$ Prose (0.321) is the second-largest, reflecting English-Prose surface overlap.

	Prose	Python	Math	Biomed	Chinese	JS
Prose	—					
Python	0.003	—				
Math	0.116	0.091	—			
biomed	0.321	0.019	0.094	—		
Chinese	0.068	0.007	0.003	0.023	—	
js	0.008	0.486	0.082	0.013	0.024	—

C.3 TFGN_GPT2M_RETROFIT (~ 739 M, Retrofit)

C.4 TFGN_LLAMA8B_FS (~ 9 B, From-Scratch, 3-phase)

Table 35: Mean $|\cos|$ between gradients of different domains for TFGN_LLAMA8B_FS. Mean $|\cos|$ overall = 0.0432; L2-orthogonal fraction = 99.91%; orth@0.1 = 87.6%; max $|\cos|$ = 0.370 (Python $\times$ JavaScript). 3-phase trained sequence (Prose / Python / Math); Biomedical, Chinese, JavaScript columns are evaluation-only.

	Prose	Python	Math	Biomed	Chinese	JS
Prose	—					
Python	0.008	—				
Math	0.006	0.006	—			
biomed	0.020	0.009	0.015	—		
Chinese	0.008	0.027	0.039	0.014	—	
js	0.013	0.370	0.064	0.001	0.051	—

C.5 TFGN_LLAMA8B_RETROFIT (~ 9 B, Retrofit, 3-phase)

Table 36: Mean $|\cos|$ between gradients of different domains for TFGN_LLAMA8B_RETROFIT. Mean $|\cos|$ overall = 0.0741; L2-orthogonal fraction = 99.72%; orth@0.1 = 69.5%; max $|\cos|$ = **0.705** (Python $\times$ JavaScript outlier — legitimate syntactic overlap). 3-phase trained sequence; the 0.705 outlier is the only pair above 0.1, with 13 of 15 unique off-diagonal pairs at $|\cos| \leq 0.1$.

	Prose	Python	Math	Biomed	Chinese	JS
Prose	—					
Python	0.015	—				
Math	0.012	0.083	—			
biomed	0.031	0.036	0.003	—		
Chinese	0.003	0.029	0.017	0.008	—	
js	0.020	0.705	0.103	0.027	0.018	—

Note on values. The per-pair $|\cos|$ values above are transcribed directly from the underlying gradient-orthogonality data (referenced in the experimental-setup section). Cells show $|\cos|$ rounded to three decimal places. For TFGN_GPT2M_FS, distributional summary statistics are reported in lieu of a per-pair 6×6 matrix; the full matrix is reserved under NDA along with the per-layer decompositions and the routing-state JSONs.

D Full PPL Matrices with bwt_d Rows

The PPL matrices for the eleven primary conditions are presented in §5.2 and §5.3 of the main results section, where they are paired with their respective per-condition narratives. Cross-referencing them in this appendix would duplicate body content.

Pointers: TFGN_LLAMA8B_RETROFIT matrix in §5.2.1; TFGN_LLAMA8B_FS in §5.2.2; TFGN_GPT2M_FS in §5.2.3; TFGN_GPT2M_RETROFIT in §5.2.4; TFGN_GPT2S_FS in §5.2.5; the five matched baselines in §5.3.

Reading rule for every PPL matrix in this paper: rows are phase trained (P1, P2, ...), columns are evaluation domains. Cell $M[t, d]$ is the held-out perplexity on domain d after phase t . The bottom row reports per-domain bwt_d for the trained domains. The diagonal is the just-trained PPL; off-diagonal cells in the trained-domain columns are forgetting (or, for evaluation-only columns, out-of-distribution evaluation PPL not CL forgetting).

E Extension A Canonical Values and 11-Condition Matrix

The full per-condition Extension A BWT/FM table is in §6.4 (Table 19). The three-axis decomposition of the 81% headline reduction is in §6.3 (Table 18). The per-phase BWT decomposition for the three reference conditions is in Table 20.

Configuration matrix. Every Extension A condition pairs a routing substrate (basic vs enhanced) with a self-regulation stack subset (none / sensing+gating / internal-world-model-only / full-stack-diagnostic / full-stack-active). The full configuration assignment per condition is reserved; access terms are in §9.2. The numerical effect of the assignment on BWT is reported in Table 19.

Per-layer gradient orthogonality. All twelve layers of the GPT-2 Small backbone report gradient orthogonal-fraction > 0.995 on every Extension A condition — orthogonality is near-uniform across depth. The full per-layer decomposition is in the gradient-orthogonality JSON artifact reserved under NDA.

F Tier A 200 M Per-Condition PPL Matrices

The four publicly-disclosed Tier A 200 M-tokens-per-phase conditions are reported here for completeness. They run on Prose $\rightarrow$ Python $\rightarrow$ Math at GPT-2 Small (~ 398 M total) with the basic-routing main-paper substrate; they differ in the self-regulation stack subset enabled. The 1 B-token Tier B and Tier C matrices are in §6.5; the canonical BWT/FM scalars across all eleven Extension A conditions are in Table 19.

Table 37: TFGN_EXT_A_BASELINE (Tier A matched control, base-consolidation only) — per-domain PPL matrix and scalar metrics (3-phase Prose $\rightarrow$ Python $\rightarrow$ Math, 200 M tokens/phase). $\mathbf{BWT}_3 = -0.00942$.

Phase trained	Prose	Python	Math
P1 Prose	77.75	51.63	126.56
P2 Python	77.75	32.93	126.32
P3 Math	77.79	33.53	116.46
bwt_d	-0.00051	-0.01822	—

Metric	Value
BWT_3 / FM	-0.00942 / +0.00942
Per-domain bwt_d	Prose -0.00051; Python -0.01822
Other	Mean cos 0.0436; L2-orth 99.905%; HellaSwag P1 $\rightarrow$ P3: 0.268 $\rightarrow$ 0.274 (+2.2%, maintained)

Table 38: TFGN_EXT_A_SENSEACT (sensing + gating pair, minimal) — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 200 M tokens/phase). $\mathbf{BWT}_3 = -0.00528$.

	Phase trained	Prose	Python	Math
	P1 Prose	73.40	45.32	118.99
	P2 Python	73.40	31.86	118.69
	P3 Math	73.46	32.17	112.08
	bwt_d	-0.00082	-0.00973	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.00528 / +0.00528
Per-domain bwt_d	Prose -0.00082; Python -0.00973
Other	HellaSwag P1→P3: 0.268 → 0.264 (-1.5%, mildly degraded)

Table 39: TFGN_EXT_A_FULL_DIAG (full stack, diagnostic consolidation; critical differentiator) — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 200 M tokens/phase). $\mathbf{BWT}_3 = -0.01041$.

	Phase trained	Prose	Python	Math
	P1 Prose	68.87	39.66	107.95
	P2 Python	68.87	27.12	107.52
	P3 Math	68.88	27.67	100.24
	bwt_d	-0.00015	-0.02028	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.01041 / +0.01041
Per-domain bwt_d	Prose -0.00015; Python -0.02028
Other	HellaSwag P1→P3: 0.286 → 0.294 (+2.8%, maintained)

Table 40: TFGN_EXT_A_CHAMPION (**Tier A champion** — full self-regulation, active consolidation) — per-domain PPL matrix and scalar metrics (3-phase Prose → Python → Math, 200 M tokens/phase). $\mathbf{BWT}_3 = -0.00277$.

	Phase trained	Prose	Python	Math
	P1 Prose	69.06	37.92	106.49
	P2 Python	69.06	27.10	106.13
	P3 Math	69.07	27.24	101.99
	bwt_d	-0.00014	-0.00517	—

Metric	Value
$\mathbf{BWT}_3$ / FM	-0.00277 / +0.00277
Per-domain bwt_d	Prose -0.00014; Python -0.00517
Other	Mean cos 0.0526; L2-orth 99.737% ; HellaSwag P1→P3: 0.280 → 0.290 (+3.6%, improved)

G Definitions and Equations Index

This appendix is a one-stop index for every load-bearing symbol, metric, and equation referenced in the paper. Each entry has a first-use section reference.

Evaluation metrics (first-use [Appendix B](#))

- BWT (Lopez-Paz adapted to PPL) — average per-domain relative degradation from just-trained PPL to final-phase PPL.
- FM — average per-domain relative degradation from just-trained PPL to worst observed PPL across later phases.
- L2-orthogonal fraction — $\sqrt{1 - |\cos|^2}$ averaged across cross-domain pairs.
- HellaSwag accuracy — zero-shot likelihood-of-correct-completion on HellaSwag validation.

Equations referenced in this paper

Load-bearing numerical constants

- $\text{BWT}_3 = -0.007$ at LLaMA 3.1 8B Retrofit (tightest in paper) — [§5.2.1](#).
- L2-orthogonal floor = 99.59% (TFGN_GPT2M_RETROFIT) — [§5.6](#).
- L2-orthogonal ceiling = 99.94% (TFGN_GPT2M_FS) — [§5.2.3](#).
- Extension A 81% reduction (BWT -0.01140 vs anchor -0.06010) — [§6.3](#).
- Extension B 99.96% reshape (mean cosine 0.9996 over 30 pairs at ~ 398 M; 0.9995 at ~ 739 M) — [§7.5](#).
- Sub-task pair cosine ceiling = 0.87 (no Python or Math sub-task pair exceeds this) — (detailed reference reserved; see [§9.2](#)).